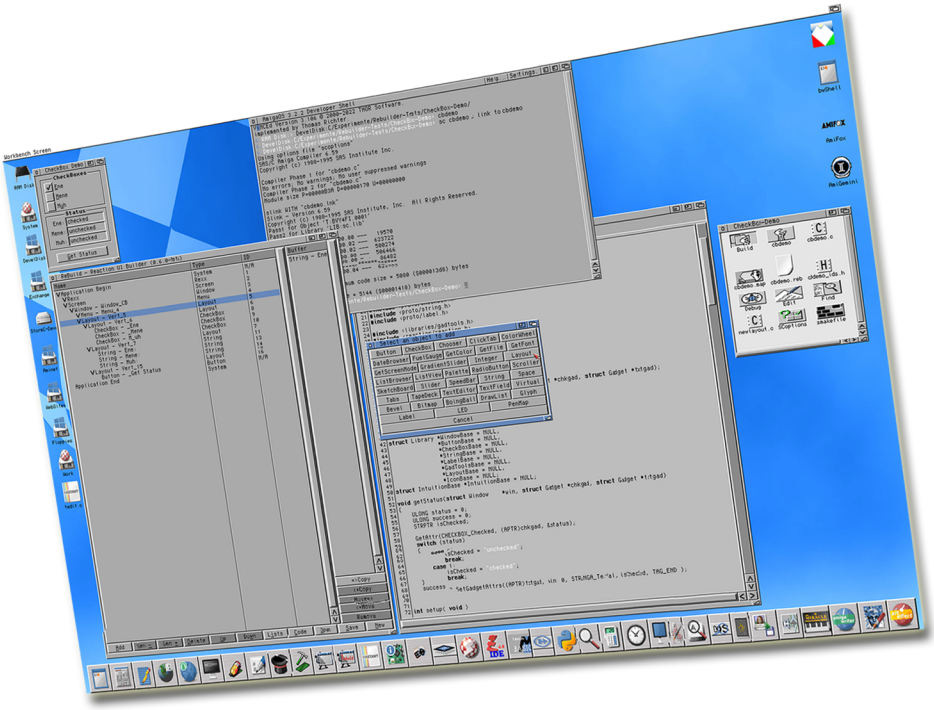
AMIGA Development Docs

ReBuild - the ReAction UI designer

a tutorial on its usage and enhancing code in C

Michael Bergmann / Darren Coles



The Art of building responsive GUI layouts for your Amiga

covers GUI creation for classic AmigaOS 3.2, 3.1 and up

Contents

Preamble 7

Credits 8

Requirements for this tutorial 9

Conventions on how to use this tutorial 10

I. Using ReBuild for GUI creation 11

1. ReBuild - understanding the GUI 12

1.1. Main window overview . . . 12

1.1.1. The layout elements area . . . 12

1.1.2. The buffer area . . . 13

1.1.3. The actions area . . . 13

1.1.4. ToolTypes - setting personal Preferences . . . 14

LANG . . . 14

USEIDS . . . 14

FULLCODE . . . 14

SAVEPATH . . . 14

CODEPATH . . . 14

SHOWBUFFER . . . 15

WARNONDELETE . . . 15

SHOWSETTINGS . . . 15

PROJECTICON . . . 15

WINDOWTOP . . . 15

WINDOWLEFT . . . 15

WINDOWWIDTH . . . 15

WINDOWHEIGHT . . . 15

1.2. Designing a GUI . . . 16

CheckBox Demo - Assumptions on creating the layout . . . 17

CheckBox Demo - a potential layout solution . . . 17

1.2.1. Editing Window properties . . . 18

Adding more window flags . . . 20

Adding IDCMP flags . . . 22

1.2.2. Adding Menus . . . 26

1.2.3. Adding Layouts . . . 27

Creating the project's basic layout . . . 30

1.2.4. Adding Gadgets . . . 31

II. Modifying and enhancing ReBuild-generated code 34

2. Overview - ReBuild-generated functions 35

2.1. Creating a demo app to work with . . . 35
2.1.1. The Window . . . 35
2.1.2. Menus . . . 36
2.1.3. Layout . . . 36
2.1.4. Code generation . . . 37
2.1.5. Code compilation . . . 38
2.2. ReBuild's exported code functions . . . 38
2.2.1. int setup(void) . . . 38
2.2.2. void cleanup(void) . . . 38
2.2.3. void runWindow(...) . . . 39
The function's IDCMP event loop . . . 41
2.2.4. void window_3(void) . . . 41
int main(...) . . . 43

3. Enhancing functionality 44

3.1. Reacting on Gadgets - event loop part 1 . . . 44
3.2. Reacting on Menus - event loop part 2 . . . 46
3.2.1. Menus in theory . . . 46
3.2.2. Enhancing the project's menu . . . 47
3.2.3. Making the project's Menu work . . . 49
3.2.4. Getting the status of checkable Menu Items . . . 51
3.3. Implementing ReAction Requesters . . . 53
3.4. Implementing ToolTips . . . 55
3.5. Querying the status of a CheckBox . . . 56
3.5.1. Implementing CheckBox status query . . . 57
3.6. Exercises . . . 60

III. Appendices 61

A. C compiler setup 62

A.1. Setting up a C compiler for NDK 3.2 . . . 63
A.1.1. SAS/C setup . . . 63
Installing SAS/C . . . 63
Backing up SAS/C include: and lib: folders . . . 63
Deleting unneeded files . . . 63
Replacing files . . . 64
Testing the installation . . . 64
A.1.2. vbcc setup . . . 65
Installing vbcc . . . 65
Editing S:User-Startup . . . 65
Testing the installation . . . 66
A.2. Sources notice . . . 66

B. Recommended readings 67

B.1. The AmigaOS 3.2 Native Developer Kit . . . 68

B.2. Amiga ROM Kernel Reference Manual: Libraries . . . 68
B.3. Amiga ROM Kernel Reference Manual: Devices . . . 68
B.4. ROM Kernel Reference Manual: Changes & Additions . . . 68
B.5. Amiga User Interface Style Guide . . . 68
B.6. The AmigaDOS Manual . . . 68
B.7. The Amiga Guru Book . . . 69
B.8. Sources information . . . 69

C. Readings on C coding **70**
C.1. General books about C . . . 71
C.1.1. The C Programming Language - ANSI C . . . 71
C.1.2. The C Answer Book . . . 71
C.1.3. Algorithms in C . . . 71
C.2. Amiga-related C readings . . . 71
C.2.1. Amiga C for Beginners . . . 71
C.2.2. Inside the Amiga with C . . . 71
C.2.3. Inside Amiga Graphics . . . 71

D. Readings on E coding **72**
D.1. A Beginner's Guide to Amiga E . . . 73
D.2. Amiga E manual v3.3a . . . 73

E. BOOPSI ReAction classes, gadgets and images **74**
E.1. Classes . . . 75
E.1.1. arexx.class . . . 75
E.1.2. requester.class . . . 75
E.1.3. window.class . . . 75
E.2. Gadgets . . . 75
E.2.1. button.gadget . . . 76
E.2.2. checkbox.gadget . . . 76
E.2.3. chooser.gadget . . . 76
E.2.4. clicktab.gadget . . . 76
E.2.5. colorwheel.gadget . . . 76
E.2.6. datebrowser.gadget . . . 77
E.2.7. fuelgauge.gadget . . . 77
E.2.8. getcolor.gadget . . . 77
E.2.9. getfile.gadget . . . 77
E.2.10. getfont.gadget . . . 77
E.2.11. getscreenmode.gadget . . . 78
E.2.12. gradientslider.gadget . . . 78
E.2.13. integer.gadget . . . 78
E.2.14. layout.gadget . . . 78
E.2.15. listbrowser.gadget . . . 78
E.2.16. listview.gadget . . . 79
E.2.17. palette.gadget . . . 79
E.2.18. radiobutton.gadget . . . 79
E.2.19. scroller.gadget . . . 79
E.2.20. sketchboard.gadget . . . 79
E.2.21. slider.gadget . . . 80

E.2.22. space.gadget . . . 80
E.2.23. speedbar.gadget . . . 80
E.2.24. string.gadget . . . 80
E.2.25. tabs.gadget . . . 80
E.2.26. tapedeck.gadget . . . 81
E.2.27. texteditor.gadget . . . 81
E.2.28. virtual.gadget . . . 81
E.3. Images . . . 81
E.3.1. bevel.image . . . 81
E.3.2. bitmap.image . . . 82
E.3.3. boingball.image . . . 82
E.3.4. drawlist.image . . . 82
E.3.5. glyph.image . . . 82
E.3.6. label.image . . . 82
E.3.7. led.image . . . 83
E.3.8. penmap.image . . . 83

F. Useful links **84**
F.1. AmigaOS Developer CD v2.1 . . . 85
F.2. AmigaOS Developer CD v2.1 online . . . 85
F.3. ReBuild repository . . . 85
F.4. Codecraft IDE . . . 85
F.5. Recent vbcc for Amiga . . . 85
F.6. Bebbo's amiga-gcc Cross Compiler . . . 85
F.7. Micha B.'s AmigaED . . . 85
F.8. E-VO compiler . . . 85

G. Useful third-party functions **86**
G.1. getCheckBoxStatus(...) . . . 87
G.2. fileExists(...) . . . 88
G.2.1. fileExists(...) example snippet . . . 88
G.3. getProgDir() . . . 89
G.4. doFileRequest(...) . . . 89
G.4.1. doFileRequest(...) example snippet . . . 91
G.5. doDirRequest() . . . 92
doDirRequest(...) example snippet . . . 93
G.6. doSaveRequest(...) . . . 94
G.6.1. doSaveRequest(...) example snippet . . . 95
G.7. doInfoRequest(...) . . . 96
G.7.1. doInfoRequest(...) example snippet . . . 98
G.8. identifyCompiler() . . . 98
G.8.1. identifyCompiler(...) example snippet . . . 99

H. Full Example Listings **100**
H.1. Full Listing - CheckBox-Demo . . . 101
H.2. Full Listing - myDemo . . . 111

Preamble

ReBuild – Amiga Reaction GUI Builder

© 2023 by Darren Coles

is an Amiga GUI development tool that allows you to easily create style guide conformant graphical user interfaces based on BOOPSI/ReAction or ClassAct. Darren created it in Amiga E, using his own E-VO compiler system. It is free to use for everybody.

Since AmigaOS 3.2, *ReAction* is the new system standard for building graphical user interface on classic Amiga computer systems. Consisting of nearly forty BOOPSI classes, gadgets and images, following a strictly hierarchical concept for building GUI by using an object-oriented paradigm, ReAction is the most powerful GUI toolkit ever seen on an Amiga, besides MUI. With the power of ReAction comes the burden of developing a GUI is often rather time-consuming and a case of trial-and-error. This is where *ReBuild* comes in.

With ReBuild, the tasks of creating screens, windows, menus and even complex layouts with gadgets the BOOPSI way become an ease. Using ReBuild's preview window, you always have full visual control on your layout. Rebuild optionally produces source code for the C and E programming languages.

This tutorial is about developing ReBuild-generated GUI in *C*. It was mainly written for programmers with at least a mediocre understanding for coding Amiga applications using the C programming language. If unsure about these requirements, please consult the additional reading references mentioned in appendices B and G. ReBuild-generated C code will happily compile with the *SAS/C v6.58* and *vbcc* C-compiler systems, vbcc starting with v0.9h (as long as no incompatible functions are used). We will use **SAS/C** for this tutorial. In order to set up both C-compiler systems for usage with AmigaOS NDK3.2, please refer to appendices sections A.1.1 and A.1.2.

Some words about language constructs: The author is *not* a native English speaker, so please forgive me if something sounds *"genglish"!* In case of semantic non-understanding, feel free to contact me at *mbergmann-sh@gmx.de*

Credits

Kudos go out to:
Darren Coles, developer of the ReBuild Amiga ReAction GUI Builder
Camilla Boemann and ***Ralf Hasemann***, main developers of the Amiga Codecraft IDE
The AmigaOS 3.2 Dev Team, providing us with such a great OS

...and last, but not least to all those people that keep the Amiga platform alive by developing new applications or using it for fun and entertainment – just like back in those glorious old days!

Special thanks hit my very dear friend ***Jutta Pistor***, who not only saved my butt once in this life, but was also not too shy to counter-read this tutorial for faulty language constructs, although she knows next to nothing about C programming and the Amiga.

Michael Bergmann, December 2023

Requirements for this tutorial

In order to benefit the most from this tutorial, the following requirements are given:

- a fundamental understanding on how to work with Amiga Workbench and Shell
- some sort of a working installation of AmigaOS 3.1 up to AmigaOS 3.2, either emulated or a real machine
- the following software installed on that system:
  - **ReBuild**, naturally...
  - either one of the **SAS/C** or **vbcc** C-compiler environments, set up for using them with AmigaOS NDK 3.2 or ClassAct
- a working knowledge about the C programming language
- at least *some* knowledge on working with Amiga data structures

Since we will concentrate on AmigaOS 3.2 development here, you should be able to adapt some code if you are working with ClassAct!

Conventions on how to use this tutorial

This tutorial splits into two main topics:

1. Using ReBuild for GUI creation
2. Enhancing the code

While the first topic deals with learning ReBuild's GUI while developing a demo project step by step, the second topic concentrates on how to modify and enhance ReBuild's C code output in order to build a window-based working application with gadgets and hand-woven menus. When generating code, we'll always use "use ids for GA_ID" and "generate full code" options. I would suggest that, if new to ReBuild, you should work through topic one at least.

Important tips and sources appear in framed boxes. Example:

> ***Tip:*** For getting hold on Rudi, the red-nosed reindeer, always carry some carrots with you. Please revisit http://amiga.mbergmann-sh.de for latest tips on Rudi.

For explanation purpose, C code examples often appear as line numbered listings. Example:

```
1   #include <dos/dos.h>
2   #include <proto/dos.h>
3   int main()
4   {
5     Printf("Hello, World!\n");
6     return(RETURN_OK);
7   }
```

Obviously, you're *not* expected to enter those line numbers when typing in an example.

In this tutorial, **SAS/C v6.58** is used as the standard C-compiler – but **vbcc** should work as well. If you run in trouble using *vbcc* or any other C-compiler, you are on your own.

...and now for something completely different: Good luck & have fun with this tutorial!

Part I

Using ReBuild for GUI creation

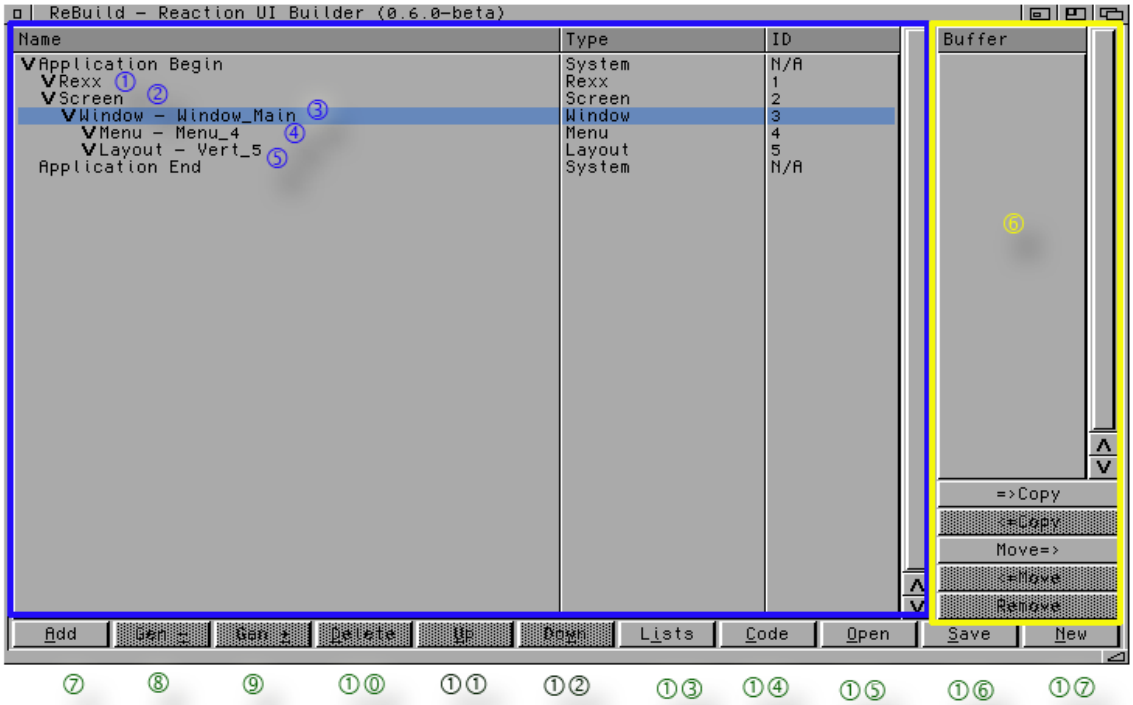
1. ReBuild - understanding the GUI

ReBuild, as our weapon of choice for creating modern GUI application skeleton, is a rather complex, yet easy-to-use tool for rapid software development.

Before we start, let's assume that in order to work through this tutorial, you have already created a drawer named "ReBuild-Tutorial" somewhere on your hard drive. Let's also assume that in that drawer, you will create another unique drawer for each project that we discuss and safe your project and source files there.

1.1. Main window overview

In order to get the most benefit out of this program, you will need to understand its elements and functionality. Let's start with knowing the GUI:



ReBuild's main window divides itself up into three areas:

1. the layout elements area (blue frame)
2. the buffer area (yellow frame)
3. the actions area (no frame)

1.1.1. The layout elements area

This is the area where you add, edit or remove screens, windows, layouts and gadgets, organized in a ListBrowser gadget. Double clicking one of them pops up the corresponding properties dialog, where you can edit different

parameters.

The **Rexx** entry (**1**) allows you to define an ARexx port for your application.
The **Screen** entry (**2**) lets you define and parameterize an own screen for your application.
The **Window** entry (**3**) allows you to edit the current window's properties.
The **Menu** entry (**4**) brings up the menu editor dialog, where you can add, edit or delete menus, menu items and sub-items for the current window. As you can see, the Menu entry is indented against the Window entry – and so is the Layout entry. This mirrors the hierarchical structure of layouting a GUI window. If you add more windows, each of them will have their own Menu and Layout entries. The Window entry itself is, by the way, inferior to the Screen entry.
The **Layout** entry (**5**) lets you define and nest layouts according to your needs. These layouts will then hold your gadgets. A new project comes with one vertical layout by default. If you double click it, its properties dialog pops up.

We will talk about the different properties dialogs for each entry more detailed when those are explained.

1.1.2. The buffer area

The **buffer** area (**6**) is good for keeping commonly used gadgets and layouts. Its buttons allow you to copy or move a selected entry from the layout elements area to the buffer - with some restrictions: Windows and Arexx entries can't be copied to the buffer.

A copied entry may be inserted into the selected window or layout of the layout elements area by using the appropriate button in the buffer area. Finally, the buffer can be emptied by removing buffered elements one by one.

You can make this area appear or disappear by selecting/deselecting "Show Buffer" from the Edit menu.

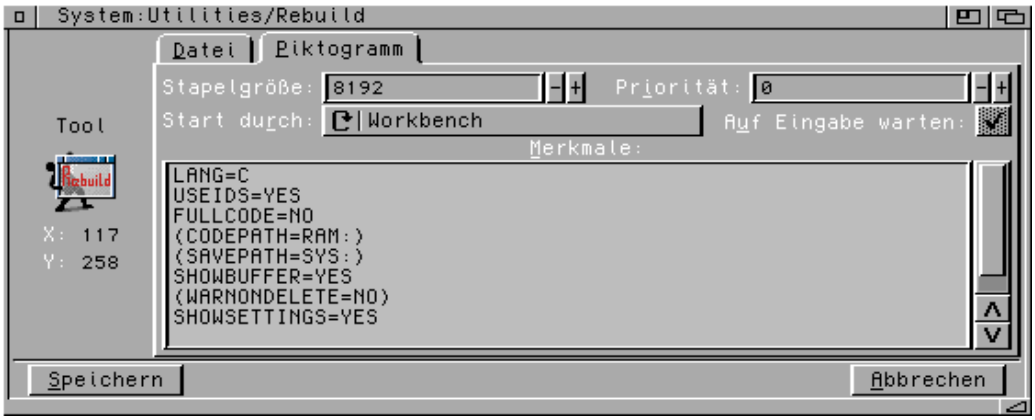
1.1.3. The actions area

The actions area on the main window's bottom holds the buttons *7* to *17*. Its purpose is to manage tasks like layout- and gadget creation, sorting and deleting – but also to save your project and generate compileable code for E or C.

The **Add** button (**7**) reacts depending on the selected entry of the layout elements area. If *Screen* or *Window* is selected, it pops up the window settings dialog, creating a new window. If *Layout* is selected, it shows the add-objects dialog.
The **Gen -** (**8**) and **Gen +** (**9**) buttons are only activated if a gadget is selected. *Gen -* puts an object hierarchically up, *Gen +* does the opposite.
The **Delete** button (**10**) does what it says: It deletes an object (window – if not the only one, layout or gadget) from the main layout. The main layout itself cannot be deleted. In order to activate the *Delete* button, a window, layout or gadget must be selected.
The **Up** (**11**) and **Down** (**12**) buttons become active, when an object is selected in the layout elements area. Their task is to sort entries within the given hierarchy.
The **Lists** button (**13**) lets you create named lists that contain entries for chooser gadget, ListView gadgets and that alike. When pressed, the Lists editor pops up.
The **Code** button (**14**) pops up the Code generator dialog, where you can choose the programming language to produce code for and ways of building up your code skeleton.
The **Open** button (**15**) opens a formerly saved Rebuild project file.
The **Save** button (**16**) saves your project to a file, allowing you to name it as you like. The project file is then saved with the suffix *.reb*.

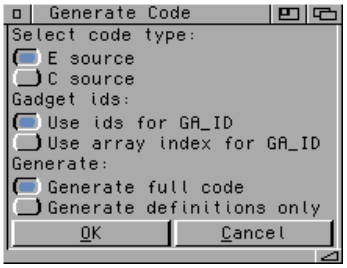
The **New** button (**17**) abandons the recent project and sets up a new one. If the recent project has been edited, you will be asked if you want to save it before a new project is created.

1.1.4. ToolTypes - setting personal Preferences



Depending on if you are a C or E programmer, you might want to customize ReBuild to start with preferences according to your personal needs. You can overwrite ReBuild's default behavior by setting *ToolTypes* in the appliation's icon:

LANG



sets the default programming language for code generation. Accepted values: *C* or *E*. Defaults to **E**.
You can always change the programming language in the "generate code" dialog.
Hint: In this tutorial, we assume *LANG=C*.

USEIDS

determines whether the generated code shall use Gadget IDs or an array index for identifying GA_ID. Accepted values: *YES* or *NO*. Defaults to **YES**. You can always change this behavior in the "generate code" dialog.
Hint: In this tutorial, we assume *USEIDS=YES*.

FULLCODE

determines whether the code shall be generated as full code *(compilable)* or defintions only *(non-compilable)*. Accepted values: *YES* or *NO*. Defaults to **YES**. You can always change this behavior in the "generate code" dialog.
Hint: In this tutorial, we assume *FULLCODE=YES*.

SAVEPATH

sets the path for saving your projects. Accepted values: *a valid, full-qualified path*. Defaults to **RAM:**

CODEPATH

sets the path for saving code files. Accepted values: *a valid, full-qualified path*. Defaults to **SYS:**

SHOWBUFFER

Shows or hides the buffer area in the main window. Accepted values: *YES* or *NO*. Defaults to **YES**. You can always make the buffer area appear or disappear by selecting/deselecting "Show Buffer" from the Edit menu.

WARNONDELETE

Manages warnings on object deletion. Accepted values: *YES* or *NO*. Defaults to **YES**. You can always change this behavior by selecting/deselecting "Warn on delete" from the Edit menu.

SHOWSETTINGS

Regulates whether the appropriate settings dialog is shown or not if an object is added. Accepted values: *YES* or *NO*. Defaults to **YES**. You can always change this behavior by selecting/deselecting "Show settings" from the Edit menu.

PROJECTICON

Regulates whether or not an icon is saved with your project file. Accepted values: *YES* or *NO*. Defaults to **YES**. You can always change this behavior by selecting/deselecting "Save Project icon" from the Edit menu.

WINDOWTOP

Customize ReBuild's main window start position: WINDOWTOP=<pixel>

WINDOWLEFT

Customize ReBuild's main window start position: WINDOWLEFT=<pixel>

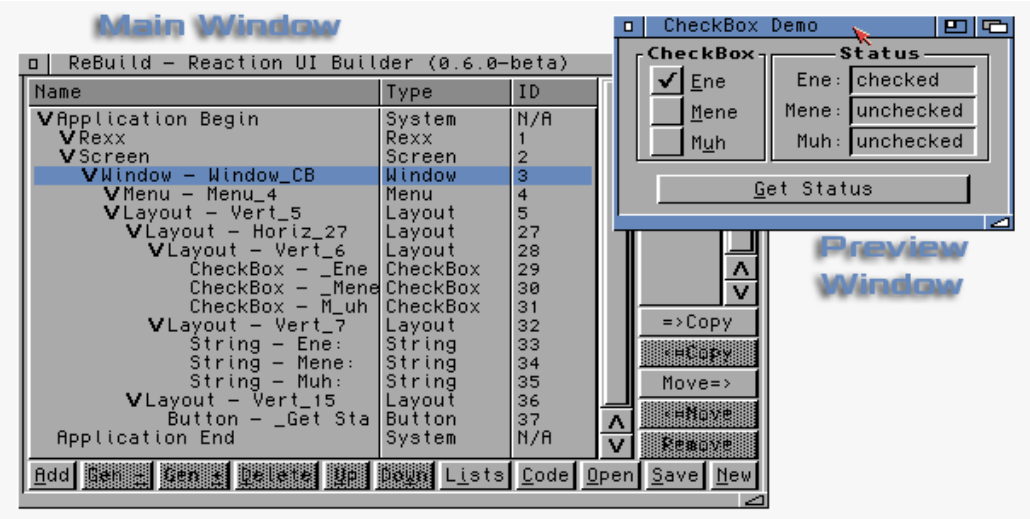
WINDOWWIDTH

Customize ReBuild's main window start position: WINDOWWIDTH=<pixel>

WINDOWHEIGHT

Customize ReBuild's main window start position: WINDOWHEIGHT=<pixel>

1.2. Designing a GUI



Designing a GUI with ReBuild allways follows these basic steps:

1. Thinking about which gadgets and menus are required – and how a certain look can be achieved by using vertically and horizontally orientated layouts. It's not the worst idea to draw your draft on a sheet of paper first!

2. creating a drawer somewhere on your hard drive in order to keep your project organized.

3. Firing up ReBuild, then saving your empty project to that drawer. *Now the real work starts:*

4. Editing the windows properties

5. eventually design menus and create lists, f.e. for Chooser or ListView gadgets.

6. adding nested layouts to your main layout, according to your draw.

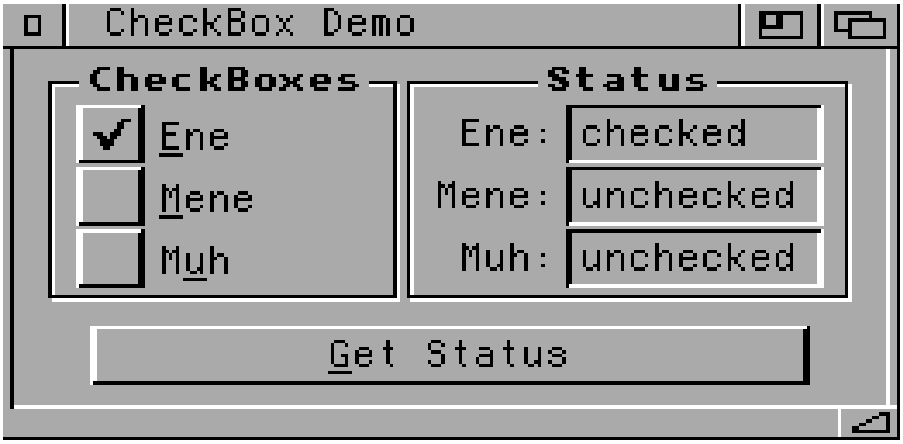
7. Placing gadgets into them layouts, deciding if they are placed suitable. You can always control the look of your GUI design within the Preview Window. If that didn't open automatically, you can bring it up by using the "Preview Windows" sub entry from the "Edit" menu.

8. When satisfied with your design, saving it by using either the "Project" menu entries or the "Save" button in the actions area.

9. generating and saving source code by using the "Project" menu's "Generate Code" entry or the "Code" button.

   Hint: Always have in mind that your GUI design must be *really* finished before editing the generated source code! All manually added changes to your source code will be *overwritten* when you change your design and afterwards let ReBuild generate code for you again.

Now let's start to design a GUI that, besides having a simple menu, holds three CheckBox gadgets, three read-only String gadgets to mirror the status of the CheckBoxes and a Button gadget that starts an action! We want to have the group of checkBoxes and the group of Strings placed *horizontally* in-parallel, while the Button shall be placed below them. Also, the CheckBox and String groups shall both have some sort of Bevel around them in order to mirror that they belong together. Have a look at the picture in order to get an idea about our layout:



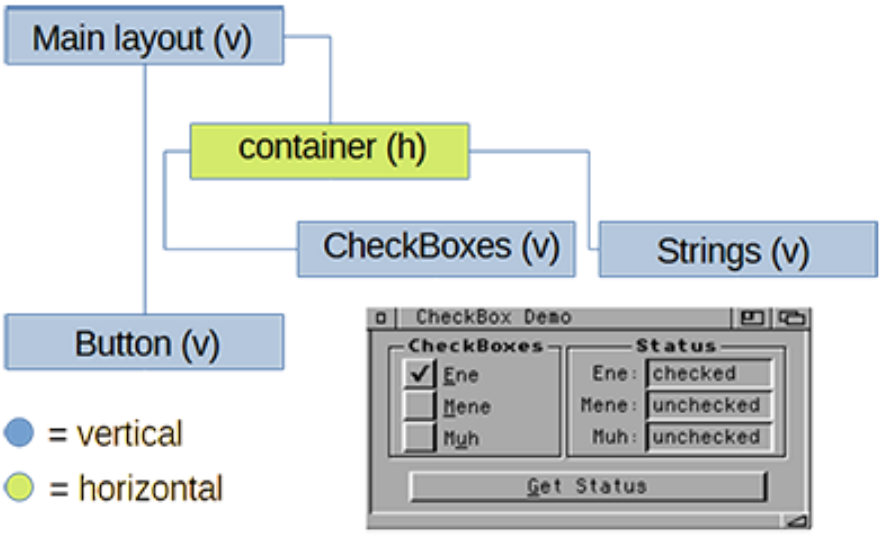
CheckBox Demo - Assumptions on creating the layout

- both our CheckBox and String groups have their objects *vertically* sorted – so we will put both groups into their own vertical layout.
- the CheckBoxes and String groups should appear side by side *horizontally* – so their layouts both must be a child of a horizontal group.
- our Button should appear *below* the horizontally layouted CheckBox and String groups. Obviously it needs to be placed in a vertical layout. Since ReBuild's main layout is by default of that type, it might work to place it there?

  Since we want to have the Button's look a bit indented and centered, it however appears to be somewhat more smart to put it into its own vertical layout and fine-tune that layout's *spacing* settings, so our Button looks the way we want it. More about the topic of fine-tuning a layout will be discussed later on...
- if we want our Button in its own vertical layout, that must be a child of the window's vertical main layout.

CheckBox Demo - a potential layout solution

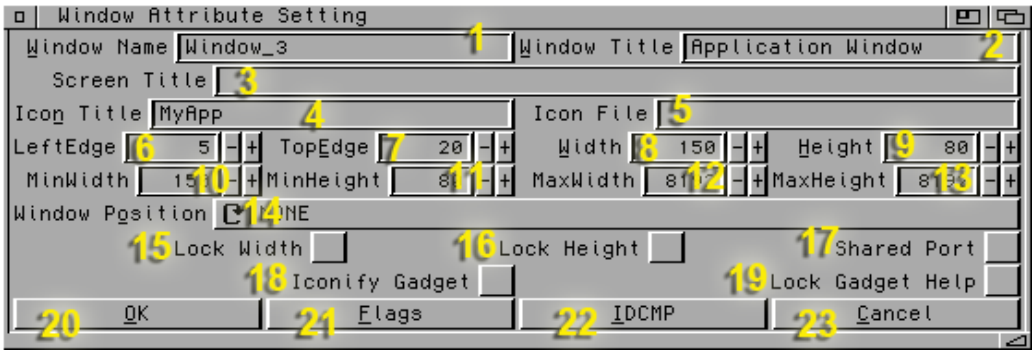
Regarding all those previously made assumptions, we will come to the following layout hierarchy:



We'll keep that in mind when it comes to adding layouts and gadgets. At least now it would be about time to create a folder for our project – name it "cbdemo" – and fire up ReBuild. We will immediately start to develop our first project! Save the project file into your "cbdemo" folder.

1.2.1. Editing Window properties

Every GUI application needs at least one window to place layouts and gadgets in. So let us define our main window's properties first: Double click the *Window* entry in the layout elements area – by default, it says: "Window - Window_3" – now the *Window Attribute Setting* dialog will pop up:



This dialog determines the appearance and behavior of your window. Here is the meaning of the dialog's parameters in detail:

1. *Window Name*: determines the identifying name a window is given – therefore, it must consists of a one-word-string. It is also used by ReBuild's code generator to name a function for setting up and opening the window. Notice that this will be a lower case function name in code, regardless of what you type in.

   For our project, set the window name to "Window_CB" now.

2. *Window Title:* This holds the Text that is shown in the window's drag bar

   For our project, set the window title to "CheckBox Demo" now. (equivalents to the WA_Title, "CheckBox Demo" Tag).

3. *Screen Title:* Same as "Window Title", but for setting the Screen title.

   For our project, set the screen title to "CheckBox Demo v1.0" now. (equivalents to the WA_ScreenTitle, "CheckBox Demo v1.0" Tag)

4. - *Icon Title:* Sets the name to be displayed under the app's icon on Worbench, if the window is iconified (Note that in order for a window to be iconified/uniconified "Iconify Gadget" *(18)* must be checked).

   For our project, set the icon title to "cbDemo" now. Anyway, we won't really use it. (equivalents to the WINDOW_IconTitle, "cbDemo" Tag)

5. *Icon File:* Determines the icon to be displayed on Worbench, if the window is iconified (Note that in order for a window to be iconified/uniconified "Iconify Gadget" *(18)* must be checked). If no icon file name is given, a default icon will be displayed on iconification.

   For our project, leave icon file blank. We don't use iconification in this app. (equivalents to the WINDOW_Icon, GetDiskObject("myIcon") Tags, if given)

6. *LeftEdge:* Sets the left edge coordinate for placing our window on Worbench. Note, that if "Window Position" *(14)* is set to anything else but *NONE*, this entry will have no effect.

   For our project, leave the value as is. (equivalents to the WA_Left, 5 Tags)

7. *TopEdge:* Sets the top edge coordinate for placing our window on Worbench. Note, that if "Window Position" *(14)* is set to anything else but *NONE*, this entry will have no effect.

   For our project, leave the value as is. (equivalents to the WA_Top, 20 Tags)

8. *Width:* Sets the nominal width of our window.

   For our project, set the value to 165. (equivalents to the WA_Width, 165 Tags). If opened, the window will have a width of 165 pixels or more, since ReAction takes care of the window's width by calculating the space needed for its layouts and gadgets!

9. *Height:* Sets the nominal height of our window.

   For our project, set the value to 20. (equivalents to the WA_Height, 20 Tags). If opened,the window will have a height of 20 pixels or more, since ReAction takes care of the window's height by calculating the space needed for its layouts and gadgets!

10. *MinWidth:* Sets the minimal width of our window. In consequence, you can't resize its width to a value smaller than given here.

    For our project, set the value to 120. (equivalents to the WA_MinWidth, 120 Tags)

11. *MinHeight:* Sets the minimal height of our window. In consequence, you can't resize its height to a value smaller than given here.

    For our project, set the value to 20. (equivalents to the WA_MinHeight, 20 Tags)

12. *MaxWidth:* Sets the maximal width of our window. In consequence, you can't resize its width to a value bigger than given here.

    For our project, leave the value as is. (equivalents to the WA_MaxWidth, 8192 Tags)

13. *MaxHeight:* Sets the maximal height of our window. In consequence, you can't resize its width to a value bigger than given here.

    For our project, set the value to 20. (equivalents to the WA_MaxHeight, 20 Tags)

14. *Window Position:* Sets predefined default values for positioning our window on the Screen. As a result, if you select anything else than *NONE*, the values for Tags WA_Left and WA_Top will be ignored. Possible values are:

    - *NONE* – leaves window positioning in the responsibility of WA_LEFT and WA_TOP values
    - *WPOS_CENTERSCREEN* – centers the winow on the screen
    - *WPOS_CENTERMOUSE* – centers the window at the recent mouse position
    - *WPOS_TOPLEFT* – places the window in the upper left corner, right under the Workbench title bar
    - *WPOS_CENTERWINDOW* – centers the window within the coordinates of a parenting window. This option only makes sense if you use more than one window in your application. If no parent window exists, this Tag behaves like if you would have choosen WPOS_CENTERSCREEN
    - *WPOS_FULLSCREEN* – opens the window in fullscreen mode

    For our project, select WPOS_TOPLEFT. (equivalents to the WINDOW_Position, WPOS_TOPLEFT Tags)

15. *Lock Width:* prohibits resizing in horizontal direction, useful in conjunction with WA_MinWidth and WA_Width having identical values

    For our project, leave this option unchecked. (equivalents to the WINDOW_LockWidth, FALSE Tags)

16. *Lock Height:* prohibits resizing in vertical direction, useful in conjunction with WA_MinHeight and WA_Height having identical values

    For our project, check this option. We want our GUI to be resizeable in horizontal direction only. (equivalents to the WINDOW_LockHeight, TRUE Tags)

17. *Shared Port:* bypasses IDCMP message passing. Sharing UserPorts means that the window is opened without any IDCMP messages enabled, and only later is ModifyIDCMP() called to turn on message passing. If the newly opened window becomes active before ModifyIDCMP() is called, the IDCMP_ACTIVEWINDOW message will not be received (because IDCMP message passing was off at the time)
    If that option was checked, the user would be responsible for implementing clean message passing!

    If checked, equivalents to the WINDOW_SharedPort, gSharedPort Tags. ***For our project, leave this option unchecked anyway.***

18. *Iconify Gadget:* adds an iconify gadget to the window's drag bar, allowing the user to iconify/uniconify the window. This option only makes sense for an application's main window.

    For our project, leave this option unchecked. (equivalents to the WINDOW_IconifyGadget, FALSE Tags)

19. *Lock Gadget Help:* if checked, enables the window to display gadget help texts. Currently, those must be implemented by the user.

    For our project, leave this option unchecked. (equivalents to the WINDOW_GadgetHelp, FALSE Tags)

20. *OK:* accepts the changed attributes and closes the dialog.

21. *Flags:* opens the "Window Flags Attribute Setting" dialog, where you can define further attributes that affect your window's behavior.

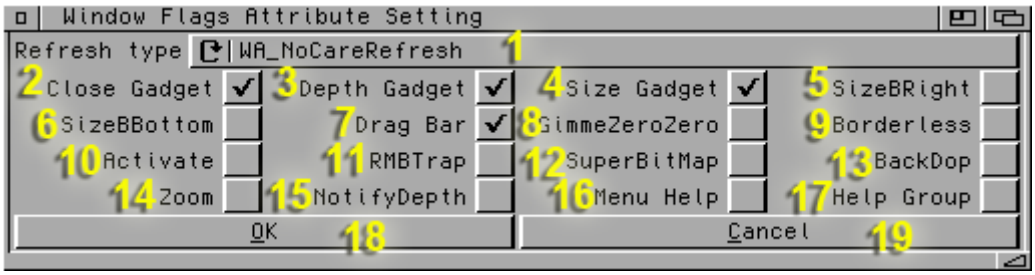
22. *IDCMP:* opens the "Window IDCMP Attribute Setting" dialog, where you can choose the IDCMP message flags that shall be available in your application.

23. *Cancel:* abandons all of your changes and closes the dialog.

We still need to do some more changes that affect our window's look. Press the "Flags" button now to open the "Window Flags Attribute Setting" dialog...

Adding more window flags

As said before, the "Window Flags Attribute Setting" dialog contains useful flags that affect the look of our window. They are:



1. *Refresh type:* this flag regulates how the window will be refreshed. Possible values are:

- *WA_NoCareRefresh* (default): If your application never needs to render in response to a refresh event, it can avoid having to call BeginRefresh()/EndRefresh() by setting the WFLG_NOCAREREFRESH flag or the WA_NoCareRefresh tag in the OpenWindowTagList() call – that's exactly what this option does for you!

  For our project, leave this as default. (equivalents to the WA_NoCareRefresh, TRUE Tag)

- *WA_SimpleRefresh*: For a Simple Refresh window, only those pixels actually on-screen are maintained by the system. When part of a Simple Refresh window is obscured, the imagery that was there is lost. When part of the window is newly revealed (either because the window was just made larger, or because that part used to be obscured by another window), the application must refresh any rendering it wishes to appear into that part. The application will learn that refresh is needed because Intuition sends an IDCMP_REFRESHWINDOW event. In consequence, you'll have to check that event in your event loop, reacting according to it!
- *WA_SmartRefresh*: The system will not only preserve those pixels which are actually on-screen, but it will save all obscured pixels that are within the current window's size. The system will refresh those parts of the window revealed by changes in the overlapping with other windows on the screen, without involving the application. Any part of the window revealed through the sizing of the window *must* be redrawn by the application. Intuition will notify the application through the IDCMP_REFRESHWINDOW event. In consequence, you'll have to check that event in your event loop, reacting according to it!
- *WA_SuperBitMap*: The SuperBitMap refresh type allows the application to provide and maintain bitmap memory for graphics in the window. The bitmap can be any size as long as the window sizing limits respect the maximum size of the bitmap. SuperBitMap windows are almost always WFLG_GIMMEZEROZERO, which renders the borders and system gadgets in a separate bitmap. If you want to use this type of window refresh, it will be a good idea to set *GimmeZeroZero* in the "Window IDCMP Attribute Setting" dialog.

2. *Close Gadget:* if checked, the window will have a close gadget (default)
3. *Depth Gadget:* if checked, the window will have a depth gadget (default)
4. *Size Gadget:* if checked, the window will have a size gadget (default)
5. *SizeBRight:* if checked, the window will have a slightly bigger right frame
6. *SizeBBottom:* if checked, the window will have a slightly bigger bottom frame
7. *Drag Bar:* if checked, the window will have a drag bar (default)
8. *GimmeZeroZero:* if checked, renders the borders and system gadgets in a separate bitmap. This flag is commonly used in a context where graphics are drawn into a window in order to avoid drawing into its borders.
9. *Borderless:* if checked, the window will appear borderless
10. *Activate:* if checked, the window immediately will be active when opened. Use this flag carefully, as it can change where the user's input is going – you might need to activate f.e a string gadget after the window opens, if that should be the the place where you want the user to start with.

    For our project, check this flag. (equivalents to the WA_Activate, TRUE Tag)

11. *RMBTrap:* Usefull if you want to prohibit a menu from popping up. A message is created by clicking the right mouse button, a menu selection in the usual way is therefore not possible. If this flag is set, your application will receive a normal MOUSEBUTTON signal instead.

12. *SuperBitMap:* useful if you want to *draw* into your window – sets the flag WA_SuperBitMap, TRUE. This is a pointer to a BitMap structure for a SuperBitMap window. The application will be allocating and maintaining its own bitmap. Equivalent to NewWindow.BitMap. Setting this tag implies the WFLG_SUPER_BITMAP property.

    > ***Tip:*** For complete information about SuperBitMap, refer to
    > http://amigadev.elowar.com/read/ADCD_2.1/Libraries_Manual_guide/node011F.html

13. *BackDrop:* Make this window a Backdrop window – only thin borders will be drawn, depth- and size gadgets will be gone.

14. *Zoom:* A zoom gadget is by default *always* supplied to a window if it has both depth and sizing gadgets. The WA_Zoom tag allows the application to open a window with a zoom gadget when the window does not have both the depth and sizing gadgets. It occurs rather seldom that one needs to set this Tag.

15. *NotifyDepth:* This flag is usefull in conjunction with event handling: By default, MHI_CHANGEWINDOW is returned when the window is moved. For resizing of the window, you would listen to WMHI_NEWSIZE. By setting *WA_NotifyDepth*, this event will also be returned for depth arranging.

16. *Menu Help:* Enables IDCMP_MENUHELP: pressing Help during menus will return IDCMP_MENUHELP message. An IDCMP_MenuHelp message is sent by Intuition when the user selects the *Help key* while the menu system is activated. If a menu item was selected, the menu number of the menu item can be found in the Code field of the IntuiMessage. If no item was selected, the Code field will be equal to MENUNULL. These messages will only be sent if the WA_MenuHelp attribute is set for the window – and that's what this option does.

17. *Help Group:* enables IDCMP help message sharing between multiple windows in the same application. Normally, only the active window can receive IDCMP_GADGETHELP messages. However, an application with multiple windows will want all its windows to be able to receive help when any of them are active. First obtain a unique help ID with utility.library/GetUniqueID(), then pass it as ti_Data of this tag to all your windows. For further information, please also refer to HelpControl() in the description for OpenWindow() here: http://amigadev.elowar.com/read/ADCD_2.1/Includes_and_Autodocs_3._guide/node03D5.html

18. *OK:* accepts the changed flags and closes the dialog.

19. *Cancel:* abandons all of your changes and closes the dialog.

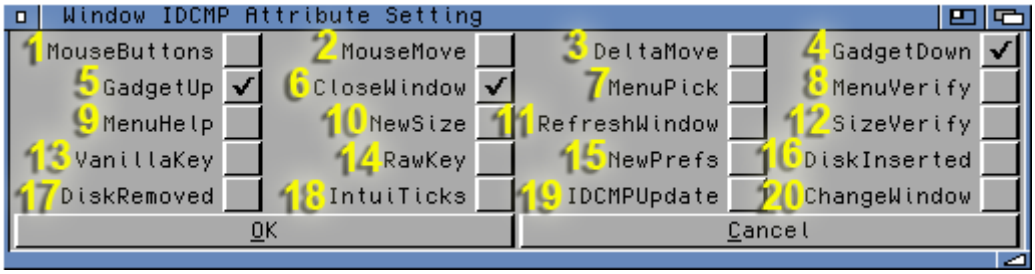
> ***Tip:*** For an in-deep description on window flags, refer to the Amiga developer docs:
> http://amigadev.elowar.com/read/ADCD_2.1/Libraries_Manual_guide/node0125.html

So far, so good...

For our project leave all defaults as-is, but additionally tick the "Activate" checkbox.
Click *OK* to accept the changes – that will close the "Window Flags Attribute Setting" dialog.

Adding IDCMP flags

IDCMP flags are important for IDCMP message event handling. In code, this is done by an *event loop* that regularly checks if certain events did happen. Those events are identified by the flags that have been set in the *"Window IDCMP Flags Attribute Setting" dialog.* Fire it up by clicking IDCMP now. Here's the meanings of the related events:



1. *MouseButtons*: IDCMP_MOUSEBUTTONS contains reports about mouse button up and down events. The events will be sent to the application only if they are *not used internally* by Intuition.

   The Code field contains information on the specific mouse button event this message represents. The Code field will be equal to SELECTDOWN, SELECTUP, MENUDOWN, MENUUP, MIDDLEDOWN or MIDDLEUP, depending on the button pressed or released. In general, the select button is the left mouse button, the menu button is the right mouse button and the middle button is an optional third button usually located between the select and menu buttons.

   Typically used if you need to identify which mouse button was pressed.

2. *MouseMove*: IDCMP_MOUSEMOVE reports about mouse movements, sent in the form of x and y coordinates relative to the upper left corner of the window. One message will be sent to the application for each "tick" of the mouse.

   Typically used if you want to draw with the mouse.

3. *DeltaMove*: IDCMP_DELTAMOVE is *not* a message type, and events will *never be sent to the application* with this IDCMP identifier. This flag is a *modifier*, which changes how mouse movements are reported. When this flag is set, mouse movements are sent as delta values rather than as absolute positions.

   The deltas are the amount of change of the mouse position from the last reported position. If the mouse does not move, then the delta values will be zero.

   This flag works in conjunction with the IDCMP_MOUSEMOVE flag. When IDCMP_DELTAMOVE is set, IDCMP_MOUSEBUTTONS messages will also have relative values, instead of the absolute window position of the mouse.

4. *GadgetDown* (default): Occurs when the user has clicking a gadget down (but not released it yet). This is an important flag for GadTools gadgets. The various GadTools gadget types require certain classes of IDCMP messages in order to work. Applications specify these IDCMP classes when the window is opened or IDCMP handling is altered. Each kind of GadTools gadget requires one or more of these IDCMP classes: IDCMP_GADGETUP, IDCMP_GADGETDOWN, IDCMP_MOUSEMOVE, IDCMP_MOUSEBUTTONS and IDCMP_INTUITICKS.

   If a certain kind of GadTools gadget is used, the window must use all IDCMP classes required by that kind of gadget. Do not omit any that are given for that class, even if the application does require the message type.

   Because of the way GadTools gadgets are implemented, programs that use them always require notification about window refresh events. Even if the application performs no rendering of its own, it may not use the WFLG_NOCAREREFRESH window flag and must always set IDCMP_REFRESHWINDOW in one or another way.

5. *GadgetUp* (default): Occurs when the user has released a formerly pressed gadget – another important GadTools flag. *You typically use it to check whether a button or similar gadget was clicked.*

6. *CloseWindow* (default): Occurs when a window's close gadget was clicked. That gadget therefore must have been activated in the "Window Attribute Settings" dialog before. No gadget – no message.

7. *MenuPick*: Occurs when the user has pressed the right mouse button while being over a window's drag bar. If not set, your application will not react on menu events.

8. *MenuVerify*: This is a special verification mode which allows the program to confirm that it is prepared to handle Intuition rendering, in this case, allowing menus to be drawn in the screen.

   This is a special kind of verification, in that any window in the entire screen that has this flag set must respond before the menu operations may proceed. Also, the active window of the screen is allowed to cancel the menu operation. This is unique to IDCMP_MENUVERIFY.

9. *MenuHelp*: This message is sent by Intuition when the user selects the Help key while the menu system is activated. If a menu item was selected, the menu number of the menu item can be found in the Code field of the IntuiMessage. If no item was selected, the Code field will be equal to MENUNULL.

   These messages will only be sent if the WA_MenuHelp attribute is set for the window.

   The menu number returned in IDCMP_MENUHELP may specify a position that cannot be generated through normal menu activity. For instance, the menu number may indicate one of the menu headers with no item or sub-item.

10. *NewSize*: Intuition sends this message after the user has resized the window. After receiving this, the program can examine the size variables in the window structure to discover the new size of the window. The message is sent, even if the size of the window did not actually change.

11. *RefreshWindow*: This message is sent whenever the window needs refreshing. This flag makes sense only with windows that have a refresh type of WA_SimpleRefresh or WA_SmartRefresh.

    As a minimum, the application must call BeginRefresh() and EndRefresh() for the window after receiving an IDCMP_REFRESHWINDOW event. Create the window with the WA_NoCareRefresh attribute if you do not want to manage these events. Most of the graphics library calls used for display output are compatible with Intuition, with the exception of ScrollRaster(). Intuition will not send an IDCMP_REFRESHWINDOW event when damage is caused to a window by ScrollRaster(). This may happen in a simple refresh window which is partially obscured by another window; the region that scrolls out from behind the front window will be damaged, but the window will receive no notification. Check the LAYERREFRESH bit in the Layer structure Flags field to see if damage did happen as a result of ScrollRaster().

12. *SizeVerify*: Set this flag if the program must complete some operation before the user sizes the window. When the user sizes the window, Intuition sends an IDCMP_SIZEVERIFY message to the application and then waits until the program replies before allowing the user to size the window.

13. *VanillaKey*: IDCMP_VANILLAKEY messages return keyboard events translated into the current default character keymap. The mapped character value is returned in the Code field of the IntuiMessage structure.

    An IDCMP_VANILLAKEY message is sent only if the translation results in a single byte value, therefore the program cannot read the "Help" or function keys using IDCMP_VANILLAKEY.

    Programs using IDCMP_VANILLAKEY which also require the additional information of special keys, such as the "Help" key and the function keys, may set both IDCMP_VANILLAKEY and IDCMP_RAWKEY. When this combination is used, all keypresses that map to single character values will be returned as IDCMP_VANILLAKEY events; all other keyboard events will be sent as IDCMP_RAWKEY messages. Note that IDCMP_VANILLAKEY processing uses all of the key-up events, so the application will only receive key-down events in the IDCMP_RAWKEY format.

14. *RawKey*: IDCMP_RAWKEY messages give the raw keycodes from the keyboard. The numeric value of the keycode is sent in the Code field. Separate codes are returned for key down and key up. Qualifier codes, such as "Shift" or "Alt" and whether this key is a repeat, may be found in the Qualifier field of the message.

    In general, the application should not assume any correspondence between the keycode and the key value. Character positions on the keyboard change from country to country, and the application should respect the keymap set by the user.

    Programs using IDCMP_RAWKEY messages should perform their own key mapping by calling the console.device function RawKeyConvert(), or the keymap.library function MapRawKey(). (The latter is a bit more convenient, but is only available under V36 and higher). The Autodoc for the MapRawKey() function shows how you can process so-called dead keys. A dead key is a key combination that has no immediate effect, but instead modifies a subsequent keystroke. For example, on the default keymap, Alt-F is a dead key for the acute accent mark. The sequence of Alt-F followed by the E key yields an e with an acute accent.

    The application can assume that certain keys will always return the same raw keycode, these keys do not have to be mapped. In general these keys are in the high part of the keymap, above hex 40, and includes all non-alphanumeric keys. The fixed keys include the function keys, backspace, delete, help and cursor keys.

15. *NewPrefs*: IDCMP_NEWPREFS messages are sent when the system Preferences are changed by a call to SetPrefs(). The program can learn of these changes by setting this flag.

    After receiving a message of class IDCMP_NEWPREFS, the application should call GetPrefs() to obtain a copy of the new Preferences.

    Under the new Preferences scheme used in Release 2 and later versions of the OS, an IDCMP_NEWPREFS message will not always be sent when the user changes a Preferences setting. Only Preferences values available under V34, i.e., those that can be modified by a call to SetPrefs(), will cause an IDCMP_NEWPREFS message to be sent. New Preferences items such as overscan or font settings rely on file system notification for monitoring changes.

    This message type is broadcast to all IDCMP that have this flag set, not just to the active window. If the application has this flag set, it should be prepared to handle the event even if it is not active.

16. *DiskInserted*: When the user inserts or removes a disk from any drive, Intuition will send one of the IDCMP_DISKINSERTED or IDCMP_DISKREMOVED message types.

    This message type is broadcast to all IDCMP that have this flag set, not just to the active window. If the application has this flag set, it should be prepared to handle the event even if it is not active.

17. *DiskRemoved*: see *DiskInserted*

18. *IntuiTicks*: Intuition sends these messages to the active window based on an internal timer which "ticks" roughly ten times a second. This provides the application with simple timer events from Intuition.

    Intuition does not allow IDCMP_INTUITICKS events to accumulate at a port. After an IDCMP_INTUITICKS message has been sent to a port, Intuition will not send another until the application replies to the first. This means that an application that has not been able to service the IDCMP for an extended period can expect at most one IDCMP_INTUITICKS message to be waiting at the port.

    These events are to be used as "prods," and not as time counters. Do not rely on the timing accuracy of the event, or on the exact frequency at which they appear. Remember, IDCMP_INTUITICKS will only be sent to the active window. If the user selects another window, the events will no longer be received at the first window.

19. *IDCMPUpdate*: Used for notification from BOOPSI custom gadgets.
20. *ChangeWindow*: This message provides the window with notification of any change in the size or position of a window.

OK and *Cancel* buttons work as usual.

> ***Tip:*** For an in-deep description on a window's IDCMP flags, refer to the Amiga developer docs:
> http://amigadev.elowar.com/read/ADCD_2.1/Libraries_Manual_guide/node01DC.html
> http://amigadev.elowar.com/read/ADCD_2.1/Libraries_Manual_guide/node025C.html
> https://wiki.amigaos.net/wiki/Intuition_Input_and_Output_Methods#IDCMP_Flags

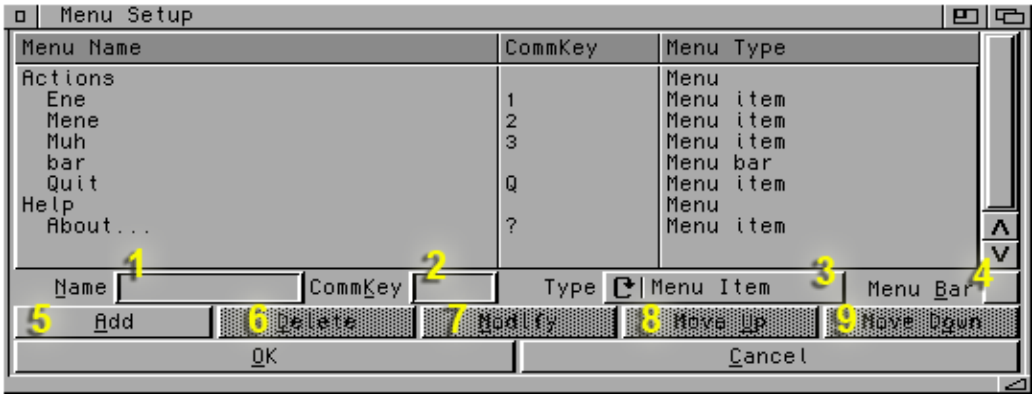
Since we want our project to have a menu and react on it, we need the application to have IDCMP_MenuPick set. Tick option *MenuPick (7)*!

Click *OK* to accept the changes – that will close the dialog. In "Window Attribute Setting" dialog, do the same. Click on *Save* to save the recent version of the project - just in case...

We are done with the window settings - now it's time to create some menus!

1.2.2. Adding Menus

Double click the "Menu - Menu_4" entry - the "Menu Setup" dialog will pop up:



1. *Name:* Enter the name of the Menu, Menu item or SubMenu item that should be added
2. *ComKey:* If wanted, enter a character to be used as a shortcut for the selected entry
3. *Type:* Chose between
   - Menu
   - Menu Item
   - Menu Sub Item

   equivalent to what kind of menu entry you want to generate
4. *Menu Bar:* tick this checkbox if you want to create a separator between two entries. Be aware that in order to be generated, a name *must* be given to it. This name will be ignored by the code generator later, so it's perfectly legal to just name it *bar* – even if you do this for multiple separators.
5. *Add:* adds a recently defined entry to the menu hierarchy
6. *Delete:* deletes a selected entry from the menu hierarchy

7. *Modify:* modify a selected entry. Simply select an entry, then change the attribute in question and click the *Modify* button
8. *Move Up:* in menu hierarchy, moves a selected entry up one step
9. *Move Down:* in menu hierarchy, moves a selected entry down one step

The OK and Cancel buttons work in the usual way.

Now let's build the menus for our Project!

1. Add a menu named *Actions*
2. Add a menu item named *Ene* with ComKey *1*
3. Add a menu item named *Mene* with ComKey *2*
4. Add a menu item named *Muh* with ComKey *3*
5. Add a menu bar named *bar* with no ComKey (tick *Menu Bar* CheckBox!)
6. Add a menu item named *Quit* with ComKey *Q*
7. Add a menu named *Help*
8. Add a menu item named *About...* with ComKey *?*

When finished with that job, your menu hierarchy should look like the one in the picture above. Now click OK to accept your work - this will close the "Menu Setup" dialog. Save the project.

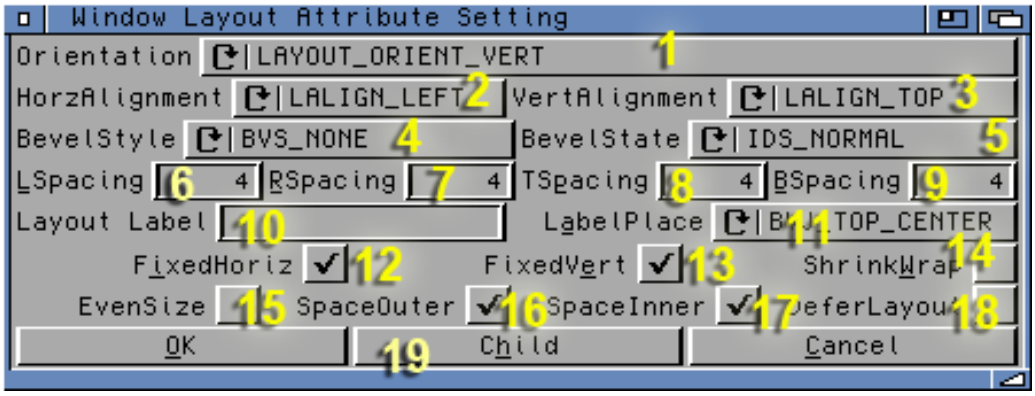
> ***Hint:*** The Menu Setup dialog only creates rather basic menu structures. In order to build more sophisticated menu entries, like f.e. ones with checkmarks or mutual exclusive behavior, you will have to edit their definitions by hand. We will talk about this when discussing code enhancements later!

Next, we will add the layouts.

1.2.3. Adding Layouts

Remember the layout structure that we developed in theory before? If unsure please re-read *"CheckBox Demo – a potential layout solution at page (17)"*

Now we are going to implement the mentioned layout – but first let's have a look at the *"Window Layout Attribute Setting" dialog.* In order to open it, you first have to select our main layout in ReBuild's *layout elements area.* Now click the *Add* button - this pops up the "Select an object to add" dialog *(will be described in detail later).* There, click the *Layout* button. This finally opens our *"Window Layout Attribute Setting" dialog*, consisting of:



1. *Orientation*: This basically sets the direction for the way that child objects will be presented: Either *vertically* – a new object will be placed *under* the one defined before it, or *horizontally* – a new object will be placed *on the right side* of the one defined before it. Values: One of
   - LAYOUT_ORIENT_VERT *or*
   - LAYOUT_ORIENT_HORIZ
2. *HorizAlignment*: sets the horizontal alignment of new objects attached to the layout – either on the *right* or *left* side. Values are:
   - LALIGN_LEFT *or*
   - LALIGN_RIGHT
3. *VertAlignment*: *same as HorizAlignment*, but for *vertical* alignment. Values:
   - LALIGN_TOP *or*
   - LALIGN_BOTTOM
4. *BevelStyle*: if set to anything else than *BVS_NONE*, draws a bevel box with a predefined style around the layout. You'll have the choice between:
   - BVS_NONE
   - BVS_THIN
   - BVS_BUTTON
   - BVS_GROUP
   - BVS_FIELD
   - BVS_DROPBOX
   - BVS_SBAR_HORIZ
   - BVS_SBAR_VERT
   - BVS_BOX
   - BVS_RADIOBUTTON
   - BVS_STANDARD

   Style Guide Warning: It is widely regarded *bad UI design* to draw BevelBoxes around layouts. In fact, in most cases this would be just an optical gimmick and does not result in a GUI that is more *intuitive* to the user. The benefits of using them are rather small.
   However for demonstration purpose, we will use that feature for this tutorial.
5. *BevelState*: with *BevelStyle* set to anything else than *BVS_NONE*, changes the look of the bevel box to:
   - IDS_NORMAL
   - IDS_SELECTED
   - IDS_DISABLED

   Note that this setting has no effect if *BevelStyle* is set to *BVS_NONE*
6. *LSpacing*: affects the space that objects added to layout are indented for the inner *left* side of the layout. Value: Amount of pixels
7. *RSpacing*: affects the space that objects added to layout are indented for the inner *right* side of the layout. Value: Amount of pixels

8. *TSpacing*: affects the space that objects added to layout are indented for the inner *top* of the layout. Value: Amount of pixels

9. *BSpacing*: affects the space that objects added to layout are indented for the inner *bottom* of the layout. Value: Amount of pixels

> ***Style Guide Warning:*** In a production scenario, you should leave the ***L R T B spacing Tags*** *untouched*, but adjust your layout in source code, by f.e. adding "blind" layouts in order to achieve the look you want.
> However for the sake of not over-complicating things for this tutorial, we will use them for demonstration purpose.

10. *Layout Label*: Text to be displayed for a bevel box. This is optional. Note that it will be ignored if *BevelStyle* is set to *BVS_NONE*

11. *LabelPlace*: defines the position where the *Layout Label* text will be placed. Positons are:
    - BVJ_TOP_CENTER
    - BVJ_TOP_LEFT
    - BVJ_TOP_RIGHT
    - BVJ_IN_CENTER
    - BVJ_IN_LEFT
    - BVJ_IN_RIGHT

    Note that as with *BevelStyle* set to *BVS_NONE*, *LabelPlace* will be ignored

12. *FixedHoriz*: LAYOUT_FixedHoriz (BOOL) Whether the width of this layout group is fixed to the width assigned by the window or parent layout group. If set to FALSE, the gadget will minimize its width to the smallest possible value. Generally you will have to also set LAYOUT_ShrinkWrap (below) to TRUE also. It is *recommended that this tag is used only in the top layout group.* Using a weighted width of 0 gives more predictable results elsewhere.

    Defaults to TRUE.

13. *FixedVert*: LAYOUT_FixedVert (BOOL) Whether the height of this layout group is fixed to the height assigned by the window or parent layout group. If set to FALSE, the gadget will minimize its height to the smallest possible value. Generally you will have to also set LAYOUT_ShrinkWrap (below) to TRUE also. *It is recommended that this tag is used only in the top layout group.* Using a weighted height of 0 gives more predictable results elsewhere.

    Defaults to TRUE.

14. *ShrinkWrap*: The LAYOUT_ShrinkWrap tag makes sure there is no surplus space between children. You can think of it as the "justify" alignment you know from word processors

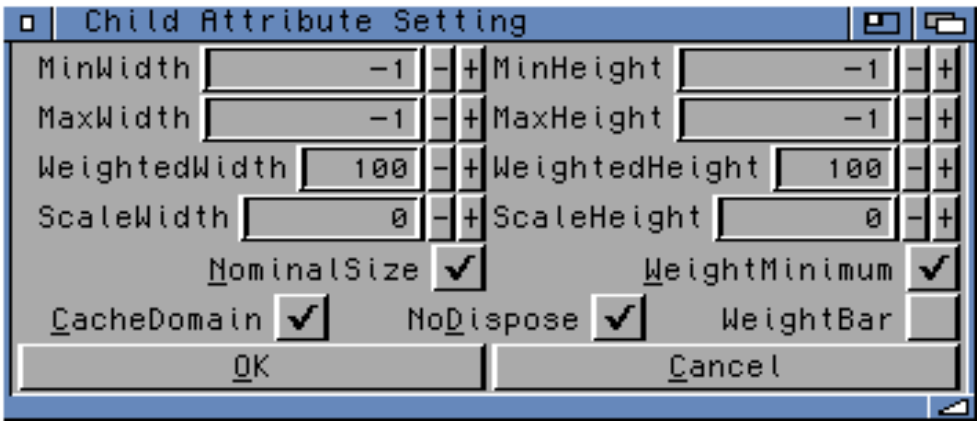
15. *EvenSize*: The LAYOUT_EvenSize tag makes the layout.gadget use some of the surplus space to make all children of equal size. It is only increased upto the largest of the children's minimum sizes. This comes in handy when you have a row of buttons. The style guide suggests that they have the same size, yet they shouldn't take up all of the space.

16. *SpaceOuter*: All around the layout you can have a margin (outer spacing) which you can turn on or off with LAYOUT_SpaceOuter.

17. *SpaceInner*: The layout can also have spacing between its children which you control with LAYOUT_SpaceInner in a similar manner.

18. *DeferLayout*: DeferLayout defines that a window object will defer GM_LAYOUT and GM_RENDER from the input.device and let the application take care of those events.

19. *Child*: opens the "Child Attribute" dialog, which is for setting different, rather self-explaining parameters for added child objects:



One important option here is *Weight Bar* CheckBox: If you add a new layout between two objects first, then open *Child Attribute Settings* and check *WeightBar*, a small, dotted handle will be placed between those objects. This weight bar allows you to proportionally scale the objects in direction of the added layout.

For our project, we don't need to mess with this dialog now. If you opened it, please close it now. We will continue with creating our project's basic layout.

OK and *Cancel* work in the usual way.

> ***Hint:*** For a detailed description on layout parameters, please refer to the documentation of layout.gadget here: **??**
> You may also find usefull information in NDK3.2 section include_h/gadgets/layout.h

Creating the project's basic layout

Now let's finally add our planned layouts to our project:

1. Select our main layout and click on *Add* - the "Window Layout Settings" dialog appears. Here, change the orientation to *LAYOUT_ORIENT_HORIZ* and click *OK*. This will place the new layout visibly indented under the main layout – a sign for its hierarchical order.

2. With the new horizontal layout selected, click on *Add*. In "Window Layout Settings", change the orientation to *LAYOUT_ORIENT_VERT*. Change the BevelStyle to BVS_THIN, too. Click *OK*. As before with main layout, the new layout is placed indented, this time under the the horizontal layout. *That'll be the one where we later will place our String gadgets.*

3. Now copy that layout to the buffer.

4. Select the horizontal layout and copy the one from buffer back to it. It will appear above our previously generated vertical layout. *Later, we will place our CheckBox gadgets there.*

   The preview window now shows two horizontally aligned, beveled layouts.

5. With the main layout selected, add another vertical layout in the way described before – *later, we will place the Button gadget there.* It will appear at the same level as the horizontal layout, being out-dented against its children. The preview window now shows two horizontally aligned, beveled layouts and a huge space under them.

We are done with the basic layout – save your work. Next, we will add the gadgets.

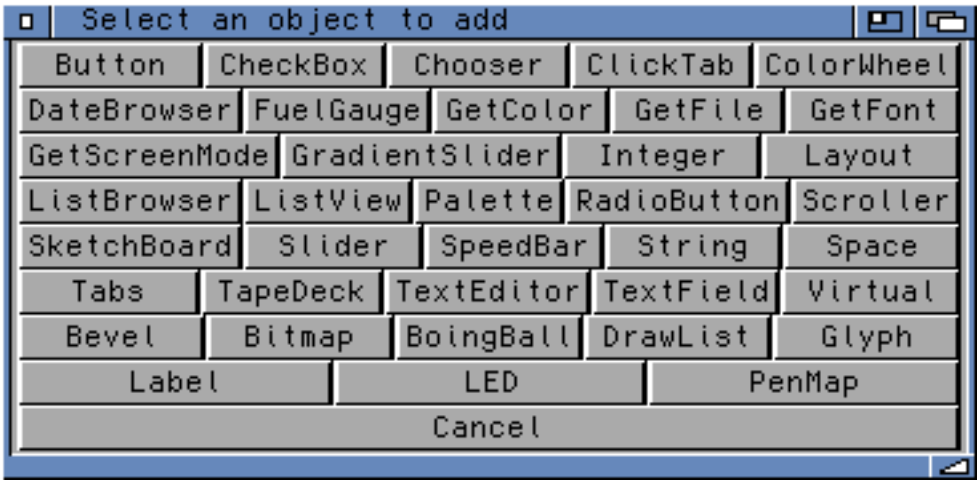
1.2.4. Adding Gadgets

We arrived at the last view steps in designing a GUI - finally let's add our gadgets as planned.

1. Select the first vertical layout below the horizontal layout - here go our CheckBox gadgets.

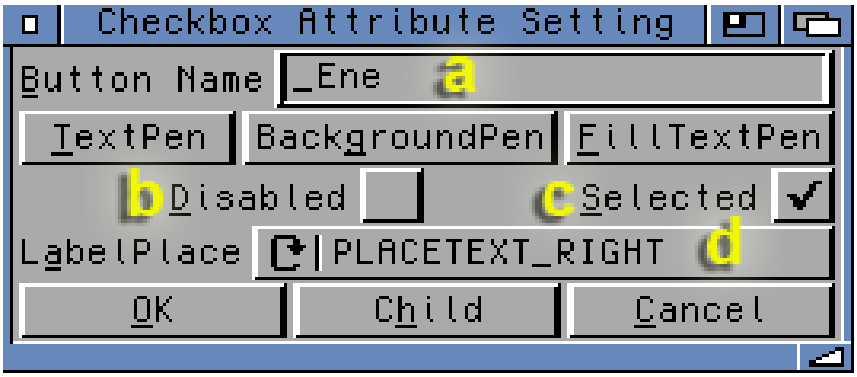
2. Click *Add* and in the "Select an Object to add" dialog, select *CheckBox*.

   Let's have a short break here! The "Select an Object to add" dialog offers you nearly all available gadgets, classes and images that ReAction has to offer:



   Each of them buttons adds the respective object to the selected layout, popping up its own unique attributes dialog. It is beyond the scope of this tutorial to explain them all in detail. For Information about each object, please refer to appendix E!

3. Selecting *CheckBox* pops up the "Checkbox Attribute Setting" dialog:



   Its elements are pretty self-explanatory. We will go for

   a) *Button Name*: The Text shown in conjunction with the CheckBox gadget. **Enter "_Ene" here.**

      Hint: The Underscore marks the letter that will be shown as a shortcut for the object. Please note that you should avoid using the same shortcut for different gadgets, unless you want to watch your application go nuts with unexpected behavior.

b) *Disabled*: Enables/disables the object. **Leave it untouched.**

c) *Selected*: Selects/deselects the CheckBox – if selected, a checkmark is shown. **Check this option.**

d) *LabelPlace*: Determines the position of the text for this object. **Leave it untouched.**

4. Click OK to create the checkbox.

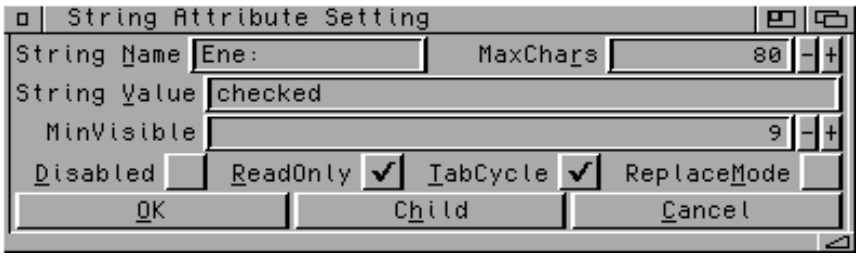
5. Copy the CheckBox gadget to the buffer, then copy it back two times. Empty the buffer afterwards.

6. Double click the second CheckBox entry, change the Button Name to "_Mene" and remove the checkmark for *Selected*, then click *OK*.

7. Do the same for the third CheckBox entry, changing the Button Name to "Mu_h".

   Now go for the next vertical layout!

8. Select it, click *Add* and in the arising dialog, select *String*. This brings up the "String Attribute Setting" dialog:



   Set the attributes according to the screenshot and click *OK*.

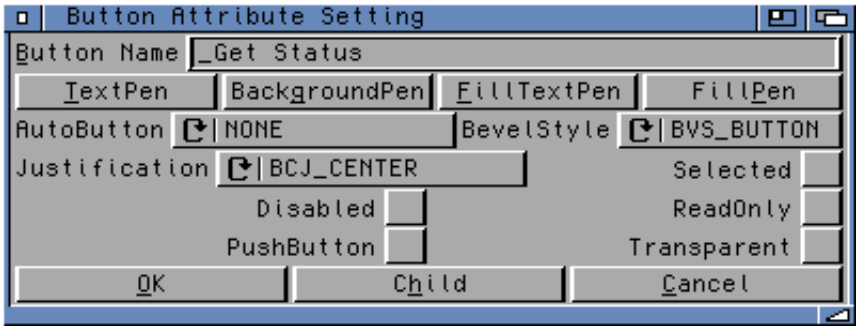
9. Copy the String gadget to the buffer and copy it back two times.

10. Double click the second String entry, changing *String Name* to "Mene" and *String Value* to "unchecked". Click *OK*.

11. Do the same for the third String entry, changing *String Name* to "Muh". Click *OK*.

    Now go for the last vertical layout!

12. Select it, click *Add* and in the arising dialog, select *Button*. This brings up the "Button Attribute Setting" dialog:



   Set the attributes according to the screenshot and click *OK*. That's it.

We are done with this chapter – don't forget to save the Project!

You may want to play around with certain settings in order to explore which effects they cause – be encouraged to do so! For example, you might want to select the last vertical layout and change the values for the different spacing attributes to make the button look indented, like initially planned.

Again: When designing a GUI for public releasing, please have in mind that this works – but is considered *bad UI design* confirming to the official Amiga Style Guide! In a production environment, you should *never* touch the *L R T B spacing Tags*, but adjust your layout in source code. Also putting BevelBoxes around layouts is widely regarded as a *Style Guide violation*. When planning a GUI design, adapt your layout draft to these recommendations!

In the next chapter, we will go for analyzing and enhancing ReBuild-produced source code, transforming it to a working application. Stay tuned!

Part II

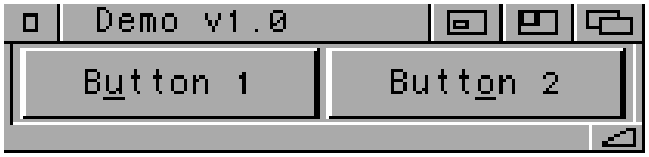
Modifying and enhancing ReBuild-generated code

2. Overview - ReBuild-generated functions

ReBuild's C code output mainly consists of four functions - we will shortly have a look at what they do. But first we need to create a quick demo application in order to share a common basis to talk about...

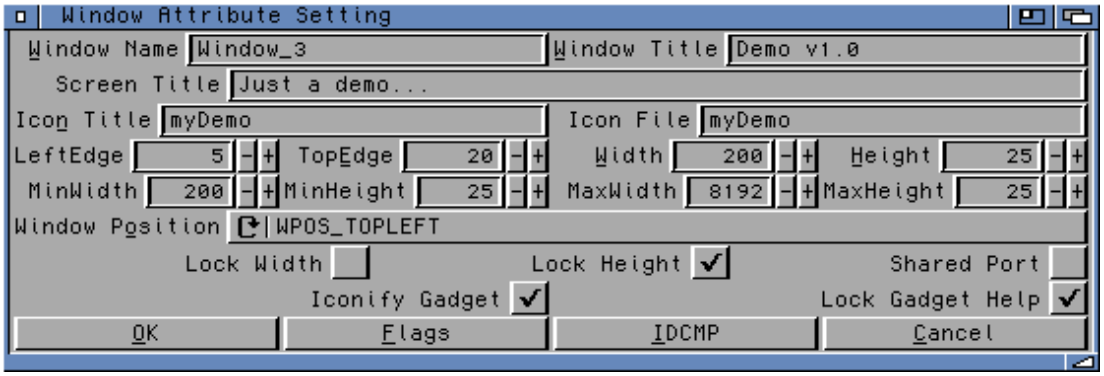
2.1. Creating a demo app to work with

Create a new folder "demoapp", then fire up ReBuild and save the project there, naming it "mydemo.reb". This project will later be intensively used for explanations on functions, IDCMP event differentiation, creating and reacting on sophisticated menus, implementing ButtonHelp. This is how the application will look like in the end:



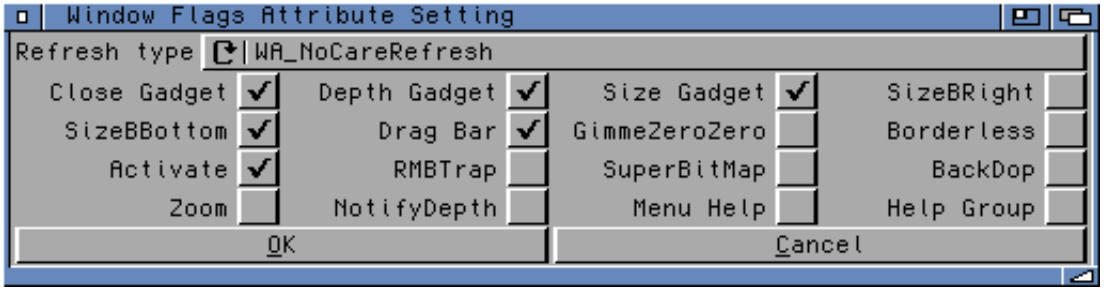
2.1.1. The Window

1. Edit the window according to this values:



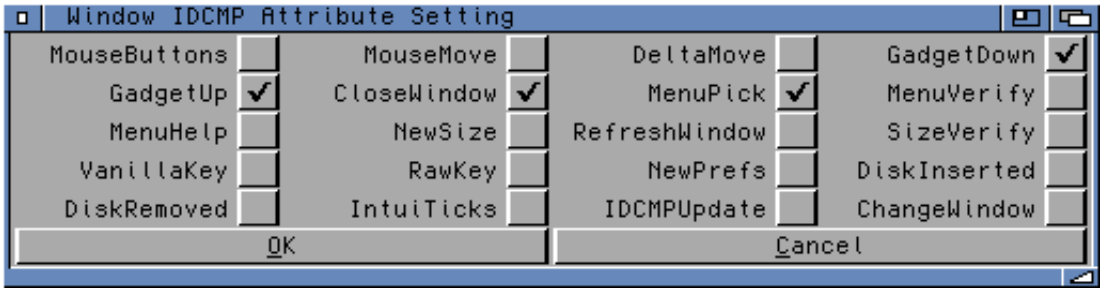
2. Edit Window Flags:

   Set values according to this:



3. Edit IDCMP Flags:

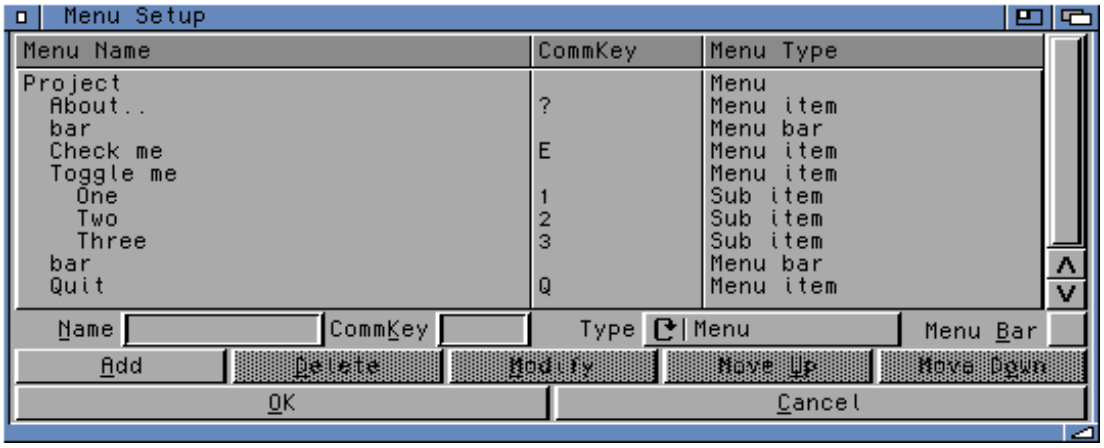
   Set values according to this:



4. Click *OK* to accept changed window attributes.

2.1.2. Menus

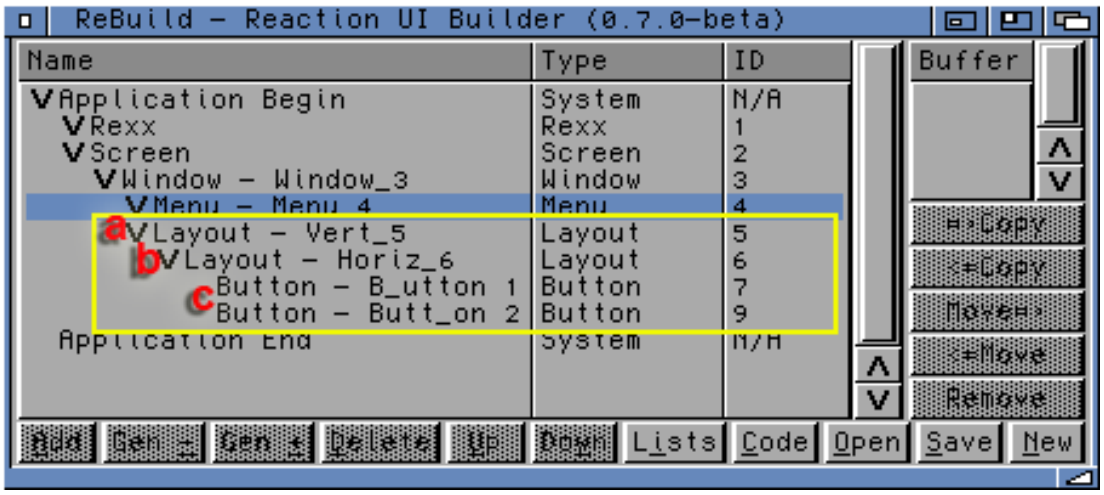
1. Create a menu hierarchy, according to this:



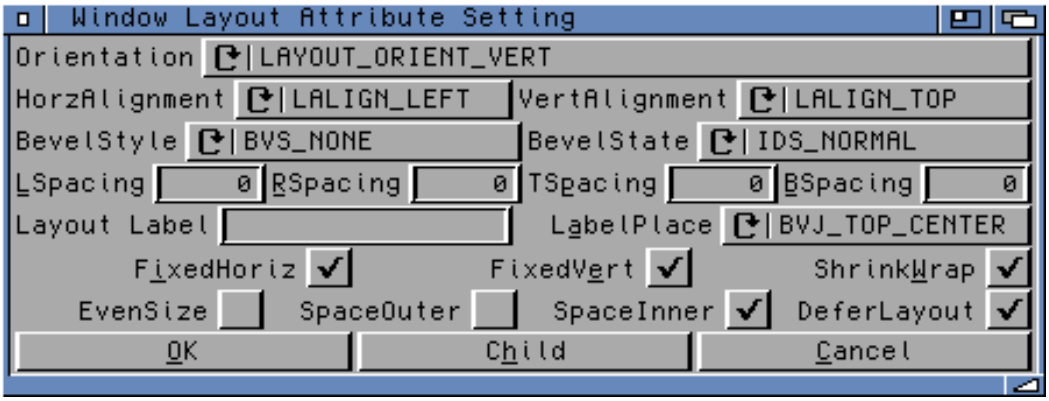
2. Click *OK* to accept changed menu attributes.

2.1.3. Layout

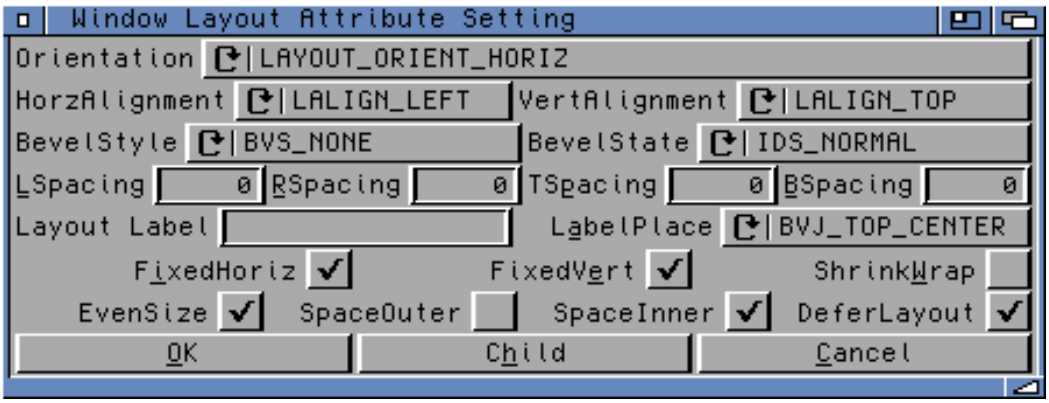
1. Create a layout hierarchy according to this:



a) For the vertical main layout, use this values:



b) For the horizontal sub layout, use this values:



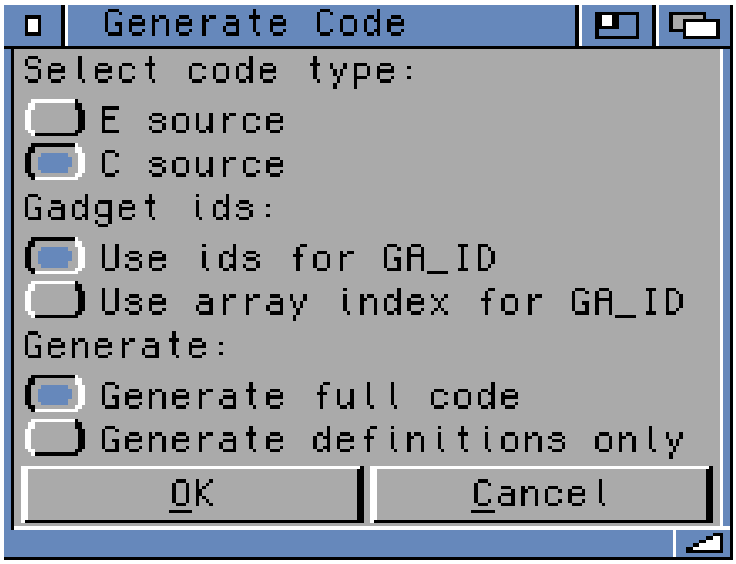
c) For the buttons, change option *Button Name* as follows:

- *Button 1:* “B_utton 1”
- *Button 2:* “Butt_on 1”

2. Now save the project!

2.1.4. Code generation

1. Click on Code and generate the source code according to these settings:



2. Click *OK* to save. A SaveFileRequest pops up – here, select your project folder and save as “myDemo.c”

2.1.5. Code compilation

Now let's compile and test the application...

1. Open a Shell and CD to your project folder
2. Compile the source:

   For ***SAS/C***, enter:

   ```
   sc ICONS myDemo.c LINK TO myDemo
   ```

   For ***vbcc***, enter:

   ```
   vc +aos68k -lamiga -o=myDemo myDemo.c
   ```

   Hint: With ***vbcc***, in order to have a custom icon when iconifying the app, you need to copy a *program type* icon to the folder where the app is located. Rename it to *myApp* for this purpose.

3. Run myDemo now. Check window sizing, menus, buttons and shortcuts, iconify/uniconify the app.

2.2. ReBuild's exported code functions

Now, that we have a common code base, let's have a deeper look at the generated functions!

2.2.1. int setup(void)

Takes care of opening libraries, classes, images and gadgets in dependence of the objects created in your GUI. Also creates a pointer to an IDCMP message port for your window.

```
/*
*  int setup(void)
*  - open libraries, initialize screen and window-handling
*  - create a pointer to a message port for IDCMP events
*/
int setup( void )
{
  if( !(IntuitionBase = (struct IntuitionBase*) OpenLibrary("intuition.library",0L)) ) return 0;
  if( !(GadToolsBase = (struct Library*) OpenLibrary("gadtools.library",0L) ) ) return 0;
  if( !(WindowBase = (struct Library*) OpenLibrary("window.class",0L) ) ) return 0;
  if( !(IconBase = (struct Library*) OpenLibrary("icon.library",0L) ) ) return 0;
  if( !(LayoutBase = (struct Library*) OpenLibrary("gadgets/layout.gadget",0L) ) ) return 0;
  if( !(ButtonBase = (struct Library*) OpenLibrary("gadgets/button.gadget",0L) ) ) return 0;
  if( !(gScreen = LockPubScreen( 0 ) ) ) return 0;
  if( !(gVisinfo = GetVisualInfo( gScreen, TAG_DONE ) ) ) return 0;
  if( !(gDrawInfo = GetScreenDrawInfo ( gScreen ) ) ) return 0;
  if( !(gAppPort = CreateMsgPort() ) ) return 0;

  return -1;
}
```

2.2.2. void cleanup(void)

Takes care of closing libraries, classes, images and gadgets in dependence of the objects created in your GUI. Also frees certain other ressources.

```
74  /*
75  *  void cleanup(void)
76  *  - close libraries
77  *  - free window resources
78  */
79  void cleanup( void )
80  {
81    if ( gDrawInfo ) FreeScreenDrawInfo( gScreen, gDrawInfo);
82    if ( gVisinfo ) FreeVisualInfo( gVisinfo );
83    if ( gAppPort ) DeleteMsgPort( gAppPort );
84    if ( gScreen ) UnlockPubScreen( 0, gScreen );
85
86    if (GadToolsBase) CloseLibrary( (struct Library *)GadToolsBase );
87    if (IconBase) CloseLibrary( (struct Library *)IconBase );
88    if (IntuitionBase) CloseLibrary( (struct Library *)IntuitionBase );
89    if (ButtonBase) CloseLibrary( (struct Library *)ButtonBase );
90    if (LayoutBase) CloseLibrary( (struct Library *)LayoutBase );
91    if (WindowBase) CloseLibrary( (struct Library *)WindowBase );
92  }
```

2.2.3. void runWindow(...)

Opens a certain window and creates an IDCMP message loop for it.

Example call: *runWindow(window_object, 3, menu_strip, main_gadgets, gadget_ids);*

The function takes 5 **Parameters:**

1. *Object *window_object*: Pointer to a window object, as declared in line 195 in function "void window_3(void)" of the full source for the example.

2. *int window_id*: This parameter identifies the window that shall be opened. The value in question has been stored in *int gadget_ids[5]* when the window was created in design. Unfortunately, you can't see that in code - in order to identify a window's ID, you will have to take a look into ReBuild's main window. Use the ID found there for identifying the window in question. To make things comfortable in a multi-window application, you might want to to define "speaking" ID and use that instead of a cryptic number, e.g.

   #define WND_FIRST_WIN 3

3. *struct Menu *menu_strip*: Pointer to a MenuStrip structure as declared in line 155 in function "void window_3(void)" of the full source for the example *(see page 41)*.

4. *struct Gadget *win_gadgets[ ]*: ***1** Pointer to a gadget array structure that holds all gadgets of a window. In this case: *main_gadgets* as declared in line 186 in function "void window_3(void)" of the full source for the example *(see page 41)*.

5. *int gadget_ids[ ]*: ***2** A gadget array that holds all gadget IDs of a window. In this case: *gadget_ids* as declared in line 187 in function "void window_3(void)" of the full source for the example *(see page 41)*.

> ***1, 2 Tip:** The gadget_ ids[ ] and the win_gadgets[ ] arrays are in sync, so it's f.e. useful if you want to get or update the value of a particular gadget based on the id – in that case you'd search the gadget_ ids[ ] array and use the corresponding entry in the win_gadgets[ ] array.

```
95  /*
96  *  void runWindow (parameters...)
97  *  - run a certain window
```

```
98   *  - provide an IDCMP message handling loop
99   */
100  void runWindow( Object *window_object, int window_id, struct Menu *menu_strip,
101                  struct Gadget *win_gadgets[], int gadget_ids[] )
102  {
103    struct Window *main_window = NULL;
104
105    if ( window_object )
106    {
107      if ( main_window = (struct Window *) RA_OpenWindow( window_object ))
108      {
109        WORD Code;
110        ULONG wait = 0, signal = 0, result = 0, done = FALSE;
111
112        /* --- event loop --- */
113        GetAttr( WINDOW_SigMask, window_object, &signal );
114        if ( menu_strip)  SetMenuStrip( main_window, menu_strip );     // attach menus
115        while ( !done)
116        {
117          wait = Wait( signal | SIGBREAKF_CTRL_C );
118
119          if ( wait & SIGBREAKF_CTRL_C )
120          done = TRUE;  // done = TRUE exits the event loop
121          else
122          while (( result = RA_HandleInput( window_object, &Code )) != WMHI_LASTMSG)
123          {
124            switch ( result & WMHI_CLASSMASK )
125            {
126              case WMHI_CLOSEWINDOW:     // window's close gadget was pressed
127              done = TRUE;
128              break;
129
130              case WMHI_MENUPICK:    // a menu was selected
131              puts("menu pick");
132              // HERE you should identify the selected item or sub item!
133              // If there are checkable entries, you should also identify
134              // the status of the entry.
135              break;
136
137              case WMHI_GADGETUP:    // a gadget was pressed and released
138              puts("gadget press");
139              // HERE you should identify the pressed gadget!
140              break;
141
142              case WMHI_ICONIFY:     // window's iconify gadget was clicked
143              if ( RA_Iconify( window_object ) )
144              main_window = NULL;
145              break;
146
147              case WMHI_UNICONIFY:  // Gadget was double clicked on Workbench
148              main_window = RA_OpenWindow( window_object );
149              // IMPORTANT: restore menu strip:
150              if ( menu_strip)  SetMenuStrip( main_window, menu_strip );
151              break;
152
153            }
154          }
155        }
156      }
157    }
```

```
158 }
```

The function's IDCMP event loop

Lines 112 to 153 of the listing contain the IDCMP event loop.

- *Lines 126 to 128*: Check if the window's close gadget was pressed.
- *Lines 130 to 135*: Check if a menu event happened. You still will have to write code that identifies the Item or SubItem that caused the event and react according to it.
- *Lines 137 to 140*: Check if a gadget was pressed and released. You still will have to write code that identifies the gadget that caused the event and react according to it.
- *Lines 142 to 145*: Check if the window's iconify gadget was pressed.
- *Lines 147 to 151*: Check if application was asked to uniconify by the user, double clicking the icon that represents the iconified app.

Note that ReAction does *not* provide equivalent *WMHI_ type events* for every IDCMP message that Intuition provides. If you need to check for an event that is not directly supported by ReAction, you may have to write a special hook function in order to collect it.

More on *enhancing* the IDCMP event loop will follow shortly. For more information, see pages 44 and 46 *ff.*

2.2.4. void window_3(void)

Remember that this function is named after the *Window Name* option in ReBuild's "Window Attribute Setting" dialog – e.g. if you defined *"Window_Main"* there, the name of the resulting function would be *void window_main(void)* ...

This function is called by main() after *setup()* in order to provide window and layout information. The function itself then calls *runWindow(...)* in order to open the window and to provide an IDCMP event loop.

You will find lots of useful information here, like e.g. identifying gadget IDs – note the listing's comments for hints!

```
161 /*
162 *  void window_xyz(void)
163 *  - defines menues, window and layout
164 *  - calls runWindow()
165 *  - frees ressources before quitting the app
166 */
167 void window_3( void )
168 {
169   /* —— Menu structure —— */
170   struct NewMenu menuData[] =
171   {
172     { NM_TITLE, "Project",0,0,0,NULL },
173     { NM_ITEM, "About..","?",0,0,NULL },
174     { NM_ITEM, NM_BARLABEL,0,0,0,NULL },
175     { NM_ITEM, "Check me","E",0,0,NULL }, // edit to be checkable
176     { NM_ITEM, "Toggle me",0,0,0,NULL },
```

```
177         { NM_SUB, "One","1",0,0,NULL },      // edit to be mutual exclusive
178         { NM_SUB, "Two","2",0,0,NULL },      // edit to be mutual exclusive
179         { NM_SUB, "Three","3",0,0,NULL },    // edit to be mutual exclusive
180         { NM_ITEM, NM_BARLABEL,0,0,0,NULL },
181         { NM_ITEM, "Quit","Q",0,0,NULL },
182         { NM_END, NULL, 0, 0, 0, (APTR)0 }
183       };
184
185       struct Menu *menu_strip = NULL;
186       struct Gadget *main_gadgets[ 5 ];
187       int gadget_ids[ 5 ];
188       Object *window_object = NULL;
189       menu_strip = CreateMenusA( menuData, TAG_END );
190       LayoutMenus( menu_strip, gVisinfo,
191       GTMN_NewLookMenus, TRUE,
192       TAG_DONE );
193
194
195       window_object = WindowObject,
196       WA_Title, "Demo v1.0",
197       WA_ScreenTitle, "Just a demo...",
198       WA_Left, 5,
199       WA_Top, 20,
200       WA_Width, 200,
201       WA_Height, 25,
202       WA_MinWidth, 200,
203       WA_MinHeight, 25,
204       WA_MaxWidth, 8192,
205       WA_MaxHeight, 25,
206       WINDOW_LockHeight, TRUE,
207       WINDOW_IconifyGadget, TRUE,
208       WINDOW_GadgetHelp, TRUE,
209       WINDOW_AppPort, gAppPort,
210       WINDOW_IconifyGadget, TRUE,
211       WA_CloseGadget, TRUE,
212       WA_DepthGadget, TRUE,
213       WA_SizeGadget, TRUE,
214       WA_DragBar, TRUE,
215       WA_Activate, TRUE,
216       WA_SizeBBottom, TRUE,
217       WINDOW_Position, WPOS_TOPLEFT,
218       WINDOW_IconTitle, "myDemo",
219       WINDOW_Icon,  GetDiskObject("myDemo"),
220       WA_NoCareRefresh, TRUE,
221       WA_IDCMP, IDCMP_GADGETDOWN | IDCMP_GADGETUP | IDCMP_CLOSEWINDOW | IDCMP_MENUPICK,
222       WINDOW_ParentGroup, VLayoutObject,
223       LAYOUT_SpaceOuter, TRUE,
224       LAYOUT_DeferLayout, TRUE,
225       LAYOUT_AddChild, main_gadgets[0] = LayoutObject,
226       GA_ID, gadget_ids[0] = 5,
227       LAYOUT_Orientation, LAYOUT_ORIENT_VERT,
228       LAYOUT_ShrinkWrap, TRUE,
229       LAYOUT_DeferLayout, TRUE,
230       LAYOUT_AddChild, main_gadgets[1] = LayoutObject,
231       GA_ID, gadget_ids[1] = 6,
232       LAYOUT_Orientation, LAYOUT_ORIENT_HORIZ,
233       LAYOUT_EvenSize, TRUE,
234       LAYOUT_DeferLayout, TRUE,
235       LAYOUT_AddChild, main_gadgets[2] = ButtonObject,
236       GA_ID, gadget_ids[2] = 7, // identifies first button with ID "7"
```

```
    GA_Text, "B_utton 1",
    GA_RelVerify, TRUE,
    GA_TabCycle, TRUE,
    BUTTON_TextPen, 1,
    BUTTON_BackgroundPen, 0,
    BUTTON_FillTextPen, 1,
    BUTTON_FillPen, 3,
    ButtonEnd,
    LAYOUT_AddChild, main_gadgets[3] = ButtonObject,
    GA_ID, gadget_ids[3] = 9,   // identifies second button with ID "9"
    GA_Text, "Butt_on 2",
    GA_RelVerify, TRUE,
    GA_TabCycle, TRUE,
    BUTTON_TextPen, 1,
    BUTTON_BackgroundPen, 0,
    BUTTON_FillTextPen, 1,
    BUTTON_FillPen, 3,
    ButtonEnd,
    LayoutEnd,
    LayoutEnd,
    LayoutEnd,
    WindowEnd;
    main_gadgets[4] = 0;
    gadget_ids[4] = 0;

    // call runwindow()
    runWindow( window_object, 3, menu_strip, main_gadgets, gadget_ids );

    if ( window_object ) DisposeObject( window_object );
    if ( menu_strip ) FreeMenus( menu_strip );
  }
```

int main(...)

There's not so much to say about the *main()* function - it is pretty self-explaining...

```
  /*
  * —— main routine ——
  */
  int main( int argc, char **argv )
  {
    if ( setup() )    // call setup() to prepare things
    {
      window_3(); // call window definitions, this will
                  // call runWindow(), too...
    }
    cleanup();    // call cleanup() for clean exiting the app
  }
```

3. Enhancing functionality

3.1. Reacting on Gadgets - event loop part 1

Gadgets usually should trigger some sort of reaction if pressed – therefore, we first need a way to distinguish *which* gadget was pressed. This is done in the event loop of the generated *runWindow()* function. If some sort of gadget is pressed and released, it usually causes an IDCMP_GADGETUP event – under ReAction, this is masked into a WMHI_GADGETUP event. The event loop only tells us *that* some gadget was pressed if WMHI_GADGETUP occurs – so we have to get hands on a method to grab the gadget in charge, identifying it for us. This also means that the gadget itself must provide some way of identification – a unique ID.

Well, let's have a look at our formerly created *myDemo* project *(see pages 111 ff)*. If not done so already, load *myDemo.reb* into ReBuild and generate the *myDemo.c* source file. Open that in your editor and go to its *RunWindow()* function, searching for a WMHI_GADGETUP entry:

```
...
case WMHI_GADGETUP:            // user clicked a gadget
  puts("gadget press");
  break;
  ...
```

That's the part where the program is told *that* a gadget was clicked. Now let's implement the part that tells the program *which* gadget caused the event. This is done by extracting WMHI_GADGETMASK):

```
1  ...
2  case WMHI_GADGETUP:
3    puts("gadget press");
4    switch (result & WMHI_GADGETMASK) // WHICH gadget caused the event?
5    {
6      // Button 1:
7      case 7:      // Button 1 ID
8        puts("Button 1 was clicked");
9        break;
10     // Button 2:
11     case 9:      // Button 2 ID
12       puts("Button 2 was clicked");
13     break;
14   }
15   break;
16 ...
```

Look at the lines *7* and *11*: Both Button gadgets are identified by there *unique ID*. This still is somewhat unsatisfying, since you will have to keep track of the ID values for each gadget you might want to create. With a GUI containing lots of gadgets, this can be confusing rather quick. Humans usually dislike remembering abstract numbers. So using IDs that don't tell us visible for what they stand might also be a source for coding bugs, as well.

A much better way would be to use *"speaking"* IDs that allow us to associate a meaning with them. Let's do so by defining "speaking" names and assign the gadget's ID to them. Place those definitions right after the last *#include instruction* in the source:

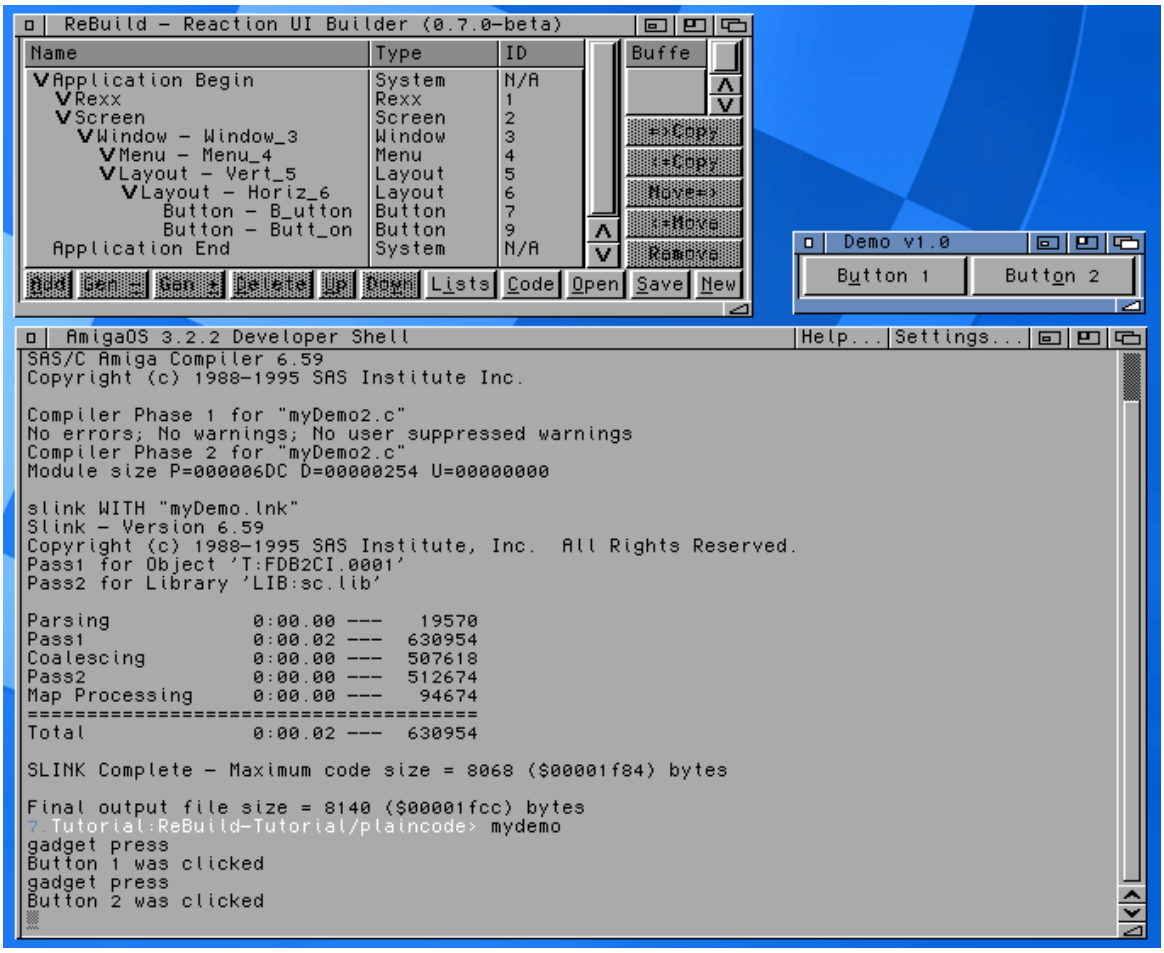
```
...
#include <classes/window.h>
#include <exec/memory.h>

#define BTN_BUTTON_1     7 // for first Button ID's value, see ReBuild's main window
#define BTN_BUTTON_2     9 // for second Button ID's value, see ReBuild's main window

case WMHI_GADGETUP:                // user clicked a gadget
  puts("gadget press");
  switch (result & WMHI_GADGETMASK) // WHICH gadget caused the event?
  {
    // Button 1:
    case BTN_BUTTON_1:       // Button 1 ID
      puts("Button 1 was clicked");
      break;
    // Button 2:
    case BTN_BUTTON_2:       // Button 2 ID
      puts("Button 2 was clicked");
      break;
  }
break;
```

Much easier to read, isn't it? A simple way of identifying a gadget's ID in order to define a "speaking" constant is to look up its value in ReBuild's main window. Another way would be to consult the *void window_3()* function and look it up in the layout section. Internally, as already mentioned those IDs are stored in the synchronized *win_gadgets[ ]* and *gadget_ids[ ]* arrays *(see page 39)*.

Now save your work as *myDemo2.c*, then compile and run it. Watch the message printed whenever you click a button!



3.2. Reacting on Menus - event loop part 2

ReBuild comes with a rather minimalist Menu assistant, providing the simplest form of a menu structure: Menus, Menu items and Sub items. If you need something more sophisticated, you will have to edit the generated code by hand.

3.2.1. Menus in theory

When talking about Menus, we talk about GadTools' NewMenu structure. It has a big advantage over classic Intuition menu structures: There's only *one* structure needed. Generally, it looks like this:

```
struct NewMenu
{
  UBYTE nm_Type;
  STRPTR nm_Label;
  STRPTR nm_CommKey;
  UWORD nm_Flags;
  LONG nm_MutualExclude;
  APTR nm_UserData;
};
```

or less abstract:

```
struct NewMenu mynewmenu[] =
{
  { NM_TITLE, "Some Title",     0 , 0, 0, 0,},              // a Menu
  {  NM_ITEM, "Some Menu item.",    "M", 0, 0, 0,},         // a Menu Item
  {  NM_ITEM, NM_BARLABEL,  0 , 0, 0, 0,},                  // a graphical divider
  {  NM_ITEM, "Some item with sub items", 0 , 0, 0, 0,},    // another Menu Item
  {   NM_SUB, "first sub item",  0 , 0, 0, 0,},             // a Sub Item
  {   NM_SUB, "second sub item",  0 , 0, 0, 0,},            // another Sub Item
  { NM_END, NULL, 0 , 0, 0, 0,},  // Used to signify the end of the NewMenu structure array
};
```

A typical entry consists of six fields:

1. Field *(STRPTR nm_Type)* – The first reflects the kind of entry and is one of
   a) NM_TITLE: creates a menu title, which then holds menu items. Used to signify a textual menu heading. Each NM_TITLE signifies the start of a new menu within the menu strip.
   b) NM_ITEM: A typical menu item. It can be standalone or holds sub items. Used to signify a textual menu heading. Each NM_TITLE signifies the start of a new menu within the menu strip
   c) NM_SUB: a sub item, must always have a "master" menu item. Used to signify a textual (NM_SUB) or graphical (IM_SUB) menu sub-item. All the consecutive NM_SUBs and IM_SUBs that follow a menu item (NM_ITEM or IM_ITEM) compose that item's sub-menu. A subsequent NM_ITEM or IM_ITEM would indicate the start of the next item in the original menu, while a subsequent NM_TITLE would begin the next menu.
2. Field – *(UBYTE nm_Label)* contains the text to be displayed (or NM_BARLABEL, if the entry is a divider).
3. Field – *(UBYTE nm_ComKey)* contains either a one-character text reflected as the shortcut for the entry or NULL. The nm_CommKey field is a pointer to a string and not a character itself. This was done in part because routines to support different languages typically return strings, not characters. The first character of the string is actually copied into the resulting MenuItem structure.

The nm_Flags field of the NewMenu structure corresponds roughly to the Flags field of the Intuition's lower-level Menu and MenuItem structures.

For programmer convenience the sense of the Intuition MENUENABLED and ITEMENABLED flags are inverted. When using GadTools menus, menu items and sub-items are enabled by default.

4. Field – *(UBYTE nm_Flags)* contains combination of flags for special abilities or NULL. This field is also used to specify checkmarked menu items. To get a checkmark that the user can toggle, set the CHECKIT and MENUTOGGLE flags in the nm_Flags field. Also set the CHECKED flag if the item or sub-item is to start in the checked state. That's the field of interest if you want checkmarked and mutually exclusive checkmarked menu items

5. Field – *(UBYTE nm_MutualExclude)* is for specifying mutual exclusion of checkmarked items. All the items or sub-items that are part of a mutually exclusive set should have the CHECKIT flag set

6. Field – *(UBYTE nm_UserData)* This data is stored with the Intuition Menu or MenuItem structures that GadTools creates.

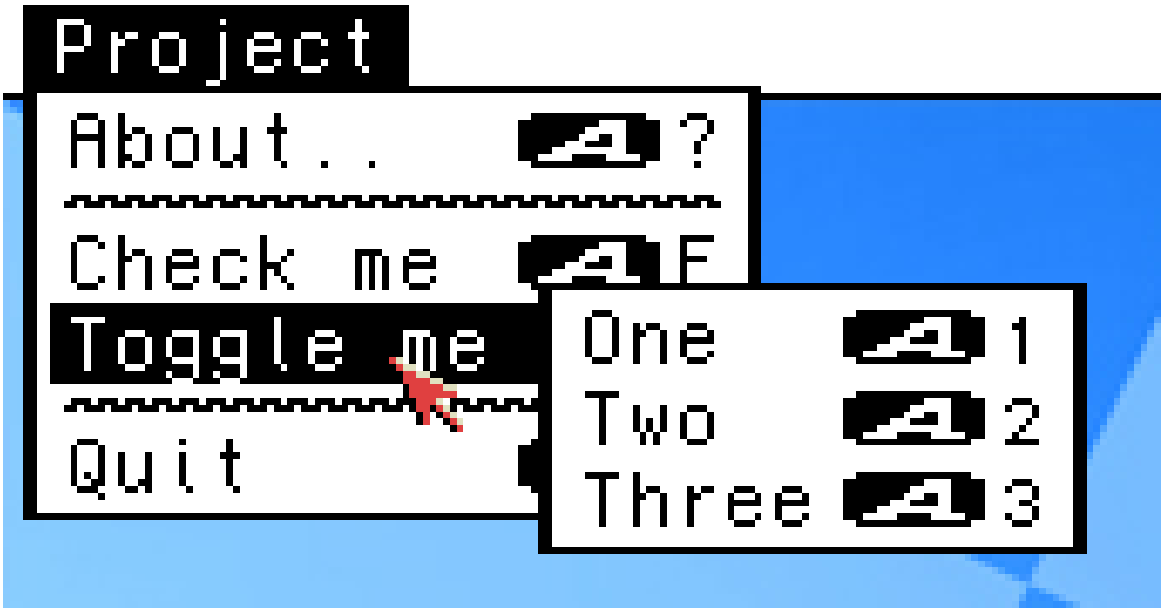
   To extract or change the user data fields of menus and menu items, respectively, use the macros GTMENU_USERDATA(menu) and GTMENUITEM_USERDATA(menuitem) defined in <libraries/gadtools.h>.

   The application may *place index numbers* in this field and perform a switch statement on them, instead of using the Intuition menu numbers. The advantage of this is that the numbers chosen remain valid even if the menus are rearranged, while the Intuition menu numbers would change when the menus are rearranged.

   Alternately, an efficient technique for menu handling is to create a handler function for each menu item and put a pointer to that function in the corresponding item's UserData field. When the program receives a IDCM_MENUPICK message it may call the selected item's function through this field.

3.2.2. Enhancing the project's menu

Let's come back to our *myDemo* project's menu *(refer to page 36)*:



In code, this is represented as

```
1  struct NewMenu menuData[] =
2  {
3    { NM_TITLE, "Project",0,0,0,NULL },    // Menu title
4    { NM_ITEM, "About..","?",0,0,NULL },   // first Menu Item
5    { NM_ITEM, NM_BARLABEL,0,0,0,NULL },   // divider
6    { NM_ITEM, "Check me","E",0,0,NULL }, // second Menu Item, shall become checkable
7    { NM_ITEM, "Toggle me",0,0,0,NULL },   // third Menu Item, holds 3 Sub Items for Mutex
8    { NM_SUB, "One","1",0,0,NULL },      // first Sub item, shall become mutual exclusive
9    { NM_SUB, "Two","2",0,0,NULL },      // second Sub item, shall become mutual exclusive
10   { NM_SUB, "Three","3",0,0,NULL },    // third Sub item, shall become mutual exclusive
11   { NM_ITEM, NM_BARLABEL,0,0,0,NULL },// divider
12   { NM_ITEM, "Quit","Q",0,0,NULL },    // fourth Menu Item
13   { NM_END, NULL, 0, 0, 0, (APTR)0 }
14 };
```

(see the first view lines of the *void window_3()* function)

We will now enhance our Menu:

1. Lets first make the second Menu Item *(line 6)* **checkable**. This is done by adding a *CHECKIT|MENUTOGGLE* flags combination to the *nm_Flags* field of the entry. If selected, it will have a visible CheckMark now.

2. Second, we will make the three Sub Items *(lines 8 to 10)* **mutual exclusive**. That's a bit more complicated than what we did to the second Menu Item: We start with adding a *CHECKIT | CHECKED* flags combination to the *nm_Flags* field of the first Sub Item. For the second and third Sub Items, we will add a *CHECKIT* flag only. If selected, a Sub Item now has a visual dot in order to show it's checked – the first Sub Item is already selected. In addition, we will have to define *what* shall be mutual excluded in the *nm_MutualExclude* field.

   Mutex bits: Doing Mutex menus means bit-shifting items to exclude each other in the MutualExlude field of the NewMenu structure. In the MutualExclude field, bit 0 refers to the first item in the item list, bit 1 to the second, bit 2 to the third, and so on.

   If we have 4 items to become mutex'ed, the first one has bits 1111 bin = 15 decimal. (if we have 5 items to become mutex'ed, the first one has bits 11111 bin = 31 decimal, and so on)

   Setting a value of 15 to the MutualExlude field would result in ALL entries being unchecked, which we don't want. To archive all others but the first one become unchecked, we need to provide a value of 1110 bin = 15 - 1 decimal. Equivalent for the second entry will then be 15 - 2 = 1101 bin. That unchecks all but the second item and sets it to CHECKED status. That behavior in a similar way applies to all items to be mutex'ed.

   Subtracting from the base of the first item is done by raising the subtrahend by the power of 2 with each following item that should be mutex'ed. For the first this is -1, for the second -2, for the third -4, and so on. So the third item will be 15 - 4 (1011 bin) and for the fourth item it will be 15 - 8 (0111 bin). So we achieve a bit pattern like this: uncheck all: 1111 First item: 1110 - check 1, uncheck 2 to 4 Second item: 1101 - check 2, uncheck 1, 3 and 4 Third item: 1011 - check 3, uncheck 1, 2 and 4 Fourth item: 0111 - check 4, uncheck 1 to 3

   The recent status of an item that is checkable can then be obtained by checking its status flag by 'menu-item.flags & CHECKED' in the event loop in function *run_window()*.

   In *myDemo.c* we have only three items that shall become mutex'ed. That results in binary 111 = decimal 7. With that, *all* Sub Items would be unchecked, so we need a start value of binary 110 = decimal 6. A Pattern for our three Sub Items would look like this:

```
111 = 7
110 = 6
101 = 5
011 = 3

We can break it down to decimal
7 - 1 (first Sub Item)
7 - 2 (second Sub Item)
7 - 4 (third Sub Item)
```

... and that's the values we will set to the *nm_MutualExclude* fields of our entries!

3. Last, but not least we need a way of identifying each Menu entry – a *unique ID*, just like with gadgets, too. That's where the *nm_UserData* field for each entry comes in: We could simply note a number for each entry that we want to be identified there. Anyway, as with gadgets, it's good practice to have "speaking" constants for menus, too!

   In order to have them, we will declare an enumeration, placing it under the definition of constants for our gadgets:

   ```
   /* —— Menu identifiers —— */
   enum menus { // Menu Project
     MN_ABOUT, MN_CHECKME,
     MN_TOGGLE_ONE, MN_TOGGLE_TWO, MN_TOGGLE_THREE,
     MN_QUIT,
   };
   ```

   Note how we only define values for needed entries! Those constants need to be placed in the *nm_UserData* field of corresponding entries.

With all changes done, our Menu structure now looks like this:

```
struct NewMenu menuData[] =
{
  { NM_TITLE, "Project",0,0,0,NULL },
  { NM_ITEM, "About..","?",0,0,(APTR)MN_ABOUT },
  { NM_ITEM, NM_BARLABEL,0,0,0,NULL },
  { NM_ITEM, "Check me","E",CHECKIT|MENUTOGGLE,0,(APTR)MN_CHECKME },
  { NM_ITEM, "Toggle me",0,0,0,NULL },
  { NM_SUB, "One","1",CHECKIT|CHECKED,7 - 1,(APTR)MN_TOGGLE_ONE },
  { NM_SUB, "Two","2",CHECKIT,7 - 2,(APTR)MN_TOGGLE_TWO },
  { NM_SUB, "Three","3",CHECKIT,7 - 4,(APTR)MN_TOGGLE_THREE },
  { NM_ITEM, NM_BARLABEL,0,0,0,NULL },
  { NM_ITEM, "Quit","Q",0,0,(APTR)MN_QUIT },
  { NM_END, NULL, 0, 0, 0, (APTR)0 }
};
```

Now save your work as *myDemo3.c*, then compile and run it. Watch the Menu Items and Sub Items when being selected!

3.2.3. Making the project's Menu work

As with Gadgets, Menu events are triggert by IDCMP *(respectively WMHI under ReAction)* events, too. This involves the project's *runWindow()* function again. Already implemented in the event loop is questioning *if* some kind of Menu event was triggered. You will find this under *"case WMHI_MENUPICK:"*

```
case WMHI_MENUPICK:    // a menu event occured
  puts("menu pick");
  break;
```

As said, the event reports *that* a menu event was triggered – not *which* menu entry did it. This is because the outer conditional loop of the event loop is checked against *WMHI_CLASSMASK*, which gives us the Class of an event: Gadgets, Menus, whatever...

In order to determine *which* menu caused an event, we will need to know its *ID* (stored in the nm_UserData field) and the *item's address*. We'll get that by checking against *WMHI_MENUMASK*. A full event loop for menu events for our *myDemo* project would look like

```
1
2   /* —— Event Loop —— */
3   while ( !done)
4   {
5     wait = Wait( signal | SIGBREAKF_CTRL_C );
6
7     if ( wait & SIGBREAKF_CTRL_C )
8     done = TRUE;
9     else
10    while (( result = RA_HandleInput( window_object, &Code )) != WMHI_LASTMSG)
11    {
12      switch ( result & WMHI_CLASSMASK )  // What kind of event occured?
13      {
14        /* —— if user selected a menu entry... ——*/
15        case WMHI_MENUPICK:
16        puts("\nYou selected some menu:");
17        /* —— ...get menu entries by using GadTools' GTMENUITEM_USERDATA field ———— */
18        switch ((long)GTMENUITEM_USERDATA(ItemAddress(menu_strip, result & WMHI_MENUMASK)))
19        {
20          case MN_ABOUT:
21          puts("About");
22          break;
23          case MN_CHECKME:
24          puts("check me");
25          break;
26          case MN_TOGGLE_ONE:
27          puts("One");
28          break;
29          case MN_TOGGLE_TWO:
30          puts("Two");
31          break;
32          case MN_TOGGLE_THREE:
33          puts("Three");
34          break;
35          case MN_QUIT:
36          puts("Quit");
37          done = TRUE;
38          break;
39        }
40        break;
41      }
42    }
43  }
```

Now please adapt your event loop, copying lines 17 to 39 directly under "case WMHI_MENUPICK:" of the *myMenu3.c* source. Save as *myDemo4.c*. Compile it and watch the output of selected menus!

3.2.4. Getting the status of checkable Menu Items

So far – so good. We now have a working app that at least reacts kind of adequate on gadgets and menus. But what is the use of a checkable menu item if you don't know if it's checked or not? Usually, you would want to react on such an item depending on its recent status, won't you?

While with *mutex'ed* Menu Items, the case is clear – they usually only need to be identified in order to do something usefull – this is not the case with *checkable* items. Here, you will need to query the *status* in addition to identifying such a Menu Item. This can be achieved by querying if the *CHECKED* flag is set in the Menu Item's corresponding *MenuItem* structure. It will be a bit of work, though:

1. We need to initialize a *struct MenuItem ** pointer in function *runWindow()*:

```
void runWindow( Object *window_object, int window_id, struct Menu *menu_strip,
                struct Gadget *win_gadgets[], int gadget_ids[] )
{
  struct Window *main_window = NULL;
  // we need a MenuItem struct to keep track on status informations:
  struct MenuItem *menuitem = NULL; // NEW: Initialize struct MenuItem *menuitem
                                    // Among others, this struct holds the status
                                    // of the CHECKED flag for a Menu Item
  ...
```

2. In case of WMHI_MENUPICK, the structure pointer must be updated:

```
...
case WMHI_MENUPICK:
  puts("\nYou selected some menu:");
  menuitem = ItemAddress(menu_strip, result & WMHI_MENUMASK); // NEW: update menuitem

  // IMPORTANT! menuitem == 0 would result in enforcer hit!
  // This might happen if user right-clicks a menu, but
  // does not select anything:
  if(!(menuitem == 0))  // only switch if menutitem isn't ZERO!
  {
    /* ---- ...get menu entries by using GadTools' GTMENUITEM_USERDATA field ---------- */
    switch ((long)GTMENUITEM_USERDATA(ItemAddress(menu_strip, result & WMHI_MENUMASK)))
    {
      case MN_ABOUT:
        puts("About");
        break;
      ...
    }
  }
...
```

3. We need to query if *CHECKED* or not:

```
...
case MN_CHECKME:
  puts("check me");
  /* NEW: evaluate, if item has checkmark set... */
  if(menuitem->Flags & CHECKED)
  {
    puts("MN_CHECKME is CHECKED!");
  }
  else
    puts("MN_CHECKME is not CHECKED!.");
```

```
        break;
    ...
```

That's it! Now update the source, save as *myDemo5.c*, compile and test the app's output.

3.3. Implementing ReAction Requesters

Implementing a ReAction requester.class Requester is one thing ReBuild doesn't do for you – you are on your own here. Anyway, that's no problem: As with any other gadget, image or class, you will first have to include the appropriate header files, then define a base for the class, afterwards open it, do your stuff and finally close it:

1. Add the appropriate header files:

```
...
#include <proto/layout.h>
#include <proto/button.h>
#include <proto/requester.h>     // NEW
#include <classes/requester.h>   // NEW
...
```

2. Add RequesterBase:

```
...
struct Library *WindowBase = NULL,
               *ButtonBase = NULL,
               *RequesterBase = NULL,   // NEW
               *GadToolsBase = NULL,
               *LayoutBase = NULL,
               *IconBase = NULL;
struct IntuitionBase *IntuitionBase = NULL;
...
```

3. In function *setup()*, open requester.class:

```
...
if( !(LayoutBase = (struct Library*) OpenLibrary("gadgets/layout.gadget",0L) ) )
  return 0;
if( !(ButtonBase = (struct Library*) OpenLibrary("gadgets/button.gadget",0L) ) )
  return 0;
if( !(RequesterBase = (struct Library*) OpenLibrary("requester.class",0L) ) ) // NEW
  return 0;
if( !(gScreen = LockPubScreen( 0 ) ) ) return 0;
...
```

4. In function *cleanup()*, close requester.class:

```
...
if (IconBase) CloseLibrary( (struct Library *)IconBase );
if (IntuitionBase) CloseLibrary( (struct Library *)IntuitionBase );
if (RequesterBase) CloseLibrary( (struct Library *)RequesterBase );    // NEW
if (ButtonBase) CloseLibrary( (struct Library *)ButtonBase );
...
```

5. Do your stuff: We will use the custom function **doInfoRequest()** *(see pages 96 ff)* in order to implement a security query requester that appears if the user selects MN_QUIT Menu item:

   a) Declare a Prototype for the function:

```
...
// Function Prototypes:
void window_3( void );
int doInfoRequest(struct Screen *screen, const char *title,
                  const char *body,
                  const char *buttons, ULONG image );   // NEW
...
```

b) Copy the *doInfoRequest()* function to some appropriate position in your source code. A good position would be *after* all global assignments, but *before* all other functions:

```
...
struct Library *WindowBase = NULL,
               *ButtonBase = NULL,
               *RequesterBase = NULL,
               *GadToolsBase = NULL,
               *LayoutBase = NULL,
               *IconBase = NULL;
struct IntuitionBase *IntuitionBase = NULL;

/* copy doInfoRequest() function HERE! */
...
```

c) In function *runWindow()* define a variable for doInfoRequest()'s return value:

```
...
WORD Code;
ULONG wait = 0, signal = 0, result = 0, done = FALSE;
ULONG req_result; // NEW
...
```

d) Also in function *runWindow()*, replace the "case MN_QUIT:" part by

```
case MN_QUIT:
  SetAttrs(window_object,
           WA_BusyPointer, TRUE, TAG_DONE);  // block calling window

  // call Requester:
  req_result = doInfoRequest(NULL, "Quit myDemo",
  " Do you really want to end this disaster?" \
  "\nYou'll have much more time for other hobbies!",
  "_Quit|_Cancel", REQIMAGE_WARNING);

  if (req_result == 1)
    done = TRUE;

  SetAttrs(window_object,
           WA_BusyPointer, FALSE, TAG_DONE); // de-block calling window
  break;
```

You've done it again! Save as *myDemo7.c*, compile and test the app.

3.4. Implementing ToolTips

The Amiga StyleGuide recommends the use of ToolTips. Therefore a nice-to-have feature of ReAction's window.class is the ability of displaying help texts for gadgets. If the mouse points to a gadget of an active window, a pre-defined short description of that gadget may be displayed, depending on *WINDOW_GadgetHelp* set to TRUE and *WINDOW_HintInfo* pointing to a *HintInfo* structure when defining the Tags for a window. With ReBuild's generated code, this happens in function *window_xyz()*. The HintInfo structure in **classes/window.h** looks like this:

```
struct HintInfo
{
  WORD hi_GadgetID; /* Gadget ID this hint belongs to, -1 indicates end. */
  WORD hi_Code;     /* Code required for this gadget, or -1 if it doesn't matter. */
  STRPTR hi_Text;   /* The text of the hint.. */
  ULONG hi_Flags;   /* Flags, currently unsupported and must be set to 0L. */
};
```

With our *myDemo* project, in the global instructions section of the source we implement it as

```
...
/* -- GadgetHelp ToolTips -- */
struct HintInfo hintinfo[] =
{
  { BTN_BUTTON_1, -1, "I am Button One, here to bore\n you with information...", 0L},
  { BTN_BUTTON_2, -1, "I'm Button One's younger brother. \nWhat he does is good.", 0L},
  { -1, -1, NULL, 0L}   // terminate!
};
...
```

Note how it's totally legal to use the *newline* escape code in hi_Text's string. Use it to distribute longer help text over several lines!

Now we just need to tell the window to make use of it. This is done by window Tags in function *window_3()*:

```
...
WINDOW_LockHeight, TRUE,
WINDOW_IconifyGadget, TRUE,
WINDOW_HintInfo, &hintinfo, // Add me!
WINDOW_GadgetHelp, TRUE,    // Add me if not already there!
...
```

All done! Save as *myDemo8.c*, compile and test the app.

3.5. Querying the status of a CheckBox

A lot of a coder's daily bread and butter when programming BOOPSI/ReAction applications is about querying and setting an object's attributes. Usually, querying is done using Intuition's ***GetAttr()***, while for setting attributes you will use ***SetGadgetAttrs()*** functions, defined as

GetAttr():

```
/* ------------------------------------------------------------------------
SYNOPSIS
attr = GetAttr( AttrID, Object, StoragePtr )

ULONG GetAttr( ULONG, APTR, ULONG * );

FUNCTION
Inquires from the specified object the value of the specified attribute.

You always pass the address of a long variable, which will
receive the same value that would be passed to SetAttrs() in
the ti_Data portion of a TagItem element.  See the documentation
for the class for exceptions to this general rule.

Not all attributes will respond to this function.  Those that
will are documented on a class-by-class basis.

INPUTS
AttrID = the attribute tag ID understood by the object's class
Object = abstract pointer to the boopsi object you are interested in
StoragePtr = pointer to appropriate storage for the answer

RESULT
Returns FALSE (0) if the inquiries of attribute are not provided
by the object's class.

NOTES
This function invokes the OM_GET method of the object.
*/ ------------------------------------------------------------------------

// Example:
ULONG signal;
GetAttr( WINDOW_SigMask, window_object, &signal );
```

SetGadgetAttrs():

```
/* ------------------------------------------------------------------------
SYNOPSIS
result = SetGadgetAttrsA( Gadget, Window, Requester, TagList )
Definition:
ULONG GetAttr( ULONG attrID, APTR object, ULONG *storagePtr );
Same as SetAttrs(), but provides context information and
arbitration for classes which implement custom Intuition gadgets.

You should use this function for boopsi gadget objects which have
already been added to a requester or a window, or for "models" which
propagate information to gadget already added.

Typically, the gadgets will refresh their visuals to reflect
changes to visible attributes, such as the value of a slider,
the text in a string-type gadget, the selected state of a button.

You can use this as a replacement for SetAttrs(), too, if you
specify NULL for the 'Window' and 'Requester' parameters.
```

```
INPUTS
Gadget = abstract pointer to a boopsi gadget
Window = window gadget has been added to using AddGList() or AddGadget()
Requester = for REQGADGETs, requester containing the gadget
TagList = array of TagItem structures with attribute/value pairs.

RESULT
The object does whatever it wants with the attributes you provide,
which might include updating its gadget visuals.

The return value tends to be non-zero if the changes would require
refreshing gadget imagery, if the object is a gadget.
*/ ------------------------------------------------------------------------
// Example:
ULONG success;
success = SetGadgetAttrs((APTR)txtgad, win, 0, STRINGA_TextVal, isChecked, TAG_END );
```

3.5.1. Implementing CheckBox status query

Let's load the CheckBox demo that we created at the beginning of this tutorial. We want the app to put the recent status of a CheckBox gadget as an informative text into its corresponding String gadget. Since *SetGadgetAttrs()* needs details that can't easily be provided in another way, we will be using a custom function *(see page 87 ff)* in order to implement that.

First we will have to adapt the source according to the information provided on page 87 ff):

1. Change function *runWindow*'s function header to:

```
void runWindow( Object *window_object, int window_id,
                struct Menu *menu_strip,
                struct Gadget *win_gadgets[] )
```

2. Copy the ***getCheckBoxStatus()*** function into *myDemo7.c*. A good position would be *after* the global instruction section, but *before* the first function definition found.

3. Add its function header to the list of function prototypes, followed by a semicolon.

4. in your window() function (usually *window_3()*), find the call to *runWindow()* and replace it by

```
// for first window: "3" identifies first window's ID...
runWindow( window_object, 3, menu_strip, main_gadgets );
```

5. Identify the particular gadget index used in main_gadgets[ ] – you can find them in the index of the particular object's main_gadgets[ ] layout definitions. Either keep them in mind, or create "speaking" constants for them, e.g.

```
...
// Example "speaking" index constants for main_gadgets[] array:
#define GADS_CB_ENE    3
#define GADS_CB_MENE   4
#define GADS_CB_MUH    5
#define GADS_STR_ENE   7
#define GADS_STR_MENE  8
#define GADS_STR_MUH   9
```

Next, we will alter the behavior of the event loop, since this doesn't identify which gadget was clicked up to now:

6. In the global instructions section of the source, define "speaking" constants for each main_gadgets[index] and gadget, assigning the appropriate value to them:

```
...
/* "speaking" main_gadgets[] index constants */
#define GADS_CB_ENE    3
#define GADS_CB_MENE   4
#define GADS_CB_MUH    5
#define GADS_STR_ENE   7
#define GADS_STR_MENE  8
#define GADS_STR_MUH   9
/* "speaking" gadget ID constants */
#define CB_ENE       29
#define CB_MENE      30
#define CB_MUH       31
#define STR_ENE      33
#define STR_MENE     34
#define STR_MUH      35
#define BTN_STATUS   37
...
```

7. In its *runWindow()* function, replace "case WMHI_GADGETUP: ... break;" with

```
case WMHI_GADGETUP:
switch (result & WMHI_GADGETMASK) // WHICH gadget caused the event?
{
  // CheckBox 1:
  case CB_ENE:
    puts("CheckBox 1 was clicked");
    // example call using "speaking" index constants:
    getCheckBoxStatus(main_window, win_gadgets[GADS_CB_ENE],
                      win_gadgets[GADS_STR_ENE]);
    break;
  // CheckBox 2:
  case CB_MENE:
    puts("CheckBox 2 was clicked");
    // example call using "speaking" index constants:
    getCheckBoxStatus(main_window, win_gadgets[GADS_CB_MENE],
                      win_gadgets[GADS_STR_MENE]);
    break;
  // CheckBox 3:
  case CB_MUH:
    puts("CheckBox 3 was clicked");
    // example call using "speaking" index constants:
    getCheckBoxStatus(main_window, win_gadgets[GADS_CB_MUH],
                      win_gadgets[GADS_STR_MUH]);
    break;
  // Button:
  case BTN_STATUS:  // has no real functionality anymore!
    puts("Status Button was clicked,\nbut you won't see any difference.");
    // example calls, using index numbers
    getCheckBoxStatus(main_window, win_gadgets[3], win_gadgets[7]);
    getCheckBoxStatus(main_window, win_gadgets[4], win_gadgets[8]);
    getCheckBoxStatus(main_window, win_gadgets[5], win_gadgets[9]);
    break;
}
break;
```

Note that BTN_STATUS condition has no real functionality anymore, since all the status text setting for the corresponding String gadget is already done when clicking a CheckBox! It is only left over for demonstration purpose.

We are done with enhancing *myDemo.c*. **Save your work as *myDemo9.c***, compile and test the app.

By now, you should be able to determine which gadget was clicked, implement requester.class informative requesters, ToolTips and Menus, query their status and the status of a CheckBox gadget, too.

If you would like to practice those abilities, switch to next page and absolve the exercises described there.

3.6. Exercises

1. myDemo.c - Implement the security query for the *WMHI_CLOSEWINDOW* event, too.
2. myDemo.c - Implement an "About" Requester for the *MN_ABOUT* event. It should have only one button and display the “Information” icon.
3. myDemo.c - For both buttons, implement a requester that contains an individual message.
4. cbdemo.c - enhance the menu with Menu Item identification
5. cbdemo.c - implement “About” and “Quit” requesters
6. cbdemo.c - implement GadgetHelp for all CheckBoxes and the Button gadget

For example solutions, refer either to Appendix H.1 and Appendix H.2 or to the sources found in ***cbDemo*** and ***myDemo*** example project folders shipped with this tutorial!

Part III

Appendices

A. C compiler setup

A.1. Setting up a C compiler for NDK 3.2

Both the **SAS/C v6.58** compiler suite and **vbcc** (Dr. Volker Bartelmann's portable ISO C compiler) are known to work well with code generated by ReBuild. However, a bit of work has to be done before you can use one of these with the AmigaOS 3.2 NDK.

A.1.1. SAS/C setup

Some years after Commodore went under, SAS Institute stated the compiler as *discontinued* and does not support it any longer. As of January 31, 1998, SAS Institute was out of stock of the SAS/C 6.50 Amiga Compiler.

It is a bit of a gray zone, but SAS/C Version 6 still is one of the best and recommendable C-compilers for classic AmigaOS development until today and may eventually be viewed as abandon ware. Though it is not sold anymore, you may find it second hand or on the net. The extensive manual is a must read and comes in two large volumes –

Vol. 1: https://archive.org/details/sasc-650-vol1

Vol. 2: https://archive.org/details/sasc-650-vol2

SAS/C is the weapon of choice for this tutorial – all code examples were compiled and tested under SAS/C v6.58 – if you bought it back in the day or can get hold on a copy: Use it. Most devs do.

Installing SAS/C

Install SAS/C by using the included installer. Afterwards reboot. Now get all available official patches from Aminet and apply them according to their README files.

Patch 1: http://aminet.net/dev/c/sc655pch.lha

Patch 2: http://aminet.net/dev/c/sc656pch.lha

Patch 3: http://aminet.net/dev/c/sc657pch.lha

Patch 4: http://aminet.net/dev/c/sc658pch.lha

Patch 5: http://aminet.net/dev/c/SASC_Y2k_patch.lha

Backing up SAS/C include: and lib: folders

In order to be able to switch back to an AmigaOS 3.1 development environment, you might want to backup the original folders containing includes and link libs (just in case, though this step is not forcefully needed):

```
copy sc:include all to sc:include31
copy sc:lib all to sc:lib31
```

Deleting unneeded files

The files which need to be replaced are found in the directories pointed to by the "lib:" and "include:" assignments. Start with the files which are used by the compiler and its assembler and delete only those which will be replaced by the 3.2 header files:

```
delete all include:clib
delete all include:datatypes
delete all include:devices
delete all include:diskfont
delete all include:dos
delete all include:exec
delete all include:gadgets
delete all include:graphics
delete all include:hardware
delete all include:intuition
```

```
delete all include:libraries
delete all include:pragmas
delete all include:prefs
delete all include:proto
delete all include:resources
delete all include:rexx
delete all include:utility
delete all include:workbench
delete include:all.gst
delete include:all.gst.info
```

Replacing files

Now change the directory to where you placed the NDK's drawer and copy the replacement files from "NDK3.2" to the "include:" and lib: directories:

```
copy NDK3.2/Include_H/#? include: all
copy NDK3.2/Include_I/#? include: all
copy NDK3.2/lib/#?.lib lib:
```

Note that the "include:all.gst" file is deleted as a precaution so that you do not use it by mistake. It contains all the old Kickstart/Workbench 3.1 header files in a precompiled form, which can be helpful to shorten compilation time. This will not work any longer, since we just replaced all the 3.1 header files with the 3.2 versions.

It is recommended not to use "include:all.gst" file but to generate that precompiled global symbol table file in each of your projects as needed. The compiler options you use to generate these header files may have an impact on what is generated, and that may not be a file which works well with a different set of compiler options.

Testing the installation

Let's enter a small C program to verify the procedure. Start any text editor of your choice and create a small C program for demonstration purposes:

```c
#include <dos/dos.h>
#include <proto/dos.h>

int main()
{
  Printf("Hello, World!\n");
  return(RETURN_OK);
}
```

Save as *hello.c* and compile with: *sc hello.c LINK TO hello* – afterwards run the program. The program uses the Printf() function of dos.library instead of the compiler-supplied printf(). So you'll be fine, if it compiles successfully. If it prints out *Hello, World!* all is fine. You're ready to use SAS/C with NDK 3.2 now.

A.1.2. vbcc setup

vbcc is a highly optimizing portable and retargetable ISO C compiler by Volker Barthelmann. It supports ISO C89 and a subset of C99. It's also still being developed and offers a variety of target distributions and cross-compiling options. Best of all: It's free. The AmigaOS distribution of vbcc is maintained by Frank Wille, who added support for the NDK3.2 in vbcc 0.9h, so you'll have to get this version or a later one.

Installing vbcc

Installation of the compiler is simple: You'll have to download the vbbc main archive suitable for your system plus at least one Amiga-related target archive, first. We'll assume you want to compile on Amiga natively, so we choose:

Main archive: http://phoenix.owl.de/vbcc/current/vbcc_bin_amigaos68k.lha

Target archive: http://phoenix.owl.de/vbcc/current/vbcc_target_m68k-amigaos.lha

Not really needed for our purpose, but a nice-to-have if you plan to develop for AmigaOS 1.3, too:

http://phoenix.owl.de/vbcc/current/vbcc_target_m68k-kick13.lha

Note that the installation of this archive is not covered here.

Open a shell. We assume you have *lha* installed. If this is not the case, get it from AmiNet by searching for "lha.run". Extract both archives to a temporary location, e.g. RAM:

```
lha x vbcc_bin_amigaos68k.lha ram:
lha x vbcc_target_m68k-amigaos.lha ram:
```

Install the NDK by extracting the archive to the desired location, e.g.:

```
lha x NDK3.2.lha Work:AmigaOS/NDK3.2
```

Now run the Installer script for both vbcc's main archive and target archive via Workbench. During installation of the target, a directory request pops up and you'll be asked for the location of the NDK's header files. Enter the directory you installed the NDK3.2 in and select the sub directory "include_h".

Editing S:User-Startup

Depending on the location you have chosen during install, an assign named vbcc: will be created. In our example, we installed vbcc to "Work:Develop/vbcc" and the NDK3.2 to "Work:AmigaOS/NDK3.2", so your paths may be different. Edit your S:User-Startup file using a text editor, and make sure it contains the following:

```
;BEGIN vbcc
assign >NIL: vbcc: Work:Develop/vbcc
assign >NIL: C: vbcc:bin ADD
setenv VBCC vbcc:
assign >NIL: vincludeos3: vbcc:targets/m68k-amigaos/include
assign >NIL: vincludeos3: Work:AmigaOS/NDK3.2/include_h ADD
assign >NIL: vlibos3: vbcc:targets/m68k-amigaos/lib
assign >NIL: vlibos3: Work:AmigaOS/NDK3.2/lib/ add
;END vbcc
```

The line "assign >NIL: vlibos3: Work:AmigaOS/NDK3.2/lib/ add" *(8)* won't be created by vbcc installer, so you'll have to add it yourself if you want to compile programs making use of debug.lib! Reboot your Amiga now.

Testing the installation

Let's enter a small C program to verify the procedure. Start any text editor of your choice and create a small C program for demonstration purposes:

```
#include <dos/dos.h>
#include <proto/dos.h>

int main()
{
  Printf("Hello, World!\n");
  return(RETURN_OK);
}
```

Save as *hello.c* and compile with: *vc +aos68k -o=hello hello.c* – afterwards run the program. The program uses the Printf() function of dos.library instead of the compiler-supplied printf(). So you'll be fine, if it compiles successfully. If it prints out *Hello, World!* all is good. You're ready to use vbcc with NDK 3.2 now.

A.2. Sources notice

Source for third-party information: ***AmigaOS Development Blog***
https://aosdg.blogspot.com/2021/06/ndk-setup-for-sasc-compiler.html
https://aosdg.blogspot.com/2022/01/ndk-setup-for-vbcc-compiler.html

B. Recommended readings

B.1. The AmigaOS 3.2 Native Developer Kit

The NDK contains autodocs and include files, but also examples and some documentation.
Get it from the official Hyperion website or from Aminet:
http://aminet.net/package/dev/misc/NDK3.2

B.2. Amiga ROM Kernel Reference Manual: Libraries

This is the first part of the Technical Reference Series by Commodore covering Release 2 of AmigaOS. and it comes with our highest recommendations. This is without a doubt the most important book to have. It deals with every library of the 2.04 except dos.library, which is covered in The AmigaDOS Manual listed below. It is not a holistic tutorial, but as the title suggests a reference manual. It is however written in an explanatory manner with for example a whole chapter about Exec Lists and contains many examples written in C. It can be found here:
https://www.retro-commodore.eu/download.php?file=Commodore_Amiga_Tech_Ref_Series_Amiga_ROM_Kernel_Reference_Manual_Libraries_3rd_edition.pdf

B.3. Amiga ROM Kernel Reference Manual: Devices

This is the second part of the Technical Reference Series by Commodore covering Release 2 of AmigaOS and it comes with our highest recommendations. This book is similar except it deals with devices instead of libraries. As such it relies on subjects like Exec Lists described in the first book. Useful for writing drivers for expansions or controlling keyboard, sound and printers among others. It can be found here:
https://www.retro-commodore.eu/download.php?file=Commodore_Amiga_Tech_Ref_Series_Amiga_ROM_Kernel_Reference_Manual_Devices_3rd_edition.pdf

B.4. ROM Kernel Reference Manual: Changes & Additions

This is an unofficial continuation of the above two books. It is a *work in progress* and only available as pdf for now. Aims to cover everything that happened since the original books were published. Despite being work in progress it already provides a lot of insight and is thus highly recommended already. Get it and be sure to come back several times a year for new chapters:
https://www.retro-commodore.eu/rkrm-wip/

B.5. Amiga User Interface Style Guide

Historically underrated, this book covers the essentials of what a programmer should take care of when designing a user interface. Although GUI design even on the Amiga somewhat evolved since the book was written, you should read it at least once to be aware of typical mistakes as well as standards a GUI should provide to the user. It can be found here:
https://www.retro-commodore.eu/download.php?file=Commodore_Amiga_Tech_Ref_Series_Amiga_User_Interface_Style_Guide.pdf

B.6. The AmigaDOS Manual

While not part of the Technical Reference Series it is still official documentation by Commodore, and the only official book to cover dos.library. Definitely something to have in your library, although it is known to contain

some errors. It can be found here:
https://www.retro-commodore.eu/download.php?file=Bantam_The_AmigaDOS_Manual_3rd_Edition.pdf

B.7. The Amiga Guru Book

by Ralph Babel – this is the *one* most informative book, containing lots of information about Amiga Programming in general and is the most in-depth resource dealing with AmigaDOS. Some of its knowledge can't be found elsewhere, not even in the official documentation. A must-read for any Amiga programmer. Unfortunately this book is print only and is highly sought after. Ralph even did an actualized second edition, but some jerk ripped and published his first version shortly before the second edition's official release. Ralph therefore stopped all activities concerning "The Guru Book". If you are *really* lucky, you might get hands on the first edition via flea markets or online sales platforms. If you find a hard copy of this book - get it immediately!

B.8. Sources information

Source for third-party information: ***AmigaOS Development Blog***
https://aosdg.blogspot.com/2022/03/recommended-reading-for-amiga-developer.html

C. Readings on C coding

C is probably the most used programming language for Amiga system programming. Therefore it's good to have a fundamental knowledge about coding in C. Whether you are already a pro or a newbie starting with C - here you will find some useful material about the C language.

C.1. General books about C

A starting point for beginners and pros as well

C.1.1. The C Programming Language - ANSI C

The C Programming Language - ANSI C, second edition by *Brian W. Kernighan* and *Dennis M. Ritchie* is a must-read for every serious C programmer. In this book, the inventors of the language explain all about ANSI C.

https://kremlin.cc/k&r.pdf

C.1.2. The C Answer Book

The C Answer Book by *Clovis L. Tondo* and *Scott E. Gimpel* provides model answers for the exercise tasks from "The C Programming Language - ANSI C, second edition".

https://seriouscomputerist.atariverse.com/media/pdf/book/C%20Answer%20Book.pdf

C.1.3. Algorithms in C

Algorithms in C by *Robert Sedgewick* deals with implementing various algorithms in C.

https://seriouscomputerist.atariverse.com/media/pdf/book/Algorithms%20in%20C.pdf

C.2. Amiga-related C readings

Learning C on the Amiga is learning about dealing with Amiga data structures, libraries, devices and GUI frameworks. Unfortunately, most available beginner's literature on the topic relies on AmigaOS 1.3 and therefore is a bit outdated. Nevertheless, it might be useful to even know how things were done back in those old days. For more recent information on coding under AmigaOS 2.04 and up, please refer to appendix B.

C.2.1. Amiga C for Beginners

Amiga C for Beginners by *Dirk Schaun* from 1990 provides basic knowledge about programming the Amiga in C.

https://archive.org/details/1990-schaun-dirk-amiga-c-for-beginners

C.2.2. Inside the Amiga with C

Inside the Amiga with C by *John Thomas Berry*, second edition from 1989 deals with programming the Amiga using Lattice C (now: SAS/C).

https://www.retro-commodore.eu/download.php?file=SAMS_Inside_the_Amiga_with_C_2nd_edition.pdf

C.2.3. Inside Amiga Graphics

not strictly for C coders only, this book by *Sheldon Leemon* provides useful information on Amiga Screen and Blitter programming.

https://www.retro-commodore.eu/download.php?file=Compute%21s_Inside_Amiga_Graphics.pdf

D. Readings on E coding

Amiga E is a programming language created by Wouter van Oortmerssen on the Amiga computer. The work on the language started in 1991 and was first released in 1993. The original incarnation of Amiga E was being developed until 1997. Amiga E combines features from several languages but follows the original C programming language most closely in terms of basic concepts. Amiga E's main benefits are fast compilation and Object-oriented programming. Beginning in 2021, *Darren Coles* continued the development of Amiga E, based on the work on others before him. His excellent E-VO compiler was used for developing ReBuild.

Since ReBuild besides C also is capable of producing E code, maybe you might be interested in learning how to code in that language? Well – here are some basic tutorials to start with.

D.1. A Beginner's Guide to Amiga E

Online tutorial on E coding: https://cshandley.co.uk/JasonHulance/

D.2. Amiga E manual v3.3a

Online manual for Wouter's Amiga E: https://cshandley.co.uk/amigae/

E. BOOPSI ReAction classes, gadgets and images

Let's have a closer look at the various BOOPSI classes and gadgets that ReAction consists of!

E.1. Classes

Classes reside in SYS:Classes and are opened with

```
<Class>Base = (struct Library *) OpenLibrary(
"<classname>.class",
<version>
);
```

E.1.1. arexx.class

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.1.2. requester.class

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:
See also: this document, page 96 and page 53 ff.

E.1.3. window.class

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2. Gadgets

Gadgets reside in SYS:Classes/Gadgets and are opened with

```
<Gadget>Base = (struct Library *) OpenLibrary(
"<gadgets/gadgetname>.gadget",
<version>
);
```

E.2.1. button.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.2. checkbox.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.3. chooser.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.4. clicktab.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.5. colorwheel.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.6. datebrowser.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.7. fuelgauge.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.8. getcolor.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.9. getfile.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.10. getfont.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.11. getscreenmode.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.12. gradientslider.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.13. integer.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.14. layout.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.15. listbrowser.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.16. listview.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.17. palette.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.18. radiobutton.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.19. scroller.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.20. sketchboard.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.21. slider.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.22. space.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides: Canvas where the user might draw in.
Tags:
Cave-At: When used to Qt/C^{++} programming, one might expect this to be an object that is used for putting space between two objects. Well, in this case this gadget is in *no way* what you might think of it. Especially it *should not be used* for spacing!
Tips: Don't confuse this gadget – as said, it is *not* meant to be used for setting spaces. If you need space between two objects, e.g. use an *empty child Label* instead.
Note:

E.2.23. speedbar.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.24. string.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.25. tabs.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.26. tapedeck.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.27. texteditor.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.2.28. virtual.gadget

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.3. Images

Images reside in SYS:Classes/Images and are opened with

```
<Image>Base = (struct Library *) OpenLibrary(
"<images/imagename>.image",
<version>
);
```

E.3.1. bevel.image

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.3.2. bitmap.image

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.3.3. boingball.image

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.3.4. drawlist.image

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.3.5. glyph.image

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.3.6. label.image

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.3.7. led.image

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

E.3.8. penmap.image

Under Construction!
Recently no further information available yet – sorry for the inconvenience. Please come back later.

Provides:
Tags:
Note:

F. Useful links

F.1. AmigaOS Developer CD v2.1

This CD contains many articles and more. A real treasure trove! Get it from archive.org:
https://archive.org/details/amigadevelopercdv2.1

F.2. AmigaOS Developer CD v2.1 online

Most parts of the docs contained on Amiga Developer CD v2.1 can be accessed online here:
http://amigadev.elowar.com/read/ADCD_2.1/

F.3. ReBuild repository

Darren Cole's repository for ReBuild on GitHub
https://github.com/dmcoles/ReBuild

F.4. Codecraft IDE

Codecraft is a feature-rich, full-fleshed development environment for AmigaOS 3.2 – actively developed and enhanced by *Camilla Boemann* and *Ralf Hasemann*. Besides other compiler systems, it actively supports SAS/C. Get it from here:
http://boemann.dk/codecraft/

F.5. Recent vbcc for Amiga

You can always get the recent version of Dr. Volker Barthelmann's ISO C compiler here:
http://sun.hasenbraten.de/vbcc/

F.6. Bebbo's amiga-gcc Cross Compiler

Bebbo's amiga-gcc makes it possible to run an Amiga C Cross-Compiler on Linux, Windows and Mac platforms. It compiles code blazing fast and consists of cross-compiling versions of GNU gcc and vbcc. Best thing: You can build it with AmigaOS 3.2 NDK enabled as default. Get the sources from GitHub:
https://github.com/bebbo/amiga-gcc

F.7. Micha B.'s AmigaED

AmigaED is an under-development GUI environment for Bebbo's amiga-gcc Cross Compiler. Get the sources from GitHub:
https://github.com/mbergmann-sh/AmigaED

F.8. E-VO compiler

Darren Coles' Amiga E-VO compiler system
https://github.com/dmcoles/EVO/releases/tag/v3.6.1

G. Useful third-party functions

Here you will find often-needed useful functions that speed up the development of *working* applications from ReBuild-generated source code.
In order to use them, it is possibly required to add include files, library bases, function prototypes, library opening and closing calls, global or local variables to your existing code. Please refer to the description of prerequisites for each function that you want to use!

G.1. getCheckBoxStatus(...)

Purpose: Get status of a CheckBox gadget, set text in a corresponding String gadget
Prototype: void getCheckBoxStatus(struct Window *win, struct Gadget *chkgad, struct Gadget *txtgad)
Requires: Changes to ReBuild-generated functions

1. Change function *runWindow*'s function header to:

```
void runWindow( Object *window_object, int window_id,
                struct Menu *menu_strip, struct Gadget *win_gadgets[] )
```

2. in your window() function (usually *window_3()*), find the call to *runWindow()* and replace it by

```
// for first window: "3" identifies first window's ID...
runWindow( window_object, 3, menu_strip, main_gadgets );
```

3. Identify the gadget ID used in main_gadgets[ ] – you can find them in the index of the particular object's main_gadgets[ ] layout definitions. Either keep them in mind, or create "speaking" constants for them, e.g.

```
#define GADS_CB_ENE    3
#define GADS_CB_MENE   4
#define GADS_CB_MUH    5
#define GADS_STR_ENE   7
#define GADS_STR_MENE  8
#define GADS_STR_MUH   9
```

Example: See pages 56 ff for a full example.
Listing:

```
/*
* Function: void getCheckBoxStatus(struct Window *win,
*                                  struct Gadget *chkgad, struct Gadget *txtgad)
* Purpose:  Get status of a CheckBox gadget, set text in a corresponding String gadget
* Version:  1.0
* Author:   Darren Coles
*/
void getCheckBoxStatus(struct Window *win, struct Gadget *chkgad, struct Gadget *txtgad)
{
  ULONG status = 0;
  ULONG success = 0;
  STRPTR isChecked;

  GetAttr(CHECKBOX_Checked, (APTR)chkgad, &status);
  switch (status)
  {
    case 0:
    isChecked = "unchecked";
    break;
    case 1:
    isChecked = "checked";
    break;
```

```
    }
    success = SetGadgetAttrs((APTR)txtgad, win, 0, STRINGA_TextVal, isChecked, TAG_END );


    printf("Status: %ld\n\n", status);
    printf("success: %ld\n\n", success);

  }
```

G.2. fileExists(...)

Purpose: Check if a certain file exists, returns (BOOL)
Prototype: BOOL fileExists(STRPTR fname)
Includes:

```
#include <stdio.h>
#include <limits.h>
#include <string.h>
#include <exec/types.h>
#include <dos/dos.h>
#include <proto/dos.h>
```

Listing:

```
/*
* Function: BOOL fileExists(STRPTR fname)
* Purpose:  Check if a certain file exists, returns (BOOL)
*           (solved UNIX-stylish)
* Version:  1.0
* Author:   Michael Bergmann
*/
BOOL fileExists(STRPTR fname)
{
  FILE *file;

  file = fopen(fname, "r");
  if (file )
  {
    fclose(file);
    return TRUE;
  }
  return FALSE;
}
```

G.2.1. fileExists(...) example snippet

```
/* ---------- DEMO ---------------------------- */
void main()
{
  STRPTR existing;

  if(fileExists("RAM:lha"))
    existing = "does exist.";
  else
    existing = "does NOT exist";

  Printf("Result: %ld - File: RAM:lha  %s\n", fileExists("RAM:lha"), existing );

```

```
13    if(fileExists("C:lha"))
14      existing = "does exist.";
15    else
16      existing = "does NOT exist";
17
18    Printf("Result: %ld - File: C:lha  %s\n", fileExists("C:lha"), existing );
19  }
```

G.3. getProgDir()

Purpose: returns PROGDIR:

Prototype: STRPTR getProgDir(void)

Includes:

```
#include <exec/types.h>
#include <exec/libraries.h>
#include <dos/dos.h>
#include <dos/dostags.h>

#include <proto/exec.h>
#include <proto/dos.h>
#include <proto/intuition.h>
#include <proto/utility.h>
```

Listing:

```
1   /*
2   * Function: STRPTR getProgDir(void)
3   * Purpose:  Gets directory where program is located,
4   *           returns (STRPTR) PROGDIR
5   *           (solved Amiga-stylish)
6   * Version:  1.0
7   * Author:   Michael Bergmann
8   */
9   STRPTR getProgDir(void)
10  {
11    #define MAXPATHLEN 512
12
13    static UBYTE progdir[MAXPATHLEN];
14    BPTR lock = Lock("PROGDIR:", ACCESS_READ);
15
16    NameFromLock(lock, progdir, MAXPATHLEN);
17    UnLock(lock);
18
19    return(STRPTR)&progdir;
20  }
```

G.4. doFileRequest(...)

Purpose: Opens an ASL file requester, returns selected filename as STRPTR (or empty string if canceled). Position and dimensions of the requester are user-definable. So is the default file name.

Include files:

```
#include <libraries/asl.h>
#include <proto/asl.h>
```

Base: *struct Library *AslBase = NULL;*

Prototype:

STRPTR doFileRequest(char *path, char *pattern, char *file, char *old_file, char *title, struct Window *window, long width, long height, BOOL reject)

Listing:

```
/*
* Function: STRPTR doFileRequest( ... )
* Purpose:  Opens an ASL File-type Requester, returns filename as STRPTR
* Version:  1.0
* Author:   Michael Bergmann
*/

#include <exec/types.h>
#include <exec/libraries.h>
#include <dos/dos.h>
#include <dos/dostags.h>
#include <libraries/asl.h>

#include <proto/exec.h>
#include <proto/dos.h>
#include <proto/intuition.h>
#include <proto/utility.h>
#include <proto/asl.h>

#include <stdio.h>
#include <string.h>

/* Prototype */
STRPTR doFileRequest(char *path, char *pattern, char *file,
char *old_file, char *title, struct Window *window,
long width, long height, BOOL reject);

/*
    Amiga ASL File-Request Function
*/
STRPTR doFileRequest(char *path, char *pattern, char *file,
char *old_file, char *title, struct Window *window,
long width, long height, BOOL reject)
{
  /* ASL-Requester: */
  struct FileRequester *request;  // Instanciate ASL-File-Requester
  static UBYTE fname[255], errorcheck[80];

  strcpy(fname, "");

  printf("old_file = %s\n", old_file);

  /* FileRequester Structure ... */
  request = (struct FileRequester *)AllocAslRequestTags(ASL_FileRequest,
  ASL_Hail, title, (struct TagItem *)TAG_DONE);

  /* ...show File-Requester ... */
  if (AslRequestTags(request,
                     ASLFR_Window, window,
                     ASLFR_SleepWindow,TRUE,
                     ASLFR_InitialLeftEdge, 20,
                     ASLFR_InitialTopEdge, 20,
                     ASLFR_InitialWidth, width,
                     ASLFR_InitialHeight, height,
                     ASLFR_RejectIcons, reject,        // NO Icons: TRUE, show: FALSE
```

```
                        ASLFR_InitialDrawer, path,
                        ASLFR_InitialFile, file,
                        ASLFR_InitialPattern, (ULONG) pattern,  // e.g. "~rexx#?|math#?",
                        ASLFR_DoPatterns,TRUE,              // show Filter-Stringgadget
                        ASLFR_RejectPattern, (ULONG) pattern,   // e.g. "~rexx#?|math#?",

                        /* uncomment these in order to use own button texts... */
                        //ASLFR_PositiveText, "Open",
                        //ASLFR_NegativeText, "Cancel",
                        (struct TagItem *)TAG_DONE))
    {
      /* check rf_Dir for empty String */
      strcpy(errorcheck, request->rf_Dir);

      /* build path name */
      strcat( fname, request->rf_Dir);
      if (fname[strlen(fname)-1] != (UBYTE)58) /* Check for : */
        strcat(fname, "/");

      strcat(fname, request->rf_File);
    }

    /* free Requester-Structure */
    if (request)
      FreeAslRequest(request);

    /* check if returned String is empty */
    if ( (strcmp(errorcheck, "") != 0))
      return(fname);
    else
      return("");
  }
```

G.4.1. doFileRequest(...) example snippet

Here is an example on how to use doFileRequest(...):

```
/*
Amiga ASL File Requester DEMO
Compile me:
SAS/C: sc asldemo.c LINK TO asldemo
vbcc: vc +aos68k -lamiga -lauto -o=asldemo asldemo.c
*/
int main(void)
{
  STRPTR myFile;

  if( !(AslBase = (struct Library*) OpenLibrary("asl.library",0L)) )
    return 0;

  Printf("\f\n*** ASL FileRequester Demo ***\n\n");

  /* saveRequest */
  myFile = doFileRequest("RAM:",                                  // path
                         "~#?.info|#?.info",                      // pattern
                         "my_file.txt",                           // file
                         "RAM:my_save.txt",                       // oldfile
                         "ASL Save Request",                      // title
                         NULL,                                    // window
                         320,                                     // width
```

```
                          400,                                    // height
                          TRUE                                    // reject
                        );

    Printf("\nASL File FILE Request\n--------------------------------\n");
    if ( (strcmp(myFile, "") != 0))
      Printf("Selected File: %s\n\n", myFile);
    else
      Printf("User canceled Requester!");

    Printf("\n****** Demo ends ******\n\n");

    if (AslBase)
      CloseLibrary( (struct Library *)AslBase );

    return 0;
  }
```

G.5. doDirRequest()

Purpose: Opens an ASL Directory requester, returns selected directory as STRPTR (or empty string if canceled). Position and dimensions of the requester are user-definable. So is the default directory name.

Include files:

```
#include <libraries/asl.h>
#include <proto/asl.h>
```

Base: *struct Library *AslBase = NULL;*

Prototype: STRPTR dirRequest(char *path, char *olddir, char *title, struct Window *window, long width, long height)

Listing:

```
/*
* Function: STRPTR doDirRequest( ... )
* Purpose:  Opens an ASL directory-type Requester, returns folder as STRPTR
* Version:  1.0
* Author:   Michael Bergmann
*/
STRPTR dirRequest(char *path, char *olddir, char *title,
struct Window *window, long width, long height)
{
  struct FileRequester *request;  // Instanciate...
  static UBYTE fname[255], errorcheck[80];
  strcpy(fname, "");

  /*build  FileRequester Structure  */
  request = (struct FileRequester *)AllocAslRequestTags(ASL_FileRequest,
  ASL_Hail, title, (struct TagItem *)TAG_DONE);

  /* ...show Directory-Requester */
  if (AslRequestTags(request,
                     ASLFR_Window, window,          // Pointer to calling window
                     ASLFR_SleepWindow,TRUE,        // modale Mode, Mousepointer wird "busy"...
                     ASLFR_InitialLeftEdge, 20,     // Coordinates
                     ASLFR_InitialTopEdge, 20,
                     ASLFR_InitialWidth, width,     // initial sizing
                     ASLFR_InitialHeight, height,
                     ASLFR_DrawersOnly, TRUE,       // show folders only
```

```
                        ASLFR_InitialDrawer, path,      // initial folder
                        ASLFR_Activate, TRUE,           // from WB 3.9 and up only!!
                        (struct TagItem *)TAG_DONE))
    {
      /* rf_Dir empty String ? */
      strcpy(errorcheck, request->rf_Dir);

      /* build pathname for return */
      strcat( fname, request->rf_Dir);
    }

    /* ...free Requester-Structure! */
    if (request)
      FreeAslRequest(request);

    /* Test returned String */
    if ( (strcmp(errorcheck, "") != 0))
      return(fname);
    else
      return ("");
  }
```

doDirRequest(...) example snippet

```
/*
Amiga ASL Directory Requester DEMO
Compile me:
SAS/C: sc asldemo.c LINK TO asldemo
vbcc: vc +aos68k -lamiga -lauto -o=asldemo asldemo.c
*/
#include <exec/types.h>
#include <exec/libraries.h>
#include <dos/dos.h>
#include <dos/dostags.h>
#include <libraries/asl.h>

#include <proto/exec.h>
#include <proto/dos.h>
#include <proto/intuition.h>
#include <proto/utility.h>
#include <proto/asl.h>

#include <stdio.h>
#include <string.h>

struct Library *AslBase = NULL;

// insert doDirRequest() HERE

/* ------------- DEMO ------------------------------------------------------------------------ */
int main(void)
{
  STRPTR myDir;

  if( !(AslBase = (struct Library*) OpenLibrary("asl.library",0L)) ) return 0;

  Printf("\f\n*** ASL Dir Requester Demo ***\n\n");

  /* --- ASL Directory Request --- */
```

```
36      // STRPTR doDirRequest(char *path, char *olddir, char *title,
37      //              struct Window *window, long width, long height)
38      myDir = doDirRequest("Libs:",                       // path
39                           "Libs:",                       // olddir
40                           "ASL Directory Request",       // title
41                           NULL,                          // window
42                           320,                           // width
43                           200                            // height
44                          );
45
46      Printf("\nASL Directory Request\n----------------------\n");
47      if ( (strcmp(myDir, "") != 0))
48        Printf("Selected Folder: %s\n\n", myDir);
49      else
50        Printf("User canceled Requester!\n");
51
52      if (AslBase)
53        CloseLibrary( (struct Library *)AslBase );
54    }
```

G.6. doSaveRequest(...)

Purpose: Opens an ASL save requester, returns selected filename as STRPTR (or empty string if canceled). Position and dimensions of the requester are user-definable.

Include files:

```
#include <libraries/asl.h>
#include <proto/asl.h>
```

Base: *struct Library *AslBase = NULL;*

Prototype:

STRPTR doSaveRequest(char *path, char *pattern, char *file, char *old_file, char *title, struct Window *window, long width, long height, BOOL reject)

Listing:

```
1   /*
2   * Function: STRPTR doSaveRequest( ... )
3   * Purpose:  Opens an ASL Save-type Requester, returns filename as STRPTR
4   * Version:  1.0
5   * Author:   Michael Bergmann
6   */
7   STRPTR doSaveRequest(char *path, char *pattern, char *file,
8   char *old_file, char *title, struct Window *window,
9   long width, long height, BOOL reject)
10  {
11    /* ASL-Requester: */
12    struct FileRequester *request;  // Instanciate as  ASL-Save-Requester
13    static UBYTE fname[255], errorcheck[80];
14
15    strcpy(fname, ""); // start with empty filename
16
17    /* FileRequester Structure ... */
18    request = (struct FileRequester *)AllocAslRequestTags(ASL_FileRequest,
19    ASL_Hail, title, (struct TagItem *)TAG_DONE);
20
21    /* ...show Save-Requester */
22    if (AslRequestTags(request,
```

```
                    ASL_FuncFlags, FILF_DOWILDFUNC | FILF_DOMSGFUNC | FILF_SAVE,
                    ASLFR_Window, window,
                    ASLFR_SleepWindow,TRUE,
                    ASLFR_InitialLeftEdge, 20,
                    ASLFR_InitialTopEdge, 20,
                    ASLFR_InitialWidth, width,
                    ASLFR_InitialHeight, height,
                    ASLFR_RejectIcons, reject,              // NO Icons:: TRUE, FALSE
                    ASLFR_InitialDrawer, path,
                    ASLFR_InitialFile, file,
                    ASLFR_InitialPattern, (ULONG) pattern,  // Example: "~rexx#?|math#?",
                    ASLFR_DoPatterns,TRUE,                  // show file-filtering Stringgadget
                    ASLFR_RejectPattern, (ULONG) pattern,   // Filter  example: "~rexx#?|math#?",
                    ASLFR_PositiveText, "Save",             // user-defined button text
                    ASLFR_NegativeText, "No, thanx!",       // user-defined button text
                    (struct TagItem *)TAG_DONE))
    {
      /* save rf_Dir for testing empty String */
      strcpy(errorcheck, request->rf_Dir);

      /* build path name */
      strcat( fname, request->rf_Dir);

      if (fname[strlen(fname)-1] != (UBYTE)58) /* Check for : */
      strcat(fname, "/");

      strcat(fname, request->rf_File);
    }

    /* Free Requester-Structure! */
    if (request)
      FreeAslRequest(request);

    /* check if return value is empty */
    if ( (strcmp(errorcheck, " ") != 0))
      return(fname);

    else
      return("");
  }
```

G.6.1. doSaveRequest(...) example snippet

Here is an example on how to use doSaveRequest(...):

```
/*
    Amiga ASL Save Requester DEMO
    Compile me:
    SAS/C: sc asldemo.c LINK TO asldemo
    vbcc: vc +aos68k -lamiga -lauto -o=asldemo asldemo.c
*/
#include <exec/types.h>
#include <exec/libraries.h>
#include <dos/dos.h>
#include <dos/dostags.h>
#include <libraries/asl.h>

#include <proto/exec.h>
#include <proto/dos.h>
#include <proto/intuition.h>
```

```
16  #include <proto/utility.h>
17  #include <proto/asl.h>
18
19  #include <string.h>
20
21  // Function Prototype
22  STRPTR doSaveRequest(char *path, char *pattern, char *file,
23  char *old_file, char *title, struct Window *window,
24  long width, long height, BOOL reject);
25
26  // Library Base
27  struct Library *AslBase = NULL;
28
29  int main(void)
30  {
31    STRPTR myFile;
32
33    if( !(AslBase = (struct Library*) OpenLibrary("asl.library",0L)) )
34      return 0;
35
36    Printf("\f\n*** ASL Save Requester Demo ***\n\n");
37
38    /* saveRequest */
39    myFile = doSaveRequest("RAM:",            // path
40                           "~#?.info|#?.info",                    // pattern
41                           "my_save.txt",                         // file
42                           "RAM:my_save.txt",                     // olfile
43                           "ASL Save Request",                    // title
44                           NULL,                                  // window
45                           320,                                   // width
46                           400,                                   // height
47                           TRUE                                   // reject
48                         );
49
50    Printf("\nASL File SAVE Request\n--------------------------------\n");
51    if ( (strcmp(myFile, "") != 0))
52      Printf("Selected SAVE File: %s\n\n", myFile);
53    else
54      Printf("User canceled Requester!");
55
56    Printf("\n****** Demo ends ******\n\n");
57
58    if (AslBase)
59      CloseLibrary( (struct Library *)AslBase );
60
61    return 0;
62  }
```

G.7. doInfoRequest(...)

This function comes in handy if you want ReAction-style INFO Requesters with them nice icons for either Information, Warning, Error, Question or Disk inserts. Be aware that you will need to include the appropriate header files and also open and close requester.class *(see also page 75 and 53)* in order how to use it.

Purpose: Opens a requester.class informal requester, returns selected button

Include files:

```
#include <proto/button.h>
#include <proto/requester.h>
#include <classes/requester.h>
```

Base: *struct Library *RequesterBase = NULL;*

Prototype:

int doInfoRequest(struct Screen *screen, const char *title, const char *body, const char *buttons, ULONG image)

Remarks: *ULONG image* Tag can be one of:

- NULL *(REQIMAGE_DEFAULT will be used in this case)*
- REQIMAGE_DEFAULT
- REQIMAGE_INFO
- REQIMAGE_WARNING
- REQIMAGE_ERROR
- REQIMAGE_QUESTION
- REQIMAGE_INSERTDISK

See also: NDK3.2:include/classes/requester.h and this document's page 75

Listing:

```
1  /*
2   *  Function: int DoInfoRequest( ... )
3   *  Purpose:  Opens a requester.class informal requester, returns selected button
4   *  Version:  1.0
5   *  Author:   Michael Bergmann
6   */
7  int doInfoRequest(struct Screen *screen,
8                    const char *title, const char *body,
9                    const char *buttons, ULONG image )
10 {
11   Object *req = 0;     // the requester itself
12   int button;      // the button that was clicked by the user
13
14   // fill in the requester structure
15   req = NewObject(REQUESTER_GetClass(), NULL,
16   REQ_Type,         REQTYPE_INFO,
17   REQ_TitleText,    (ULONG)title,
18   REQ_BodyText,     (ULONG)body,
19   REQ_GadgetText,   (ULONG) buttons ,
20   REQ_Image,        image,
21   TAG_DONE);
22
23   if (req)
24   {
25     struct orRequest reqmsg;
26
27     reqmsg.MethodID  = RM_OPENREQ;
28     reqmsg.or_Attrs  = NULL;
29     reqmsg.or_Window = NULL;
30     reqmsg.or_Screen = screen;
31
32     button = DoMethodA(req, (Msg) &reqmsg);
```

```
33      DisposeObject(req);
34
35      return button;
36
37    }  else
38        printf("[request] Could not allocate requester\n");
39
40    return 0;
41  }
```

G.7.1. doInfoRequest(...) example snippet

Function may be used like this:

```
...
case BTN_ERRORREQUEST:
    SetAttrs(window_object, WA_BusyPointer, TRUE, TAG_DONE);    // open modal
    /* —— call Requester —— */
    (void)doInfoRequest(NULL, "Fake Error Requester",     // Title text
    " Mistake! You didn't brew coffee.\n System halted.", // Body (Message) text
    "Oh, didn't I?",                                      // Button(s)
    REQIMAGE_ERROR);                                      // Image
    SetAttrs(window_object, WA_BusyPointer, FALSE, TAG_DONE);   // end modal mode
break;
case BTN_QUIT:
    SetAttrs(window_object, WA_BusyPointer, TRUE, TAG_DONE);  // open modal
    /* —— call Requester —— */
    req_result = doInfoRequest(NULL, "Quit this app",
    " Do you reall want to end this disaster?",
    "_Quit|_Cancel",    // TWO Buttons!
    REQIMAGE_WARNING);
    printf("result was: %ld\n", req_result);
    if (req_result == 1)
      done = TRUE;
    SetAttrs(window_object, WA_BusyPointer, FALSE, TAG_DONE); // end modal mode
break;
...
```

G.8. identifyCompiler()

Sometimes you may want to identify the C-compiler used for building an application, e.g for using this information in your About-Requester or -Window.

Purpose: returns a STRPTR to a string containing information about the C-Compiler used for compiling a program

Prototype: STRPTR identifyCompiler(void)

Listing:

```
1   /*
2   * Function: STRPTR identifyCompiler(void)
3   * Purpose:  Collect compiler system information
4   * Version:  1.0
5   * Author:   Michael Bergmann
6   */
7   STRPTR identifyCompiler(void)
8   {
9     char myCompiler[512];
10
```

```
11    #ifdef __SASC
12    const int sasversion = __VERSION__;
13    const int sasrev = __REVISION__;
14    #endif
15    #if defined(__STORM__)
16    /* Compiler is StormC3 */
17    sprintf(myCompiler, "StormC3");
18    #elif defined(__STORMGCC__)
19    /* Compiler is StormGCC4 */
20    sprintf(myCompiler, "GNU gcc, StormC4 flavour");
21    #elif defined(__MAXON__)
22    /* Compiler is Maxon/HiSoft C++ */
23    sprintf(myCompiler, "Maxon/HiSoft C++");
24    #elif defined(__GNUC__)
25    /* Compiler is gcc */
26    sprintf(myCompiler, "GNU gcc v%ld.%ld", __GNUC__, __GNUC_MINOR__);
27    #ifdef __GNUC_PATCHLEVEL__
28    strcat(myCompiler, ".");
29    strcat(myCompiler, __GNUC_PATCHLEVEL__);
30    #endif
31    #elif defined(__VBCC__)
32    /* Compiler is vbcc */
33    sprintf(myCompiler, "vbcc - Volker Barthelmann's retargetable C compiler");
34    #elif defined(__SASC)
35    /* Compiler is SAS/C */
36    sprintf(myCompiler, "SAS/C, Version %ld.%ld", sasversion, sasrev);
37    #elif defined(LATTICE)
38    /* Compiler is Lattice C */
39    sprintf(myCompiler, "Lattice C");
40    #elif defined(AZTEC_C)
41    /* Compiler is Aztec C */
42    sprintf(myCompiler, "Manx Aztec C v%ld", __VERSION);
43    #elif defined(_DCC)
44    /* Compiler is dice */
45    sprintf(myCompiler, "Matt Dillon's DICE, propably v3.15");
46    #else
47    /* Compiler not identified */
48    sprintf(myCompiler, "UNKNOWN");
49    #endif
50
51    return(myCompiler);
52  }
```

G.8.1. identifyCompiler(...) example snippet

It's as simple as calling the function...

```
1   /*
2       identifyCompilerDemo.c
3   */
4   #include <dos/dos.h>
5   #include <proto/dos.h>
6   #include <stdio.h>
7
8   // ... identifyCompiler() function HERE
9
10  void main()
11  {
12    Printf("Program compiled with %s\n", identifyCompiler());
13  }
```

H. Full Example Listings

H.1. Full Listing - CheckBox-Demo

cbdemo-final.c

```
/*
  Project:    ReBuild Tutorial
  File:       cbdemo.c
  Purpose:    explaination basis
  Author:     M. Bergmann
*/
#include <clib/macros.h>
#include <clib/alib_protos.h>
#include <clib/compiler-specific.h>

#include <proto/exec.h>
#include <proto/dos.h>
#include <proto/utility.h>
#include <proto/graphics.h>
#include <proto/intuition.h>
#include <proto/gadtools.h>
#include <proto/icon.h>

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include <proto/window.h>
#include <proto/layout.h>
#include <proto/button.h>
#include <proto/checkbox.h>
#include <proto/string.h>
#include <proto/label.h>
#include <proto/requester.h>

#include <classes/requester.h>

#include <libraries/gadtools.h>
#include <reaction/reaction.h>
#include <intuition/gadgetclass.h>
#include <reaction/reaction_macros.h>
#include <classes/window.h>
#include <exec/memory.h>

/* "speaking" main_gadgets[] index constants */
#define GADS_CB_ENE    3
#define GADS_CB_MENE   4
#define GADS_CB_MUH    5
#define GADS_STR_ENE   7
#define GADS_STR_MENE  8
#define GADS_STR_MUH   9
/* "speaking" gadget ID constants */
#define CB_ENE       29
#define CB_MENE      30
#define CB_MUH       31
#define STR_ENE      33
#define STR_MENE     34
#define STR_MUH      35
#define BTN_STATUS   37

/* Menu identifiers */
enum menus {
```

```
58      // Menu "Project"
59      MN_ENE,
60      MN_MENE,
61      MN_MUH,
62      MN_QUIT,
63      // Menu "Help"
64      MN_ABOUT,
65    };
66
67    /* GadgetHelp */
68    struct HintInfo hintinfo[] =
69    {
70      { CB_ENE, -1, "I am  CheckBox #1, setting status text in String #1", 0L},
71      { CB_MENE, -1, "I am  CheckBox #2, setting status text in String #2", 0L},
72      { CB_MUH, -1, "I am  CheckBox #3, setting status text in String #3", 0L},
73      { BTN_STATUS, -1, "I have no real effect, since setting status" \
74        "\ntexts is done by the CheckBoxes already!", 0L},
75      { -1, -1, NULL, 0L}   // terminate!
76    };
77
78
79
80    /* Function Prototypes */
81    void window_cb( void );
82
83    void getCheckBoxStatus(struct Window *win,
84    struct Gadget *chkgad,
85    struct Gadget *txtgad);
86
87    int doInfoRequest(struct Screen *screen, const char *title,
88    const char *body, const char *buttons,
89    ULONG image );
90
91
92    /* other global assignments */
93    struct Screen *gScreen = NULL;
94    struct DrawInfo *gDrawInfo = NULL;
95    APTR gVisinfo = NULL;
96    struct MsgPort  *gAppPort = NULL;
97
98    struct Library *WindowBase = NULL,
99    *ButtonBase = NULL,
100   *CheckBoxBase = NULL,
101   *StringBase = NULL,
102   *LabelBase = NULL,
103   *RequesterBase,
104   *GadToolsBase = NULL,
105   *LayoutBase = NULL,
106   *IconBase = NULL;
107   struct IntuitionBase *IntuitionBase = NULL;
108
109
110   /*
111   * Function: void GetCheckBoxStatus( ... )
112   * Purpose:  get CheckBox status, set String text
113   */
114   void getCheckBoxStatus(struct Window *win, struct Gadget *chkgad, struct Gadget *txtgad)
115   {
116     ULONG status = 0;
117     ULONG success = 0;
```

```
118     STRPTR isChecked = "empty";
119
120     GetAttr(CHECKBOX_Checked, (APTR)chkgad, &status);
121     switch (status)
122     {
123       case 0:
124       isChecked = "unchecked";
125       break;
126       case 1:
127       isChecked = "checked";
128       break;
129     }
130     success = SetGadgetAttrs((APTR)txtgad, win, 0, STRINGA_TextVal, isChecked, TAG_END );
131
132     printf("Status: %ld\n\n", status);
133     printf("success: %ld\n\n", success);
134   }
135
136   /*
137   * Function: int doInfoRequest( ... )
138   * Purpose:  display Requester, return button value
139   */
140   int doInfoRequest(struct Screen *screen, const char *title,
141                     const char *body, const char *buttons, ULONG image )
142   {
143     Object *req = 0;     // the requester itself
144     int button;      // the button that was clicked by the user
145
146     // fill in the requester structure
147     req = NewObject(REQUESTER_GetClass(), NULL,
148     REQ_Type,        REQTYPE_INFO,
149     REQ_TitleText,   (ULONG)title,
150     REQ_BodyText,    (ULONG)body,
151     REQ_GadgetText,  (ULONG) buttons ,
152     REQ_Image,       image,
153     TAG_DONE);
154
155     if (req)
156     {
157       struct orRequest reqmsg;
158
159       reqmsg.MethodID  = RM_OPENREQ;
160       reqmsg.or_Attrs  = NULL;
161       reqmsg.or_Window = NULL;
162       reqmsg.or_Screen = screen;
163
164       button = DoMethodA(req, (Msg) &reqmsg);
165       DisposeObject(req);
166
167       return button;
168
169     }  else
170     printf("[doInfoRequest] Failed to allocate requester\n");
171
172     return 0;
173   }
174
175
176   /*
177   * Function: int setup()
```

```
178  * Purpose:  open libs, prepare app
179  */
180  int setup( void )
181  {
182    if( !(IntuitionBase = (struct IntuitionBase*) OpenLibrary("intuition.library",0L)) ) return 0;
183    if( !(GadToolsBase = (struct Library*) OpenLibrary("gadtools.library",0L) ) ) return 0;
184    if( !(WindowBase = (struct Library*) OpenLibrary("window.class",0L) ) ) return 0;
185    if( !(IconBase = (struct Library*) OpenLibrary("icon.library",0L) ) ) return 0;
186    if( !(LayoutBase = (struct Library*) OpenLibrary("gadgets/layout.gadget",0L) ) ) return 0;
187    if( !(ButtonBase = (struct Library*) OpenLibrary("gadgets/button.gadget",0L) ) ) return 0;
188    if( !(CheckBoxBase = (struct Library*) OpenLibrary("gadgets/checkbox.gadget",0L) ) ) return 0;
189    if( !(StringBase = (struct Library*) OpenLibrary("gadgets/string.gadget",0L) ) ) return 0;
190    if( !(LabelBase = (struct Library*) OpenLibrary("images/label.image",0L) ) ) return 0;
191    if( !(RequesterBase = (struct Library*) OpenLibrary("requester.class",0L) ) ) return 0;
192    if( !(gScreen = LockPubScreen( 0 ) ) ) return 0;
193    if( !(gVisinfo = GetVisualInfo( gScreen, TAG_DONE ) ) ) return 0;
194    if( !(gDrawInfo = GetScreenDrawInfo ( gScreen ) ) ) return 0;
195    if( !(gAppPort = CreateMsgPort() ) ) return 0;
196
197    return -1;
198  }
199
200  /*
201  * Function: void cleanup()
202  * Purpose:  close libs, clean up things
203  */
204  void cleanup( void )
205  {
206    if ( gDrawInfo ) FreeScreenDrawInfo( gScreen, gDrawInfo);
207    if ( gVisinfo ) FreeVisualInfo( gVisinfo );
208    if ( gAppPort ) DeleteMsgPort( gAppPort );
209    if ( gScreen ) UnlockPubScreen( 0, gScreen );
210
211    if (GadToolsBase) CloseLibrary( (struct Library *)GadToolsBase );
212    if (RequesterBase) CloseLibrary( (struct Library *)RequesterBase );
213    if (IconBase) CloseLibrary( (struct Library *)IconBase );
214    if (IntuitionBase) CloseLibrary( (struct Library *)IntuitionBase );
215    if (ButtonBase) CloseLibrary( (struct Library *)ButtonBase );
216    if (CheckBoxBase) CloseLibrary( (struct Library *)CheckBoxBase );
217    if (StringBase) CloseLibrary( (struct Library *)StringBase );
218    if (LabelBase) CloseLibrary( (struct Library *)LabelBase );
219    if (LayoutBase) CloseLibrary( (struct Library *)LayoutBase );
220    if (WindowBase) CloseLibrary( (struct Library *)WindowBase );
221  }
222
223
224  /*
225  * Function: void runWindow( ... )
226  * Purpose:  open window, handle event loop
227  */
228  void runWindow( Object *window_object, int window_id,
229  struct Menu *menu_strip, struct Gadget *win_gadgets[] )
230  {
231    struct Window *main_window = NULL;
232    // we need a MenuItem struct to keep
233    // track on status informations:
234    struct MenuItem *menuitem = NULL;
235
236    if ( window_object )
237    {
```

```
238     if ( main_window = (struct Window *) RA_OpenWindow( window_object ))
239     {
240       WORD Code;
241       ULONG wait = 0, signal = 0, result = 0, done = FALSE;
242       ULONG req_result = NULL;
243
244       GetAttr( WINDOW_SigMask, window_object, &signal );
245       if ( menu_strip)  SetMenuStrip( main_window, menu_strip );
246       while ( !done)
247       {
248         wait = Wait( signal | SIGBREAKF_CTRL_C );
249
250         if ( wait & SIGBREAKF_CTRL_C )
251         done = TRUE;
252         else
253         while (( result = RA_HandleInput( window_object, &Code )) != WMHI_LASTMSG)
254         {
255           switch ( result & WMHI_CLASSMASK )
256           {
257             case WMHI_CLOSEWINDOW:
258             SetAttrs(window_object, WA_BusyPointer, TRUE, TAG_DONE); // block window
259             req_result = doInfoRequest(NULL, "Quit cbDemo",
260             " Do you really want to end this disaster?" \
261             "\n Select 'Quit' to leave me alone!",
262             "_Quit|_Cancel", REQIMAGE_QUESTION);
263             printf("result was: %ld\n", req_result);
264             if (req_result == 1)
265             done = TRUE;
266             SetAttrs(window_object, WA_BusyPointer, FALSE, TAG_DONE); // de-block window
267             break;
268
269             case WMHI_MENUPICK:
270             puts("menu pick");
271             menuitem = ItemAddress(menu_strip, result & WMHI_MENUMASK);
272             if(!(menuitem == 0)) // check against NULL, otherwise enforcer hit!
273             {
274               switch ((long)GTMENUITEM_USERDATA(menuitem))
275               {
276                 case MN_QUIT:
277                   SetAttrs(window_object, WA_BusyPointer, TRUE, TAG_DONE); // block window
278                   req_result =
279                   doInfoRequest(NULL, "Quit cbDemo",
280                   " Do you really want to end this disaster?" \
281                   "\n Select 'Quit' to leave me alone!",
282                   "_Quit|_Cancel", REQIMAGE_QUESTION);
283                   printf("result was: %ld\n", req_result);
284                   if (req_result == 1)
285                     done = TRUE;
286                   SetAttrs(window_object, WA_BusyPointer, FALSE, TAG_DONE); // de-block window
287                   break;
288                 case MN_ENE:
289                   puts("Project Menu: Ene selected");
290                   /* evaluate, if item has checkmark set... */
291                   if(menuitem->Flags & CHECKED)
292                   {
293                     puts("MenuItem is CHECKED.");
294                   }
295                   else
296                     puts("MenuItem is NOT CHECKED.");
297                   break;
```

```
298
299                 case MN_MENE:
300                   puts("Project Menu: Mene selected");
301                   if(menuitem->Flags & CHECKED)
302                   {
303                     puts("MenuItem is CHECKED.");
304                   }
305                   else
306                     puts("MenuItem is NOT CHECKED.");
307                   break;
308
309                 case MN_MUH:
310                   puts("Project Menu: Muh selected");
311                   if(menuitem->Flags & CHECKED)
312                   {
313                     puts("MenuItem is CHECKED.");
314                   }
315                   else
316                     puts("MenuItem is NOT CHECKED.");
317                   break;
318
319                 case MN_ABOUT:
320                   puts("Help Menu: About selected");
321                   SetAttrs(window_object, WA_BusyPointer, TRUE, TAG_DONE);
322                   doInfoRequest(NULL, "About cbDemo",
323                   " cbDemo v1.0\n\nfirst ReBuild tutorial Demo.",
324                   "Alright then...", REQIMAGE_INFO);
325                   SetAttrs(window_object, WA_BusyPointer, FALSE, TAG_DONE);
326                   break;
327
328                 default:
329                 puts("Something strange happened - please debug me!");
330               }
331             }
332             break;
333
334             case WMHI_GADGETUP:
335             switch (result & WMHI_GADGETMASK) // WHICH gadget caused the event?
336             {
337               // CheckBox 1:
338               case CB_ENE:
339                 puts("CheckBox 1 was clicked");
340                 // example call with "speaking" index constants:
341                 getCheckBoxStatus(main_window, win_gadgets[GADS_CB_ENE], win_gadgets[GADS_STR_ENE]);
342                 break;
343               // CheckBox 2:
344               case CB_MENE:
345                 puts("CheckBox 2 was clicked");
346                 // example call with "speaking" index constants:
347                 getCheckBoxStatus(main_window, win_gadgets[GADS_CB_MENE], win_gadgets[GADS_STR_MENE]);
348                 break;
349               // CheckBox 3:
350               case CB_MUH:
351                 puts("CheckBox 3 was clicked");
352                 // example call with "speaking" index constants:
353                 getCheckBoxStatus(main_window, win_gadgets[GADS_CB_MUH], win_gadgets[GADS_STR_MUH]);
354                 break;
355               // Button:
356               case BTN_STATUS:
357                 puts("Status Button was clicked - you won't notice anything!");
```

```
                    // example calls, using index numbers
                    getCheckBoxStatus(main_window, win_gadgets[3], win_gadgets[7]);
                    getCheckBoxStatus(main_window, win_gadgets[4], win_gadgets[8]);
                    getCheckBoxStatus(main_window, win_gadgets[5], win_gadgets[9]);
                    break;
                }
                break;
            }
        }
    }
  }
}


/*
* Function: void window_cb
* Purpose:  define window, menus, layout and gadgets
*/
void window_cb( void )
{
  struct NewMenu menuData[] =
  {
    { NM_TITLE, "Action",0,0,0,NULL },
    { NM_ITEM, "Ene","1",CHECKIT|MENUTOGGLE,0,(APTR)MN_ENE },
    { NM_ITEM, "Mene","2",CHECKIT|MENUTOGGLE,0,(APTR)MN_MENE },
    { NM_ITEM, "Muh","3",CHECKIT|MENUTOGGLE,0,(APTR)MN_MUH },
    { NM_ITEM, NM_BARLABEL,0,0,0,NULL },
    { NM_ITEM, "Quit","Q",0,0,(APTR)MN_QUIT },
    { NM_TITLE, "Help",0,0,0,NULL },
    { NM_ITEM, "About...","?",0,0,(APTR)MN_ABOUT },
    { NM_END, NULL, 0, 0, 0, (APTR)0 }
  };

  struct Menu *menu_strip = NULL;
  struct Gadget *main_gadgets[ 13 ];
  int gadget_ids[ 13 ];
  Object *window_object = NULL;
  menu_strip = CreateMenusA( menuData, TAG_END );
  LayoutMenus( menu_strip, gVisinfo,
  GTMN_NewLookMenus, TRUE,
  TAG_DONE );


  window_object = WindowObject,
  WA_Title, "CheckBox Demo",
  WA_ScreenTitle, "CheckBox Demo v1.0",
  WA_Left, 5,
  WA_Top, 20,
  WA_Width, 120,
  WA_Height, 20,
  WA_MinWidth, 120,
  WA_MinHeight, 20,
  WA_MaxWidth, 8192,
  WA_MaxHeight, 20,
  WINDOW_LockHeight, TRUE,
  WINDOW_HintInfo, &hintinfo,
  WINDOW_GadgetHelp, TRUE,
  WINDOW_AppPort, gAppPort,
  WA_CloseGadget, TRUE,
```

```
418     WA_DepthGadget, TRUE,
419     WA_SizeGadget, TRUE,
420     WA_DragBar, TRUE,
421     WA_Activate, TRUE,
422     WA_SizeBBottom, TRUE,
423     WINDOW_Position, WPOS_TOPLEFT,
424     WINDOW_IconTitle, "cbDemo",
425     WA_NoCareRefresh, TRUE,
426     WA_IDCMP, IDCMP_GADGETDOWN | IDCMP_GADGETUP | IDCMP_CLOSEWINDOW | IDCMP_MENUPICK,
427     WINDOW_ParentGroup, VLayoutObject,
428     LAYOUT_SpaceOuter, TRUE,
429     LAYOUT_DeferLayout, TRUE,
430     LAYOUT_AddChild, main_gadgets[0] = LayoutObject,
431     GA_ID, gadget_ids[0] = 5,
432     LAYOUT_Orientation, LAYOUT_ORIENT_VERT,
433     LAYOUT_SpaceOuter, TRUE,
434     LAYOUT_LeftSpacing, 4,
435     LAYOUT_RightSpacing, 4,
436     LAYOUT_TopSpacing, 4,
437     LAYOUT_BottomSpacing, 4,
438     LAYOUT_ShrinkWrap, TRUE,
439     LAYOUT_DeferLayout, TRUE,
440     LAYOUT_AddChild, main_gadgets[1] = LayoutObject,
441     GA_ID, gadget_ids[1] = 27,
442     LAYOUT_Orientation, LAYOUT_ORIENT_HORIZ,
443     LAYOUT_AddChild, main_gadgets[2] = LayoutObject,
444     GA_ID, gadget_ids[2] = 28,
445     LAYOUT_Orientation, LAYOUT_ORIENT_VERT,
446     LAYOUT_BevelStyle, BVS_GROUP,
447     LAYOUT_SpaceOuter, TRUE,
448     LAYOUT_LeftSpacing, 2,
449     LAYOUT_RightSpacing, 2,
450     LAYOUT_TopSpacing, 2,
451     LAYOUT_BottomSpacing, 2,
452     LAYOUT_Label, "CheckBoxes",
453     LAYOUT_DeferLayout, TRUE,
454     LAYOUT_AddChild, main_gadgets[3] = CheckBoxObject,
455     GA_ID, gadget_ids[3] = 29,
456     GA_Text, "_Ene",
457     GA_RelVerify, TRUE,
458     GA_TabCycle, TRUE,
459     GA_Selected, TRUE,
460     CHECKBOX_TextPlace, PLACETEXT_RIGHT,
461     CheckBoxEnd,
462     LAYOUT_AddChild, main_gadgets[4] = CheckBoxObject,
463     GA_ID, gadget_ids[4] = 30,
464     GA_Text, "_Mene",
465     GA_RelVerify, TRUE,
466     GA_TabCycle, TRUE,
467     CHECKBOX_TextPlace, PLACETEXT_RIGHT,
468     CheckBoxEnd,
469     LAYOUT_AddChild, main_gadgets[5] = CheckBoxObject,
470     GA_ID, gadget_ids[5] = 31,
471     GA_Text, "M_uh",
472     GA_RelVerify, TRUE,
473     GA_TabCycle, TRUE,
474     CHECKBOX_TextPlace, PLACETEXT_RIGHT,
475     CheckBoxEnd,
476     LayoutEnd,
477     LAYOUT_AddChild, main_gadgets[6] = LayoutObject,
```

```
478     GA_ID, gadget_ids[6] = 32,
479     LAYOUT_Orientation, LAYOUT_ORIENT_VERT,
480     LAYOUT_BevelStyle, BVS_GROUP,
481     LAYOUT_SpaceOuter, TRUE,
482     LAYOUT_LeftSpacing, 2,
483     LAYOUT_RightSpacing, 2,
484     LAYOUT_TopSpacing, 2,
485     LAYOUT_BottomSpacing, 2,
486     LAYOUT_Label, "Status",
487     LAYOUT_DeferLayout, TRUE,
488     LAYOUT_AddChild, main_gadgets[7] = StringObject,
489     GA_ID, gadget_ids[7] = 33,
490     GA_RelVerify, TRUE,
491     GA_TabCycle, TRUE,
492     GA_ReadOnly, TRUE,
493     STRINGA_TextVal, "checked",
494     STRINGA_MaxChars, 80,
495     STRINGA_MinVisible, 9,
496     StringEnd,
497     CHILD_Label, LabelObject,
498     LABEL_Text, "Ene:",
499     LabelEnd,
500     LAYOUT_AddChild, main_gadgets[8] = StringObject,
501     GA_ID, gadget_ids[8] = 34,
502     GA_RelVerify, TRUE,
503     GA_TabCycle, TRUE,
504     GA_ReadOnly, TRUE,
505     STRINGA_TextVal, "unchecked",
506     STRINGA_MaxChars, 80,
507     STRINGA_MinVisible, 9,
508     StringEnd,
509     CHILD_Label, LabelObject,
510     LABEL_Text, "Mene:",
511     LabelEnd,
512     LAYOUT_AddChild, main_gadgets[9] = StringObject,
513     GA_ID, gadget_ids[9] = 35,
514     GA_RelVerify, TRUE,
515     GA_TabCycle, TRUE,
516     GA_ReadOnly, TRUE,
517     STRINGA_TextVal, "unchecked",
518     STRINGA_MaxChars, 80,
519     STRINGA_MinVisible, 9,
520     StringEnd,
521     CHILD_Label, LabelObject,
522     LABEL_Text, "Muh:",
523     LabelEnd,
524     LayoutEnd,
525     LayoutEnd,
526     LAYOUT_AddChild, main_gadgets[10] = LayoutObject,
527     GA_ID, gadget_ids[10] = 36,
528     LAYOUT_Orientation, LAYOUT_ORIENT_VERT,
529     LAYOUT_SpaceOuter, TRUE,
530     LAYOUT_LeftSpacing, 6,
531     LAYOUT_RightSpacing, 6,
532     LAYOUT_TopSpacing, 6,
533     LAYOUT_BottomSpacing, 1,
534     LAYOUT_AddChild, main_gadgets[11] = ButtonObject,
535     GA_ID, gadget_ids[11] = 37,
536     GA_Text, "_Get Status",
537     GA_RelVerify, TRUE,
```

```
538     GA_TabCycle, TRUE,
539     BUTTON_TextPen, 1,
540     BUTTON_BackgroundPen, 0,
541     BUTTON_FillTextPen, 1,
542     BUTTON_FillPen, 3,
543     ButtonEnd,
544     LayoutEnd,
545     LayoutEnd,
546     LayoutEnd,
547     WindowEnd;
548     main_gadgets[12] = 0;
549     gadget_ids[12] = 0;
550
551     //runWindow( window_object, 3, menu_strip, main_gadgets, gadget_ids );
552     runWindow( window_object, 3, menu_strip, main_gadgets );
553
554     if ( window_object ) DisposeObject( window_object );
555     if ( menu_strip ) FreeMenus( menu_strip );
556   }
557
558
559   /*
560   * Function: int main( ... )
561   * Purpose:  manage flow
562   */
563   int main( int argc, char **argv )
564   {
565     if ( setup() )
566     {
567       window_cb();
568     }
569     cleanup();
570   }
```

H.2. Full Listing - myDemo

myDemo-final.c

```
/*
  Project:    ReBuild Tutorial
  File:       mydemo.c
  Purpose:    enhancing basics
  Author:     M. Bergmann
*/
#include <clib/macros.h>
#include <clib/alib_protos.h>
#include <clib/compiler-specific.h>

#include <proto/exec.h>
#include <proto/dos.h>
#include <proto/utility.h>
#include <proto/graphics.h>
#include <proto/intuition.h>
#include <proto/gadtools.h>
#include <proto/icon.h>

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include <proto/window.h>
#include <proto/layout.h>
#include <proto/button.h>
#include <proto/requester.h>    // NEW
#include <classes/requester.h>  // NEW

#include <libraries/gadtools.h>
#include <reaction/reaction.h>
#include <intuition/gadgetclass.h>
#include <reaction/reaction_macros.h>
#include <classes/window.h>
#include <exec/memory.h>

/* "speaking" name IDs */
#define BTN_BUTTON_1    7 // for first Button ID's value, see ReBuild's main window
#define BTN_BUTTON_2    9 // for second Button ID's value, see ReBuild's main window

/* -- "speaking" Menu identifiers -- */
enum menus { // Menu Project
  MN_ABOUT, MN_CHECKME,
  MN_TOGGLE_ONE, MN_TOGGLE_TWO, MN_TOGGLE_THREE,
  MN_QUIT,
};

/* -- GadgetHelp -- */
struct HintInfo hintinfo[] =
{
  { BTN_BUTTON_1, -1, "I am Button 1, here to \nbore you with informations...", 0L},
  { BTN_BUTTON_2, -1, "I'm Button 1's younger brother. \nWhat he does is good.", 0L},
  { -1, -1, NULL, 0L}   // terminate!
};

// Function prototypes:
void window_3( void );
int doInfoRequest(struct Screen *screen, const char *title,
```

```
58  const char *body, const char *buttons, ULONG image );  // NEW
59
60  // Globals variables:
61  struct Screen *gScreen = NULL;
62  struct DrawInfo *gDrawInfo = NULL;
63  APTR gVisinfo = NULL;
64  struct MsgPort  *gAppPort = NULL;
65
66  struct Library *WindowBase = NULL,
67  *ButtonBase = NULL,
68  *RequesterBase = NULL,    // NEW
69  *GadToolsBase = NULL,
70  *LayoutBase = NULL,
71  *IconBase = NULL;
72  struct IntuitionBase *IntuitionBase = NULL;
73
74  /* copy doInfoRequest() function HERE!
75  *
76  *  int doInfoRequest()
77  *  - creates a requester.class INFO type Requester
78  *  - returns ID of pressed Buttons
79  */
80  int doInfoRequest(struct Screen *screen, const char *title,
81                    const char *body, const char *buttons, ULONG image )
82  {
83    Object *req = 0;     // the requester itself
84    int button;      // the button that was clicked by the user
85
86    // fill in the requester structure
87    req = NewObject(REQUESTER_GetClass(), NULL,
88    REQ_Type,         REQTYPE_INFO,
89    REQ_TitleText,    (ULONG)title,
90    REQ_BodyText,     (ULONG)body,
91    REQ_GadgetText,   (ULONG) buttons ,
92    REQ_Image,        image,
93    TAG_DONE);
94
95    if (req)
96    {
97      struct orRequest reqmsg;
98
99      reqmsg.MethodID  = RM_OPENREQ;
100     reqmsg.or_Attrs  = NULL;
101     reqmsg.or_Window = NULL;
102     reqmsg.or_Screen = screen;
103
104     button = DoMethodA(req, (Msg) &reqmsg);
105     DisposeObject(req);
106
107     return button;
108
109   }  else printf("[request] Failed to allocate requester\n");
110
111   return 0;
112 }
113
114 /*
115 *  int setup(void)
116 *  - open libraries, initialize screen and window-handling
117 *  - create a message port for IDCMP events
```

```
118  */
119  int setup( void )
120  {
121    if( !(IntuitionBase = (struct IntuitionBase*) OpenLibrary("intuition.library",0L)) ) return 0;
122    if( !(GadToolsBase = (struct Library*) OpenLibrary("gadtools.library",0L) ) ) return 0;
123    if( !(WindowBase = (struct Library*) OpenLibrary("window.class",0L) ) ) return 0;
124    if( !(IconBase = (struct Library*) OpenLibrary("icon.library",0L) ) ) return 0;
125    if( !(LayoutBase = (struct Library*) OpenLibrary("gadgets/layout.gadget",0L) ) ) return 0;
126    if( !(ButtonBase = (struct Library*) OpenLibrary("gadgets/button.gadget",0L) ) ) return 0;
127    if( !(RequesterBase = (struct Library*) OpenLibrary("requester.class",0L) ) ) return 0;
128    if( !(gScreen = LockPubScreen( 0 ) ) ) return 0;
129    if( !(gVisinfo = GetVisualInfo( gScreen, TAG_DONE ) ) ) return 0;
130    if( !(gDrawInfo = GetScreenDrawInfo ( gScreen ) ) ) return 0;
131    if( !(gAppPort = CreateMsgPort() ) ) return 0;
132
133    return -1;
134  }
135
136
137  /*
138  *  void cleanup(void)
139  *  - close libraries
140  *  - free window resources
141  */
142  void cleanup( void )
143  {
144    if ( gDrawInfo ) FreeScreenDrawInfo( gScreen, gDrawInfo);
145    if ( gVisinfo ) FreeVisualInfo( gVisinfo );
146    if ( gAppPort ) DeleteMsgPort( gAppPort );
147    if ( gScreen ) UnlockPubScreen( 0, gScreen );
148
149    if (GadToolsBase) CloseLibrary( (struct Library *)GadToolsBase );
150    if (IconBase) CloseLibrary( (struct Library *)IconBase );
151    if (IntuitionBase) CloseLibrary( (struct Library *)IntuitionBase );
152    if (RequesterBase) CloseLibrary( (struct Library *)RequesterBase );    // NEW
153    if (ButtonBase) CloseLibrary( (struct Library *)ButtonBase );
154    if (LayoutBase) CloseLibrary( (struct Library *)LayoutBase );
155    if (WindowBase) CloseLibrary( (struct Library *)WindowBase );
156  }
157
158
159  /*
160  *  void runWindow (parameters...)
161  *  - run a certain window
162  *  - provide an IDCMP message handling loop
163  */
164  void runWindow( Object *window_object, int window_id, struct Menu *menu_strip,
165  struct Gadget *win_gadgets[], int gadget_ids[] )
166  {
167    struct Window *main_window = NULL;
168    // we need a MenuItem struct to keep
169    // track on status informations:
170    struct MenuItem *menuitem = NULL; // NEW: initialize MenuItem structure
171
172    if ( window_object )
173    {
174      if ( main_window = (struct Window *) RA_OpenWindow( window_object ))
175      {
176        WORD Code;
177        ULONG wait = 0, signal = 0, result = 0, done = FALSE;
```

```
178         ULONG req_result;
179
180         GetAttr( WINDOW_SigMask, window_object, &signal );
181         if ( menu_strip)
182           SetMenuStrip( main_window, menu_strip );
183
184         /* —— Event Loop —— */
185         while ( !done)
186         {
187           wait = Wait( signal | SIGBREAKF_CTRL_C );
188
189           if ( wait & SIGBREAKF_CTRL_C )
190           done = TRUE;
191           else
192           while (( result = RA_HandleInput( window_object, &Code )) != WMHI_LASTMSG)
193           {
194             switch ( result & WMHI_CLASSMASK )  // What kind of event occured?
195             {
196               case WMHI_CLOSEWINDOW:
197                 SetAttrs(window_object, WA_BusyPointer, TRUE, TAG_DONE);
198                 req_result = doInfoRequest(NULL, "Quit myDemo",
199                                            " Do you really want to end this disaster?" \
200                                            "\n Select 'Quit' to leave me alone!",
201                                            "_Quit|_Cancel", REQIMAGE_QUESTION);
202                 printf("result was: %ld\n", req_result);
203                 if (req_result == 1)
204                   done = TRUE;
205                 SetAttrs(window_object, WA_BusyPointer, FALSE, TAG_DONE);
206                 break;
207
208               /* —— if user selected a menu entry... ——*/
209               case WMHI_MENUPICK:
210                 puts("\nYou selected some menu:");
211
212                 menuitem = ItemAddress(menu_strip, result & WMHI_MENUMASK);
213                 printf("DEBUG: ItemAdress = %ld\n", menuitem);
214
215                 if(!(menuitem == 0))
216                 {
217                   /* get menu entries by using GadTools' GTMENUITEM_USERDATA field macro */
218                   switch ((long)GTMENUITEM_USERDATA(ItemAddress(menu_strip,
219                                                     result & WMHI_MENUMASK)))
220                   {
221                     case MN_ABOUT:
222                       printf("DEBUG: ItemAdress = %ld\n", menuitem);
223                       puts("About");
224                       SetAttrs(window_object, WA_BusyPointer, TRUE, TAG_DONE);
225                       doInfoRequest(NULL, "About myDemo",
226                       " cbDemo v1.o\n\nsecond ReBuild tutorial Demo.",
227                       "Alright then...", REQIMAGE_INFO);
228                       SetAttrs(window_object, WA_BusyPointer, FALSE, TAG_DONE);
229                       break;
230                     case MN_CHECKME:
231                       printf("DEBUG: ItemAdress = %ld\n", menuitem);
232                       puts("check me");
233                       /* evaluate, if item has checkmark set... */
234                       if(menuitem->Flags & CHECKED)
235                       {
236                         puts("MN_CHECKME is CHECKED!");
237                       }
```

```
238                     else
239                     puts("MN_CHECKME is not CHECKED!.");
240                     break;
241                   case MN_TOGGLE_ONE:
242                   puts("One");
243                   break;
244                   case MN_TOGGLE_TWO:
245                     puts("Two");
246                     break;
247                   case MN_TOGGLE_THREE:
248                     puts("Three");
249                     break;
250                   case MN_QUIT:
251                     // call Requester:
252                     SetAttrs(window_object, WA_BusyPointer, TRUE, TAG_DONE);
253                     req_result =
254                     doInfoRequest(NULL, "Quit myDemo",
255                     " Do you really want to end this disaster?" \
256                     "\n Select 'Quit' to leave me alone!",
257                     "_Quit|_Cancel", REQIMAGE_QUESTION);
258                     printf("result was: %ld\n", req_result);
259                     if (req_result == 1)
260                       done = TRUE;
261                     SetAttrs(window_object, WA_BusyPointer, FALSE, TAG_DONE);
262                     break;
263
264                   default:
265                     puts("Something strange happened - please debug me!");
266                 }
267               }
268               break;
269
270             case WMHI_GADGETUP:
271               puts("gadget press");
272               switch (result & WMHI_GADGETMASK) // WHICH gadget caused the event?
273               {
274                 // Button 1:
275                 case BTN_BUTTON_1:       // Button 1 ID
276                   puts("Button 1 was clicked");
277                   break;
278                 // Button 2:
279                 case BTN_BUTTON_2:       // Button 2 ID
280                   puts("Button 2 was clicked");
281                   break;
282               }
283               break;
284
285             case WMHI_ICONIFY:
286               if ( RA_Iconify( window_object ) )
287               main_window = NULL;
288               break;
289
290             case WMHI_UNICONIFY:
291               main_window = RA_OpenWindow( window_object );
292               if ( menu_strip)  SetMenuStrip( main_window, menu_strip );
293               break;
294
295           }
296         }
297       }
```

```
298         }
299       }
300     }
301
302
303     /*
304     *  void window_xyz(void)
305     *  - defines menues, window and layout
306     *  - calls runWindow()
307     *  - frees ressources before quitting the app
308     */
309     void window_3( void )
310     {
311       struct NewMenu menuData[] =
312       {
313         { NM_TITLE, "Project",0,0,0,NULL },
314         { NM_ITEM, "About..","?",0,0,(APTR)MN_ABOUT },
315         { NM_ITEM, NM_BARLABEL,0,0,0,NULL },
316         { NM_ITEM, "Check me","E",CHECKIT|MENUTOGGLE,0,(APTR)MN_CHECKME },
317         { NM_ITEM, "Toggle me",0,0,0,NULL },
318         { NM_SUB, "One","1",CHECKIT|CHECKED,7 - 1,(APTR)MN_TOGGLE_ONE },
319         { NM_SUB, "Two","2",CHECKIT,7 - 2,(APTR)MN_TOGGLE_TWO },
320         { NM_SUB, "Three","3",CHECKIT,7 - 4,(APTR)MN_TOGGLE_THREE },
321         { NM_ITEM, NM_BARLABEL,0,0,0,NULL },
322         { NM_ITEM, "Quit","Q",0,0,(APTR)MN_QUIT },
323         { NM_END, NULL, 0, 0, 0, (APTR)0 }
324       };
325
326       struct Menu *menu_strip = NULL;
327       struct Gadget *main_gadgets[ 5 ];
328       int gadget_ids[ 5 ];
329       Object *window_object = NULL;
330       menu_strip = CreateMenusA( menuData, TAG_END );
331       LayoutMenus( menu_strip, gVisinfo,
332       GTMN_NewLookMenus, TRUE,
333       TAG_DONE );
334
335
336       window_object = WindowObject,
337       WA_Title, "Demo v1.0",
338       WA_ScreenTitle, "Just a demo...",
339       WA_Left, 5,
340       WA_Top, 20,
341       WA_Width, 200,
342       WA_Height, 25,
343       WA_MinWidth, 200,
344       WA_MinHeight, 25,
345       WA_MaxWidth, 8192,
346       WA_MaxHeight, 25,
347       WINDOW_LockHeight, TRUE,
348       WINDOW_IconifyGadget, TRUE,
349       WINDOW_HintInfo, &hintinfo, // NEW
350       WINDOW_GadgetHelp, TRUE,    // NEW
351       WINDOW_AppPort, gAppPort,
352       WINDOW_IconifyGadget, TRUE,
353       WA_CloseGadget, TRUE,
354       WA_DepthGadget, TRUE,
355       WA_SizeGadget, TRUE,
356       WA_DragBar, TRUE,
357       WA_Activate, TRUE,
```

```
    WA_SizeBBottom, TRUE,
    WINDOW_Position, WPOS_TOPLEFT,
    WINDOW_IconTitle, "myDemo",
    WINDOW_Icon,  GetDiskObject("myDemo"),
    WA_NoCareRefresh, TRUE,
    WA_IDCMP, IDCMP_GADGETDOWN | IDCMP_GADGETUP | IDCMP_CLOSEWINDOW | IDCMP_MENUPICK,
    WINDOW_ParentGroup, VLayoutObject,
    LAYOUT_SpaceOuter, TRUE,
    LAYOUT_DeferLayout, TRUE,
    LAYOUT_AddChild, main_gadgets[0] = LayoutObject,
    GA_ID, gadget_ids[0] = 5,
    LAYOUT_Orientation, LAYOUT_ORIENT_VERT,
    LAYOUT_ShrinkWrap, TRUE,
    LAYOUT_DeferLayout, TRUE,
    LAYOUT_AddChild, main_gadgets[1] = LayoutObject,
    GA_ID, gadget_ids[1] = 6,
    LAYOUT_Orientation, LAYOUT_ORIENT_HORIZ,
    LAYOUT_EvenSize, TRUE,
    LAYOUT_DeferLayout, TRUE,
    LAYOUT_AddChild, main_gadgets[2] = ButtonObject,
    GA_ID, gadget_ids[2] = 7, // identifies first button with ID "7"
    GA_Text, "B_utton 1",
    GA_RelVerify, TRUE,
    GA_TabCycle, TRUE,
    BUTTON_TextPen, 1,
    BUTTON_BackgroundPen, 0,
    BUTTON_FillTextPen, 1,
    BUTTON_FillPen, 3,
    ButtonEnd,
    LAYOUT_AddChild, main_gadgets[3] = ButtonObject,
    GA_ID, gadget_ids[3] = 9,   // identifies second button with ID "9"
    GA_Text, "Butt_on 2",
    GA_RelVerify, TRUE,
    GA_TabCycle, TRUE,
    BUTTON_TextPen, 1,
    BUTTON_BackgroundPen, 0,
    BUTTON_FillTextPen, 1,
    BUTTON_FillPen, 3,
    ButtonEnd,
    LayoutEnd,
    LayoutEnd,
    LayoutEnd,
    WindowEnd;
    main_gadgets[4] = 0;
    gadget_ids[4] = 0;

    //
    runWindow( window_object, 3, menu_strip, main_gadgets, gadget_ids );

    if ( window_object ) DisposeObject( window_object );
    if ( menu_strip ) FreeMenus( menu_strip );
  }


  /* —— main routine —— */
  int main( int argc, char **argv )
  {
    if ( setup() )
    {
      window_3();
```

```
418     }
419     cleanup();
420   }
```